

# STONEGATE SOC

## Microsoft Sentinel Incident Response Simulation

---

A Cloud-Native SOC Build, Detection Engineering &  
Incident Response Portfolio Project

Prepared by

**Mojalefa L. Letsoara**  
**SOC Analyst — SC-200 Candidate**

**Stonegate Retail & Logistics — Security Operations**

**Lab Build Period: 26 - 28 June 2026**

# Table of Contents

---

|                                                                        |    |
|------------------------------------------------------------------------|----|
| Table of Contents .....                                                | 2  |
| Project Objective.....                                                 | 4  |
| Executive Summary.....                                                 | 4  |
| Environment Architecture.....                                          | 5  |
| Governance Tags.....                                                   | 5  |
| Phase 1 — Environment Setup.....                                       | 6  |
| 1.1 Resource Group Creation.....                                       | 6  |
| 1.2 Log Analytics Workspace Deployment .....                           | 8  |
| 1.3 Attaching Microsoft Sentinel .....                                 | 10 |
| 1.4 Built-in Connectors & Initial Ingestion Check .....                | 11 |
| Phase 2 — Diagnostics, Storage & CLI Verification .....                | 13 |
| 2.1 First Cloud Shell Session .....                                    | 13 |
| 2.2 Disabled Diagnostic Settings (Baseline State).....                 | 13 |
| 2.3 Storage Account Provisioning.....                                  | 14 |
| 2.4 Configuring the Diagnostic Setting .....                           | 15 |
| 2.5 Troubleshooting: “No Results Found” .....                          | 17 |
| 2.6 CLI Verification of the Diagnostic Setting.....                    | 17 |
| Phase 3 — Virtual Machine Deployment.....                              | 20 |
| Phase 4 — Data Collection Rule Creation .....                          | 23 |
| Phase 5 — Azure Monitor Agent Installation & Troubleshooting .....     | 25 |
| Phase 6 — VM Access & Audit Policy Configuration .....                 | 27 |
| 6.1 Establishing RDP Access .....                                      | 27 |
| 6.2 Reviewing Existing Event Logs.....                                 | 29 |
| 6.3 Local Security Policy — Before .....                               | 30 |
| 6.4 Local Security Policy — After .....                                | 33 |
| 6.5 A Note on the Legacy Diagnostics Extension .....                   | 36 |
| Phase 7 — Content Hub & Workbook Deployment.....                       | 37 |
| Phase 8 — The DCR Stream Misconfiguration: Diagnosis & Fix.....        | 41 |
| 8.1 Granting Workspace Permissions .....                               | 41 |
| 8.2 Isolating the Fault — Heartbeat Works, SecurityEvent Doesn't ..... | 41 |
| 8.3 Applying the Fix via PowerShell .....                              | 42 |
| 8.4 Final Verification — Both Streams Aligned .....                    | 44 |
| 8.5 Detection Validation — Real Telemetry, Finally .....               | 45 |
| Phase 9 — Analytics Rule Engineering.....                              | 48 |
| 9.1 General Rule Configuration .....                                   | 48 |
| 9.2 MITRE ATT&CK Mapping .....                                         | 48 |
| 9.3 Rule Logic — The Detection Query .....                             | 49 |
| 9.4 The Scheduling Bug — A 5-Minute Window That Was Too Narrow.....    | 49 |
| 9.5 Entity Mapping & Incident Settings .....                           | 50 |

---

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| 9.6 Re-validating the General Tab.....                                     | 51 |
| 9.7 Review + Create — Before and After the Fix.....                        | 52 |
| 9.8 The Rule Goes Live.....                                                | 53 |
| 9.9 The Brute-Force Simulation Succeeds — Account Lockout.....             | 53 |
| Phase 10 — Incident Response & Closure .....                               | 55 |
| 10.1 The Incident Queue.....                                               | 55 |
| 10.2 First-Pass Incident Detail.....                                       | 55 |
| 10.3 The Attack Story — Incident Graph .....                               | 56 |
| 10.4 Triage Step 1 — Assignment & Status .....                             | 56 |
| 10.5 Triage Step 2 — Classification & Analyst Commentary .....             | 57 |
| 10.6 Full Audit Trail .....                                                | 58 |
| 10.7 Triage Step 3 — Resolution.....                                       | 58 |
| 10.8 Scaling Up — Bulk Incident Management.....                            | 59 |
| 10.9 Final State — A Clean, Fully Resolved Queue .....                     | 60 |
| SC-200 Exam Coverage .....                                                 | 61 |
| Lessons Learned & Recommendations .....                                    | 62 |
| Lessons Learned.....                                                       | 62 |
| Recommendations for a Production Deployment.....                           | 62 |
| Analyst Profile & Certification Roadmap .....                              | 63 |
| Appendix — KQL Reference Library .....                                     | 64 |
| A.1 Ingestion & Pipeline Health .....                                      | 64 |
| A.2 Authentication & Brute-Force Detection .....                           | 64 |
| A.3 Privilege Use & Object Access (Identified for Future Detections) ..... | 65 |
| A.4 Azure Activity & Diagnostic Validation.....                            | 65 |

## Project Objective

---

To design, deploy and operate a cloud-native Security Operations Centre (SOC) simulation environment on Microsoft Azure using Microsoft Sentinel as the SIEM platform. This project demonstrates end-to-end SOC analyst capability, from infrastructure provisioning and log pipeline engineering through threat detection and KQL-based investigation, to full incident response by simulating a real-world brute-force attack scenario against a Windows Server endpoint.

The project was undertaken as hands-on preparation for the Microsoft SC-200 (Security Operations Analyst) certification and is intended to demonstrate practical, evidence-backed competency to prospective employers as part of an internship application.

## Executive Summary

---

Stonegate Retail & Logistics identified a need to validate its security monitoring capability ahead of a planned investment in a production SOC deployment. Acting as the project's SOC analyst, I designed and built a fully functional Azure-based SOC lab environment from the ground up, encompassing a Log Analytics workspace, a Microsoft Sentinel SIEM, a Windows Server 2022 endpoint, the Azure Monitor Agent (AMA) and a custom Data Collection Rule (DCR) pipeline.

During the build, a critical ingestion failure was identified and resolved: The Data Collection Rule had been misconfigured to stream Windows Event Logs into the generic Microsoft-Event table rather than the Microsoft-SecurityEvent table that Sentinel's detection engine depends on. This fault was invisible in the Azure Portal UI and was only diagnosed by inspecting the DCR's underlying JSON via Azure Cloud Shell and PowerShell, then corrected programmatically using Update-AzDataCollectionRule.

With the ingestion pipeline operational, a controlled brute-force simulation was executed against the Windows Server endpoint generating repeated failed RDP logon attempts. A custom KQL-based Scheduled Analytics Rule was engineered to detect this pattern, four or more failed logons from the same account/IP within a five-minute window mapped to MITRE ATT&CK technique T1110 (Brute Force) under the Credential Access tactic. Once the rule's lookback window was corrected, it fired continuously, generating 18 incidents over the simulation period.

Each incident was triaged and closed following a structured SOC Level 1 incident response workflow: assignment, status progression to In Progress, analyst commentary documenting the technical findings, classification as a true-positive controlled test and final resolution. The full incident queue was closed using both individual and bulk-incident management workflows in the Microsoft Defender portal.

This project demonstrates practical, hands-on competency across Microsoft Sentinel administration, Azure Monitor Agent and Data Collection Rule configuration, Kusto Query Language (KQL), MITRE ATT&CK threat mapping and SOC Level 1 incident management directly aligned with the Microsoft SC-200 Security Operations Analyst certification syllabus.

## Environment Architecture

The lab environment was built entirely within a single Azure subscription, in the South Africa North region to keep latency low and align with regional data residency expectations for a South African retail business. The table below summarizes the core infrastructure components that together form the SOC telemetry pipeline.

| Component               | Resource Name                                 | Purpose                                            |
|-------------------------|-----------------------------------------------|----------------------------------------------------|
| Resource Group          | rg-stonegate-soc-prod                         | Logical container for all production SOC resources |
| Region                  | South Africa North                            | Primary deployment region for all resources        |
| Log Analytics Workspace | law-stonegate-soc                             | Central data repository for all ingested telemetry |
| SIEM Platform           | Microsoft Sentinel (on law-stonegate-soc)     | Detection, analytics and incident management       |
| Storage Account         | ststonegatesoc01                              | Log archival / diagnostic data retention           |
| Virtual Machine         | soc-lab-vm01 (Windows Server 2022 Datacenter) | Simulated endpoint generating telemetry            |
| Monitoring Agent        | Azure Monitor Agent (AMA) v1.43.0.0           | Collects Windows Event Logs from the endpoint      |
| Data Collection Rule    | dcr-soc-lab                                   | Defines what is collected and where it is sent     |

## Governance Tags

Consistent tagging was applied across the resource group and Log Analytics workspace to support cost tracking, ownership clarity and resource organisation which is a practice expected in any production-grade Azure deployment.

| Tag         | Value                        |
|-------------|------------------------------|
| Project     | SOC-Foundation               |
| Environment | SOC-Lab                      |
| Company     | Stonegate Retail & Logistics |
| Owner       | Mojalefa Letsoara            |
| Workload    | SecurityOperations           |

## Phase 1 — Environment Setup

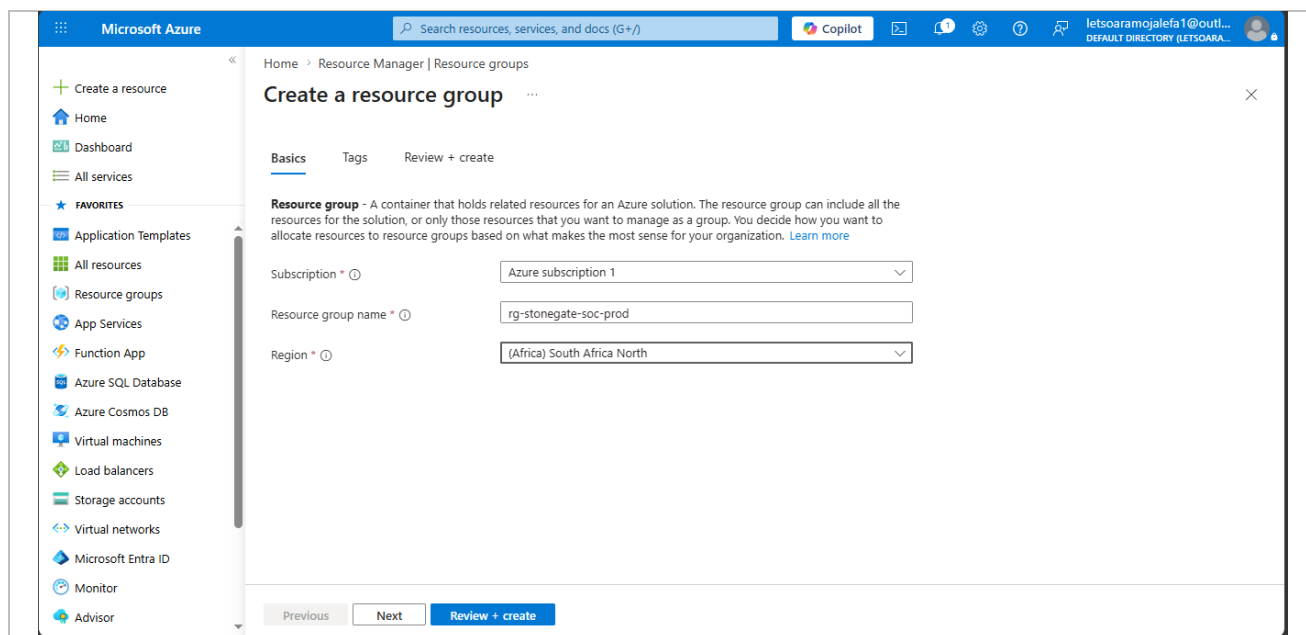
**Goal:** Provision the foundational Azure infrastructure a dedicated resource group and Log Analytics workspace to host all subsequent SOC telemetry, then attach Microsoft Sentinel as the SIEM layer on top of that workspace.

**Tools:** Azure Portal: Resource Manager, Log Analytics Workspaces, Microsoft Sentinel.

**Outcome:** A production-named resource group (rg-stonegate-soc-prod) and Log Analytics workspace (law-stonegate-soc) were deployed in South Africa North with full governance tagging. Microsoft Sentinel was successfully attached, activating its 31-day free trial with seven built-in data connectors automatically connected.

### 1.1 Resource Group Creation

The first step was establishing a dedicated resource group to logically contain every resource related to the SOC build. Isolating the project this way keeps billing, access control and lifecycle management separate from any other workloads in the subscription which is a standard practice before provisioning anything else.



**Figure 1:** Creating the *rg-stonegate-soc-prod* resource group in the South Africa North region.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

letsoaramojalefa1@outl...  
DEFAULT DIRECTORY (LETSOARA...)

Home > Log Analytics workspaces

### Create Log Analytics workspace

Basics Tags Review + Create

Tags are name/value pairs that enable you to categorize resources and view consolidated billing by applying the same tag to multiple resources and resource groups. [Learn more](#)

| Name        | Value                        |
|-------------|------------------------------|
| Environment | SOC-Lab                      |
| Company     | Stonegate Retail & Logistics |
| Project     | SOC-Foundation               |
| Owner       | Mojalefa Letsoara            |
| Workload    | SecurityOperations           |

Review + Create Previous Next: Review + Create >

**Figure 2:** Applying governance tags (Project, Environment, Company, Owner, Workload) to the resource group during creation.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

letsoaramojalefa1@outl...  
DEFAULT DIRECTORY (LETSOARA...)

Home > Resource Manager | Resource groups

### Create a resource group

Basics Tags Review + create

[Automation Link](#)

**Basics**

Subscription Azure subscription 1

Resource group name rg-stonegate-soc-prod

Region South Africa North

**Tags**

Environment SOC-Lab

Company Stonegate Retail & Logistics

Project SOC-Foundation

Owner Mojalefa Letsoara

Workload SecurityOperations

Previous Next Create

**Figure 3:** Final review of the resource group configuration before deployment, confirming subscription, name, region and tags.

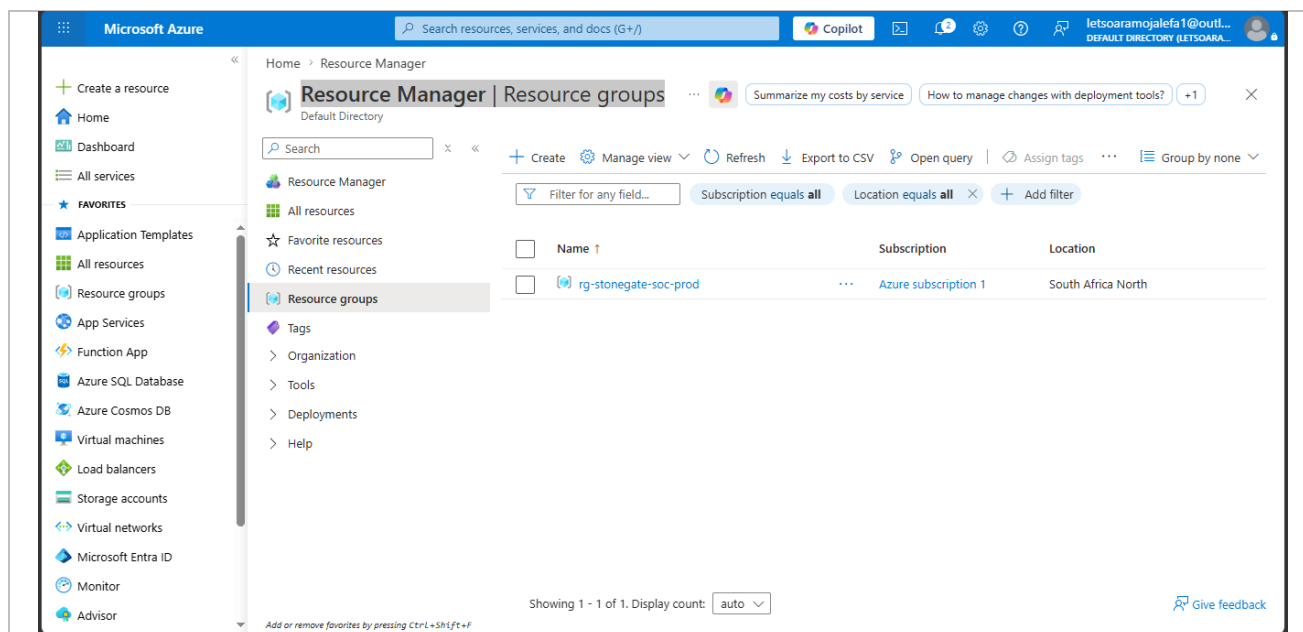


Figure 4: Confirming *rg-stonegate-soc-prod* is live and visible under Resource Manager > Resource Groups.

## 1.2 Log Analytics Workspace Deployment

With the resource group in place, a Log Analytics workspace — *law-stonegate-soc* was created inside it. This workspace becomes the central data lake for every log source in the project: Azure Activity Logs, Windows Security Events, Heartbeat data and ultimately Sentinel's own analytics and incident data.

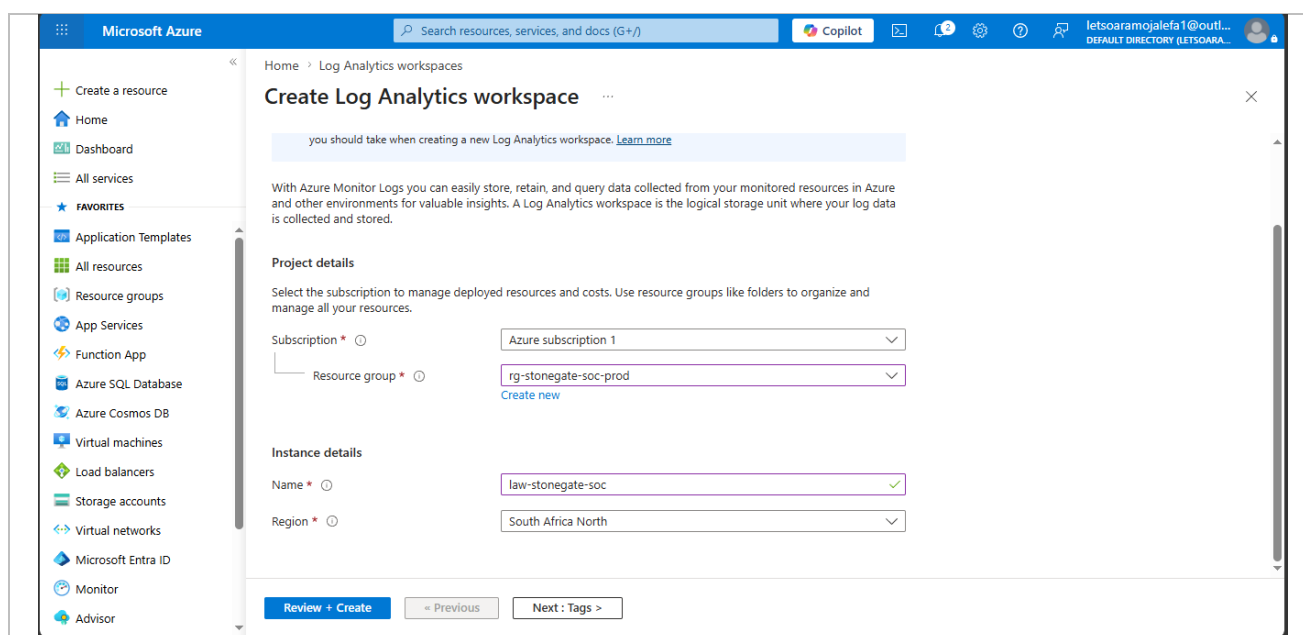


Figure 5: Configuring the new Log Analytics workspace *law-stonegate-soc* inside *rg-stonegate-soc-prod*, South Africa North region.

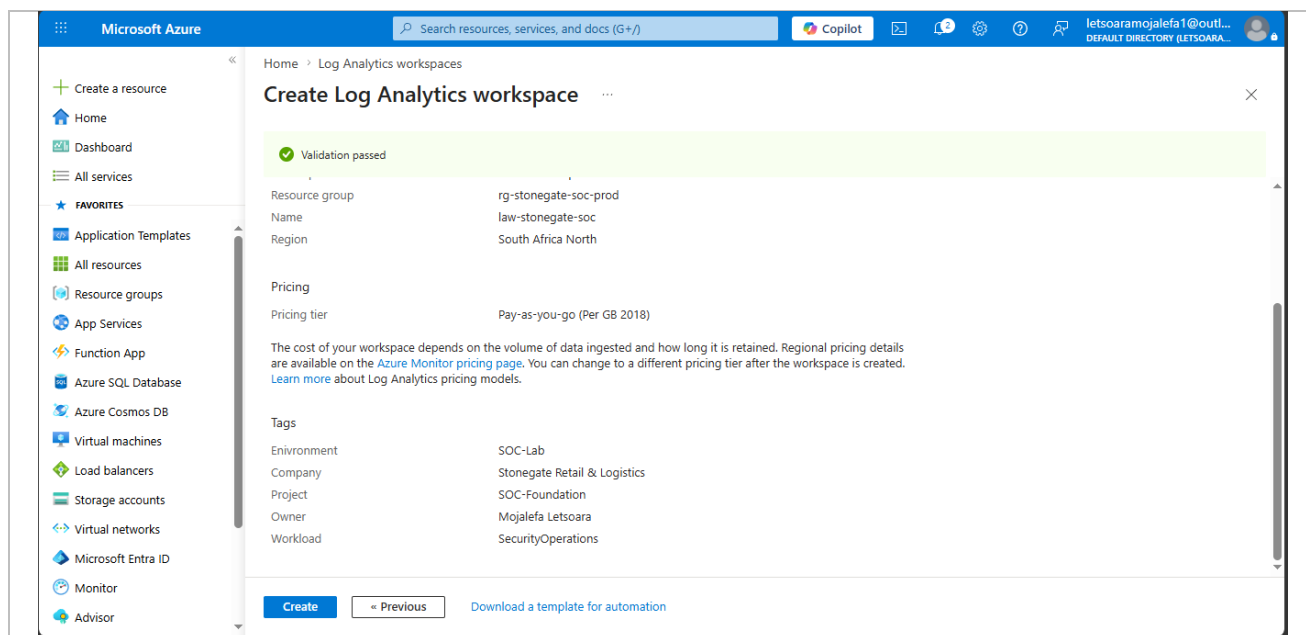


Figure 6: Reviewing the workspace configuration — pay-as-you-go pricing tier, region and inherited tags prior to deployment.

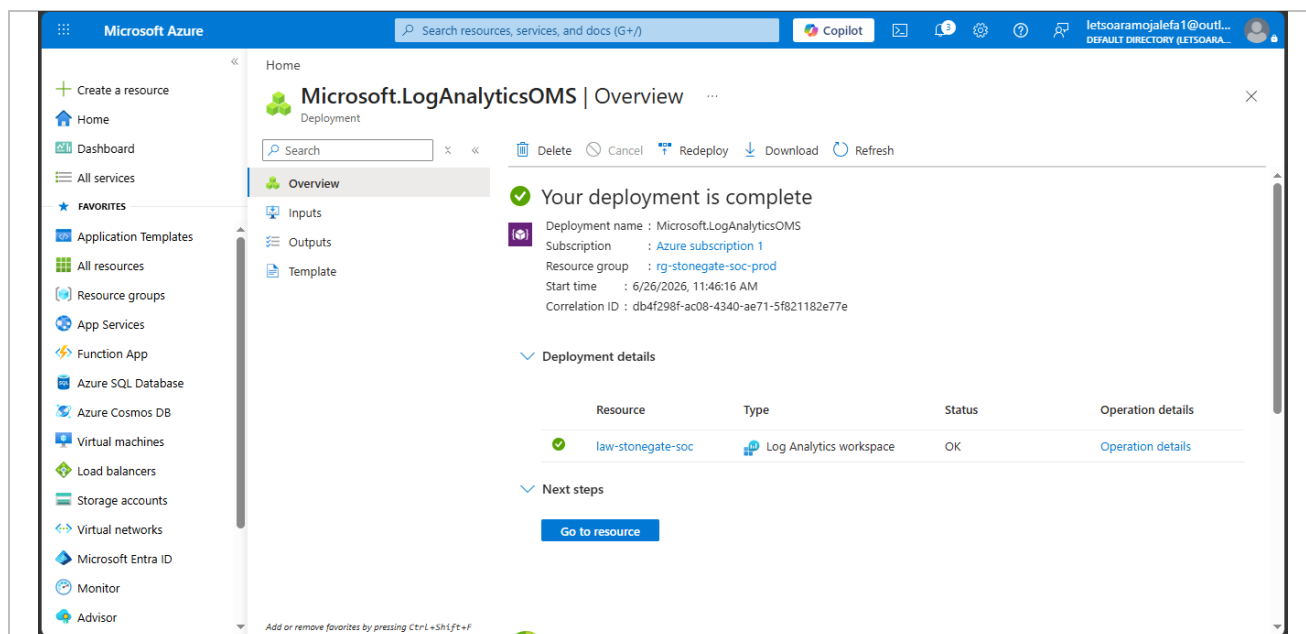
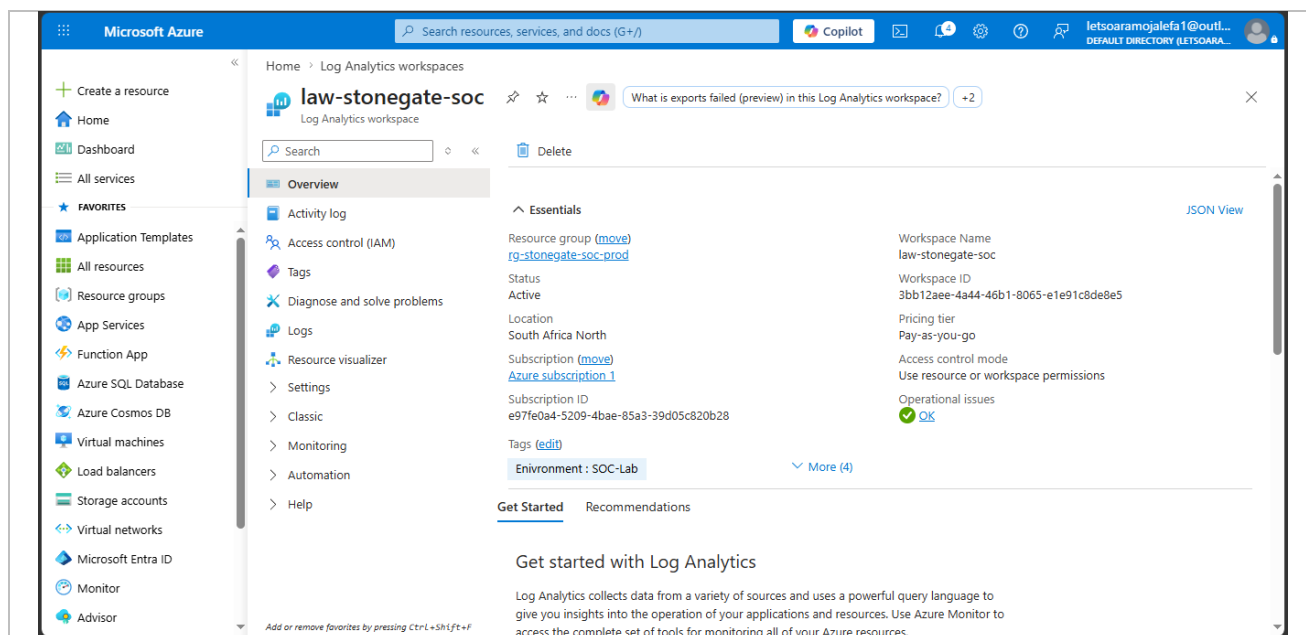


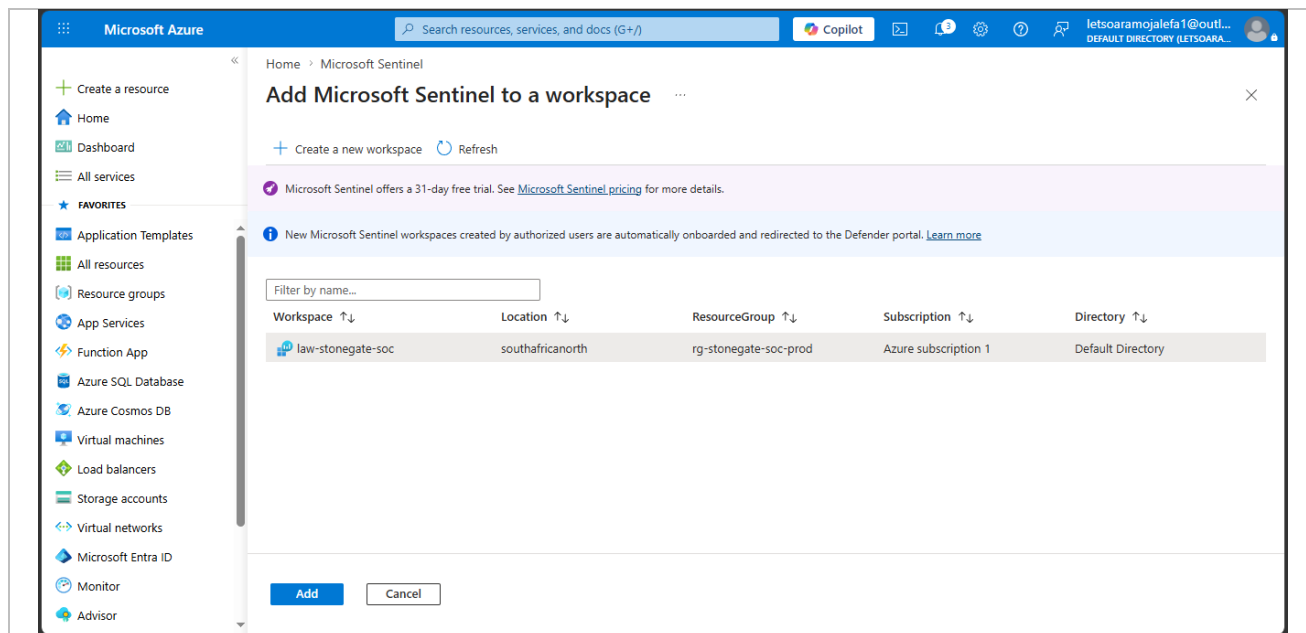
Figure 7: Deployment confirmation: Microsoft.LogAnalyticsOMS deployment completed successfully.



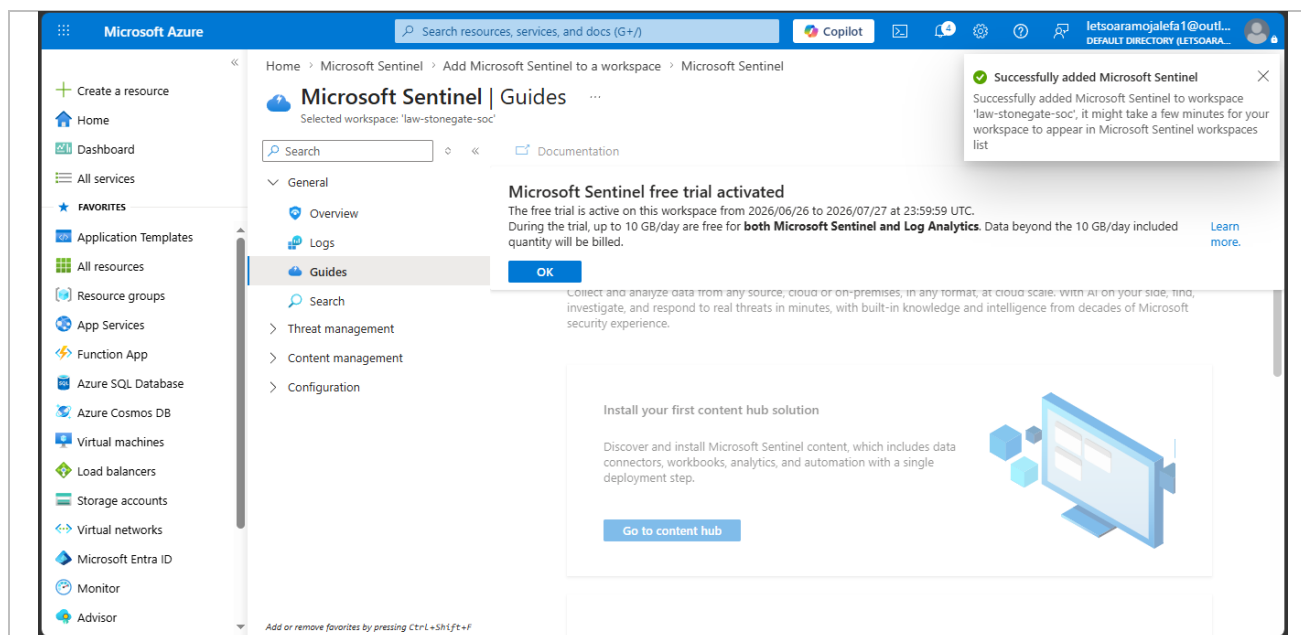
**Figure 8:** The completed law-stonegate-soc workspace overview, confirming Status: Active in South Africa North with Workspace ID and pricing tier visible.

### 1.3 Attaching Microsoft Sentinel

Microsoft Sentinel was then onboarded onto the law-stonegate-soc workspace. Sentinel is not a standalone resource, instead it is a capability layered directly on top of a Log Analytics workspace - so this step converts the plain data store into a fully functional SIEM with analytics rules, incident management and workbook capabilities.



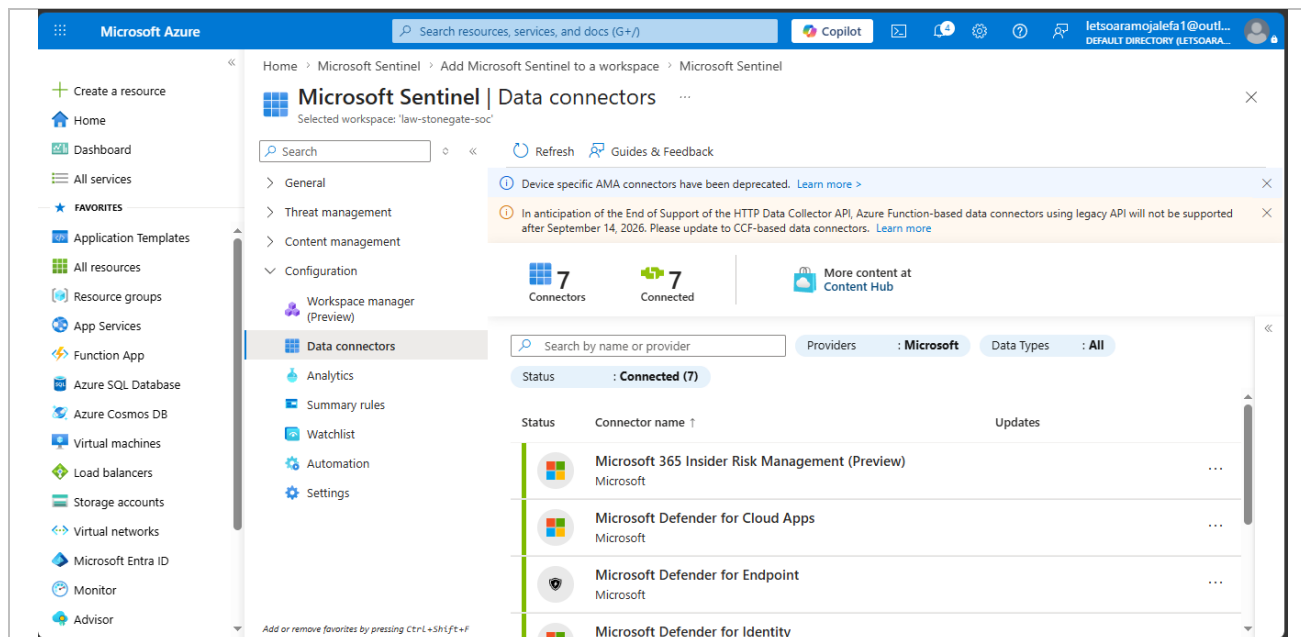
**Figure 9:** Selecting law-stonegate-soc as the target workspace to attach Microsoft Sentinel to.



**Figure 10:** Confirmation banner: “Successfully added Microsoft Sentinel” and the 31-day free trial is active with 10 GB/day of free ingestion for both Sentinel and Log Analytics.

## 1.4 Built-in Connectors & Initial Ingestion Check

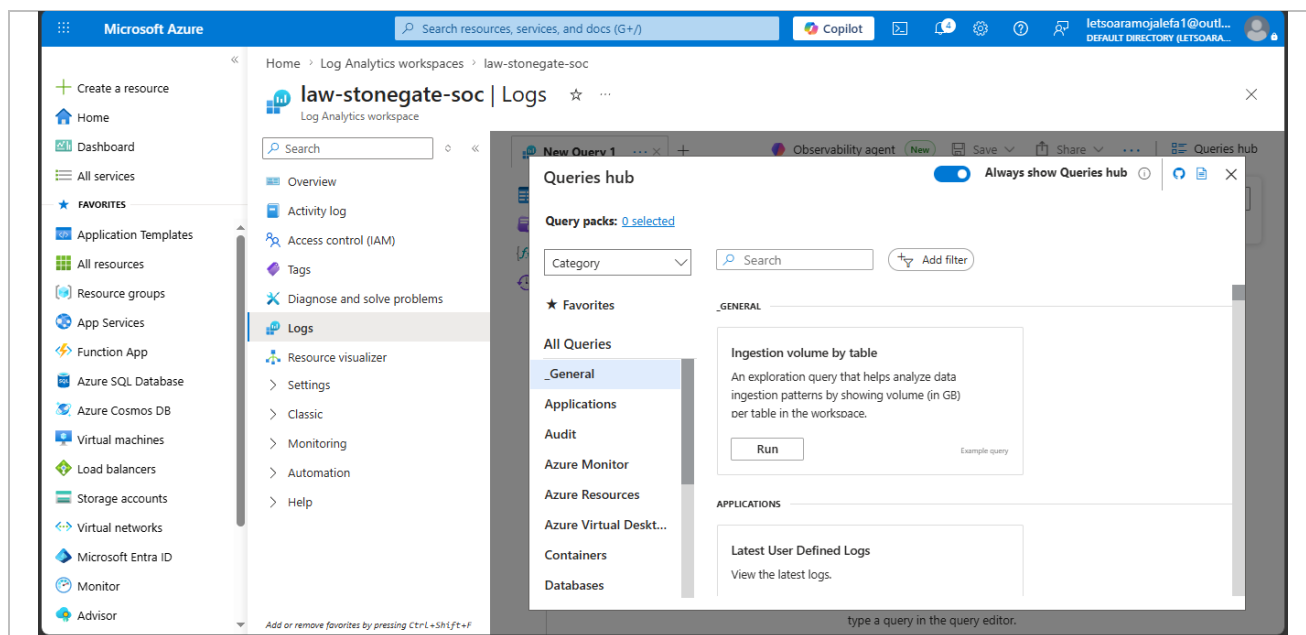
Onboarding Sentinel automatically connected seven built-in Microsoft data connectors (Microsoft 365 Insider Risk Management, Defender for Cloud Apps, Defender for Endpoint, Defender for Identity and others). Before any custom VM telemetry was wired up, a baseline KQL query was run to confirm the query engine itself was operational.



**Figure 11:** Seven built-in data connectors automatically connected immediately after Sentinel onboarding.

### Baseline ingestion check run from Microsoft Sentinel > Logs:

```
search *
| where TimeGenerated > ago(1h)
| summarize Count=count() by $table
```



**Figure 12:** First exploratory query in Sentinel Logs and at this point only LAQueryLogs present, confirming the workspace and query engine are active but no external telemetry sources have been connected yet.

## Phase 2 — Diagnostics, Storage & CLI Verification

**Goal:** Establish a storage account for log archival, configure subscription-level diagnostic settings to stream Azure Activity Logs into the Log Analytics workspace and verify the entire pipeline using both the Azure Portal and Azure CLI via Cloud Shell.

**Tools:** Azure Portal (Storage Accounts, Monitor > Diagnostic Settings), Azure Cloud Shell (PowerShell, az CLI).

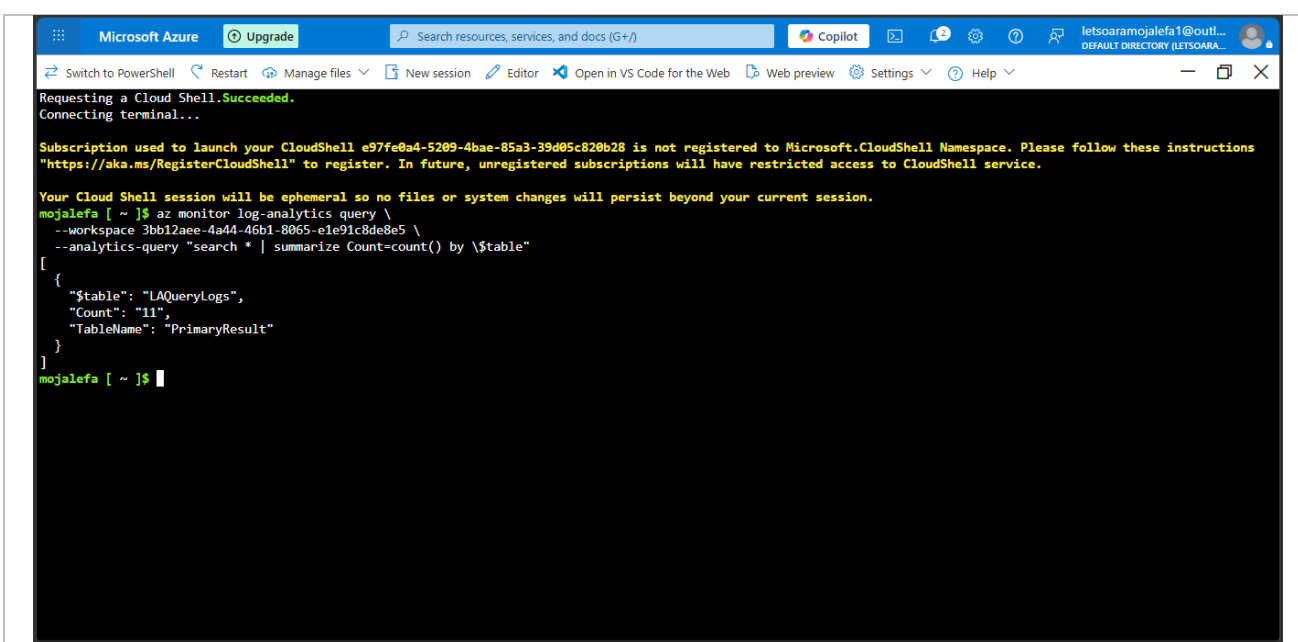
**Outcome:** A storage account (ststonegatesoc01) was deployed for archival and a diagnostic setting (diag-subscription-to-law / soc-activity-log-to-law) was configured to route seven categories of subscription-level logs into law-stonegate-soc. CLI-based verification confirmed the configuration was correctly applied at the JSON level, even before any portal query returned visible results.

### 2.1 First Cloud Shell Session

Before building out the diagnostic pipeline, an initial Azure Cloud Shell session was opened to validate that the Azure CLI could authenticate against the subscription and query the new workspace directly. This is a useful sanity check and the first hands-on use of the `az monitor log-analytics query` command that would later prove essential for diagnosing the Data Collection Rule fault in Phase 8.

**Command run from Cloud Shell:**

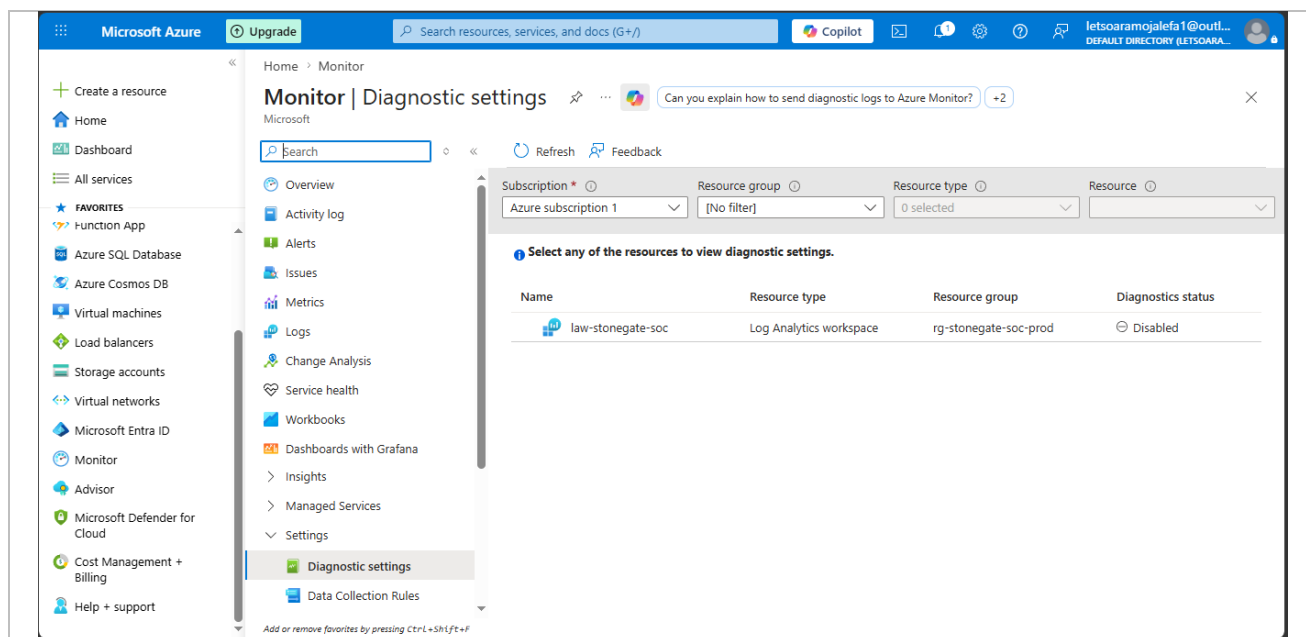
```
az monitor log-analytics query \  
  --workspace 3bb12aee-4a44-46b1-8065-e1e91c8de8e5 \  
  --analytics-query "search * | summarize Count=count() by \table"
```



**Figure 13:** First Cloud Shell session: querying the workspace by GUID confirms LAQueryLogs is the only table present (Count: 11) It is expected since there are no external sources connected yet.

### 2.2 Disabled Diagnostic Settings (Baseline State)

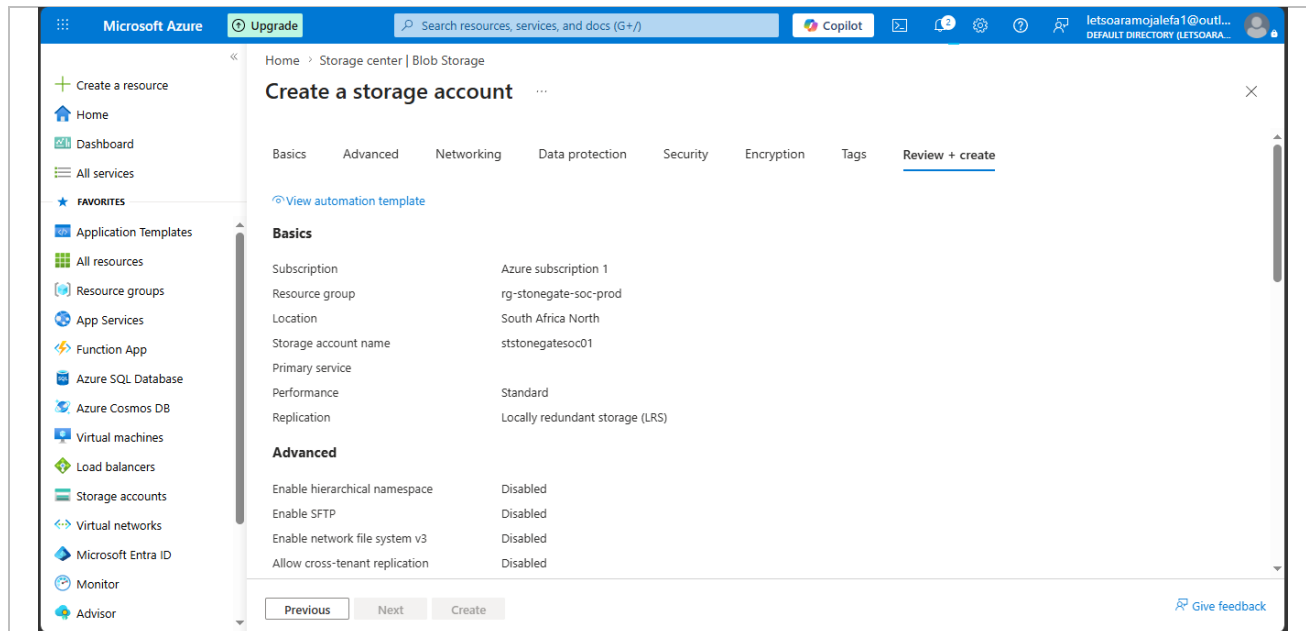
Before any diagnostic setting was created, the Monitor > Diagnostic Settings blade confirmed that no logging pipeline existed yet for the workspace resource itself - status showed as Disabled.



**Figure 14:** Monitor > Diagnostic Settings showing law-stonegate-soc with Diagnostics status: Disabled prior to configuration.

## 2.3 Storage Account Provisioning

A dedicated storage account, ststonegatesoc01 was created in the same resource group and region to support log archival as an optional destination alongside the Log Analytics workspace, following the principle of having a durable, low-cost retention path independent of Sentinel's own ingestion billing.



**Figure 15:** Reviewing the storage account configuration: Standard performance, Locally Redundant Storage (LRS), South Africa North.

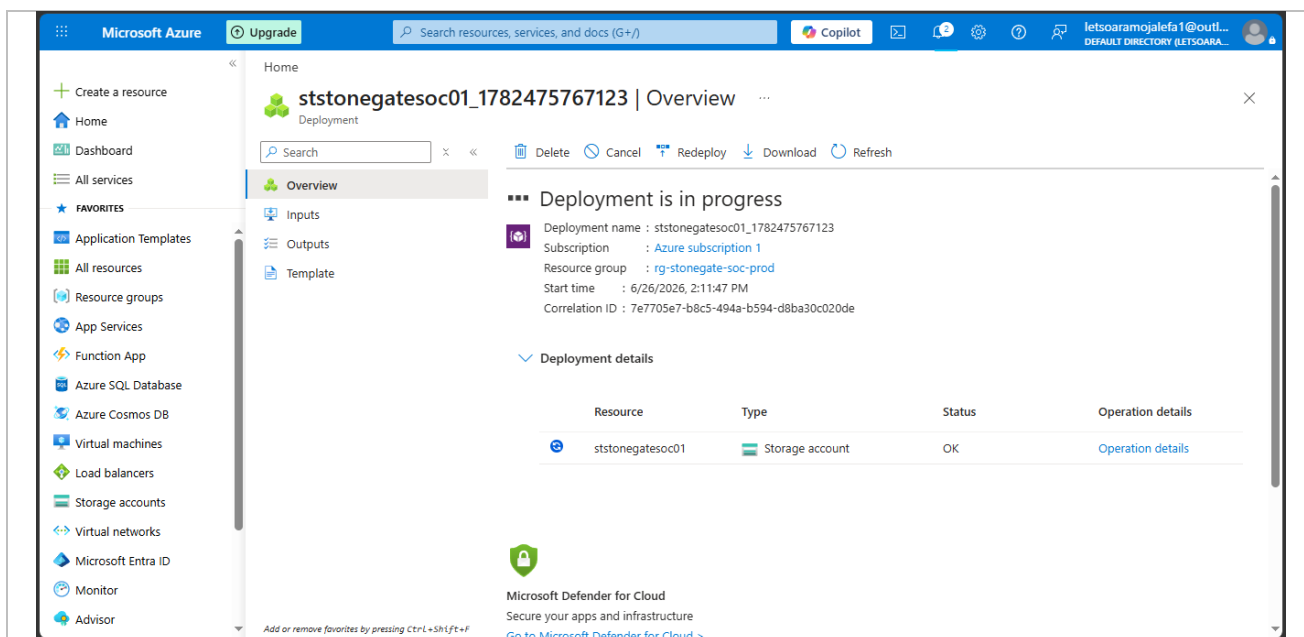


Figure 16: Storage account deployment in progress.

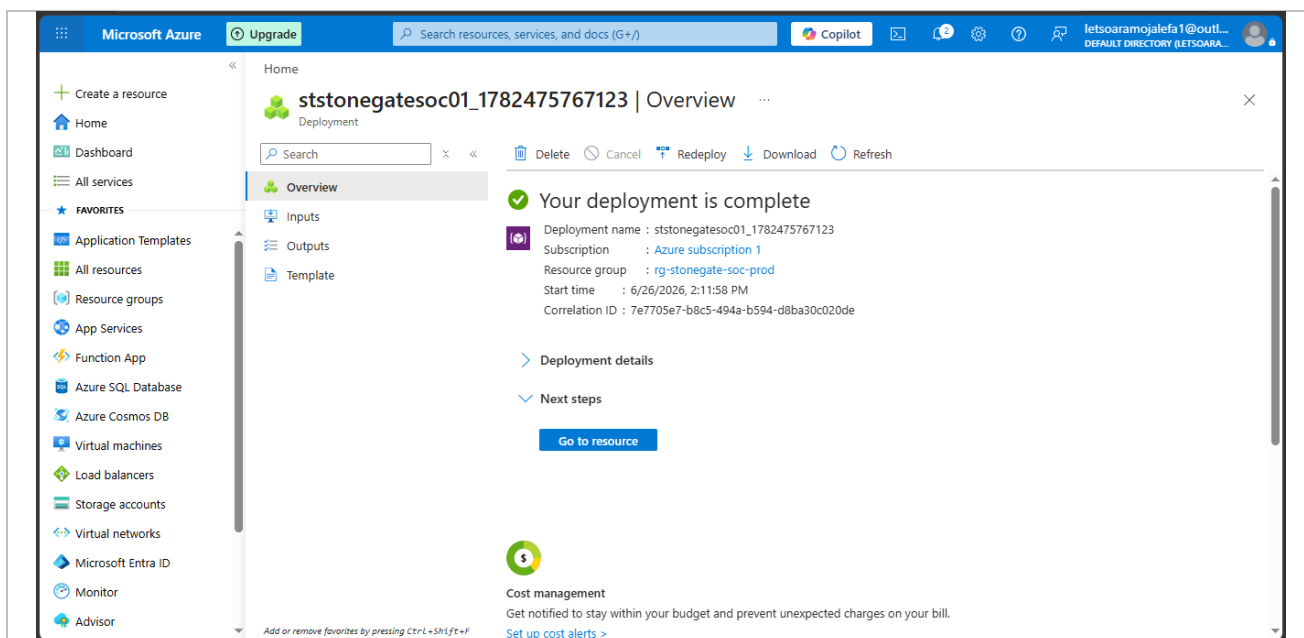
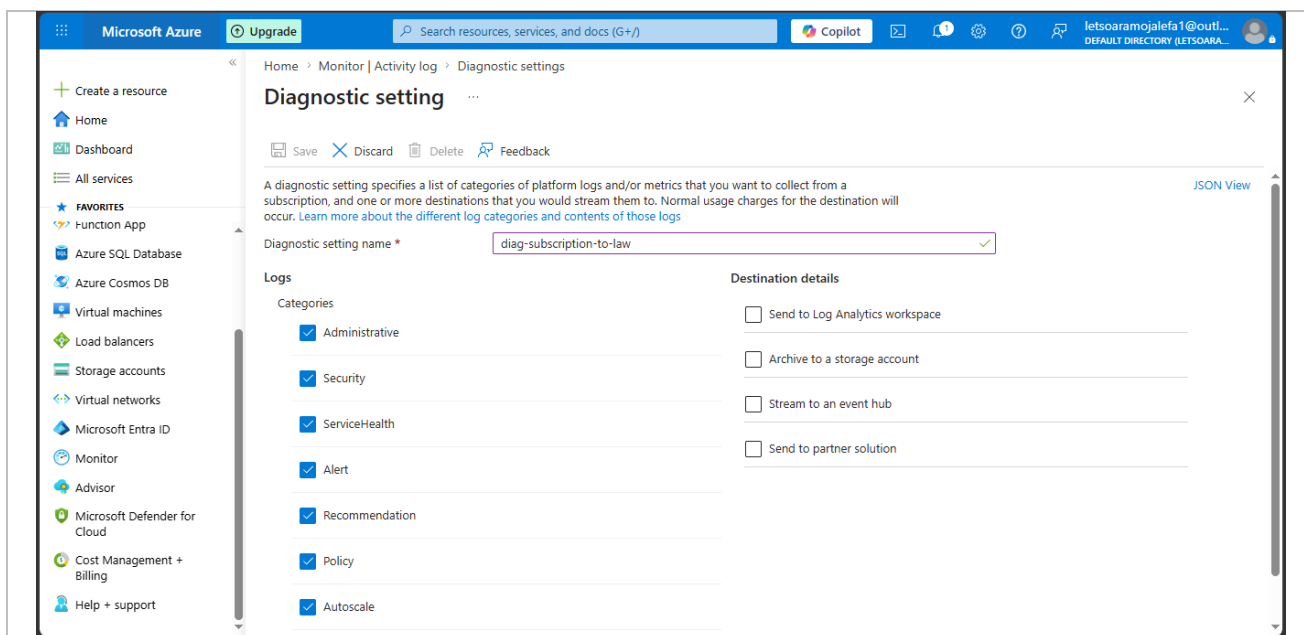


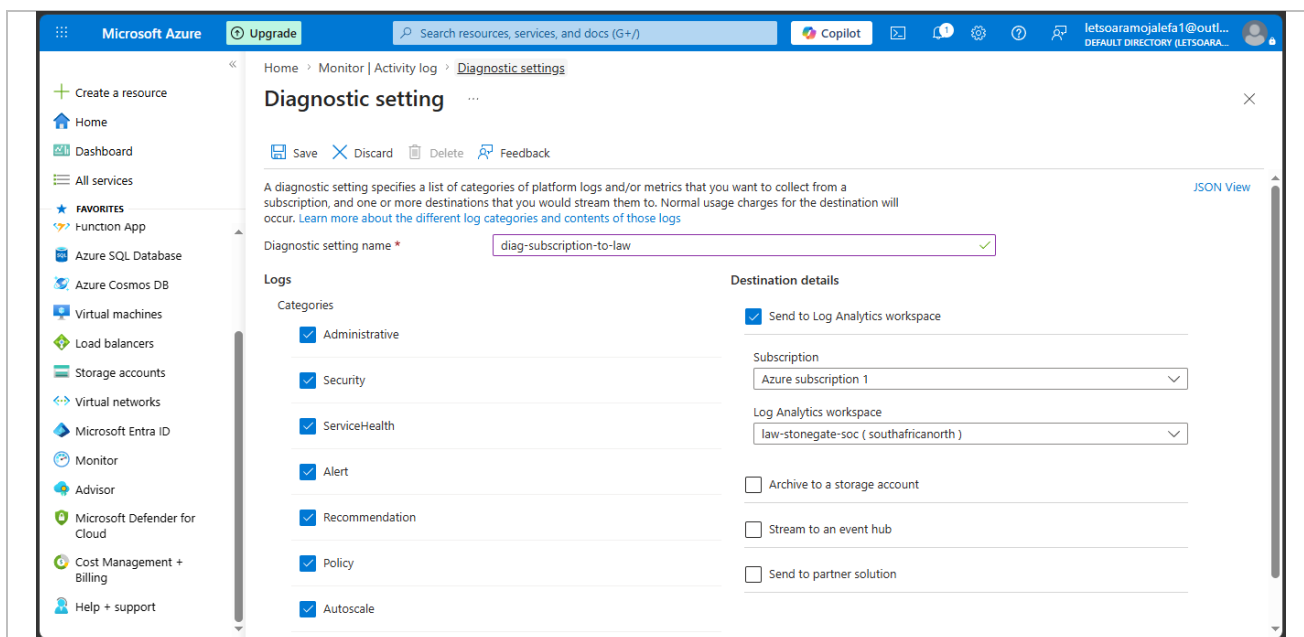
Figure 17: Storage account deployment confirmed complete.

## 2.4 Configuring the Diagnostic Setting

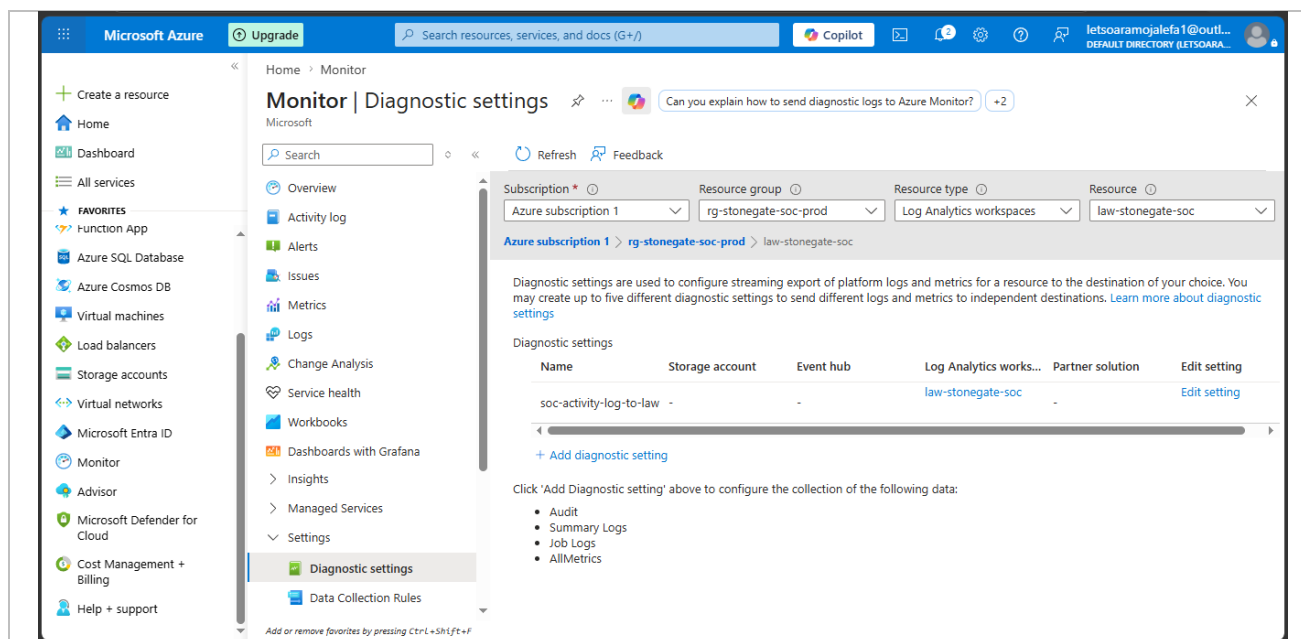
With the workspace and storage account both in place, a subscription-level diagnostic setting was created to stream Azure Activity Log categories - Administrative, Security, ServiceHealth, Alert, Recommendation, Policy and Autoscale into law-stonegate-soc. The first save attempt was made before a destination had been selected; the configuration was then corrected to explicitly target the Log Analytics workspace.



**Figure 18:** Diagnostic setting *diag-subscription-to-law* configured with all seven log categories enabled, but no destination yet selected.



**Figure 19:** Corrected configuration: “Send to Log Analytics workspace” is now checked, targeting *law-stonegate-soc* (South Africa North) under Azure subscription 1.



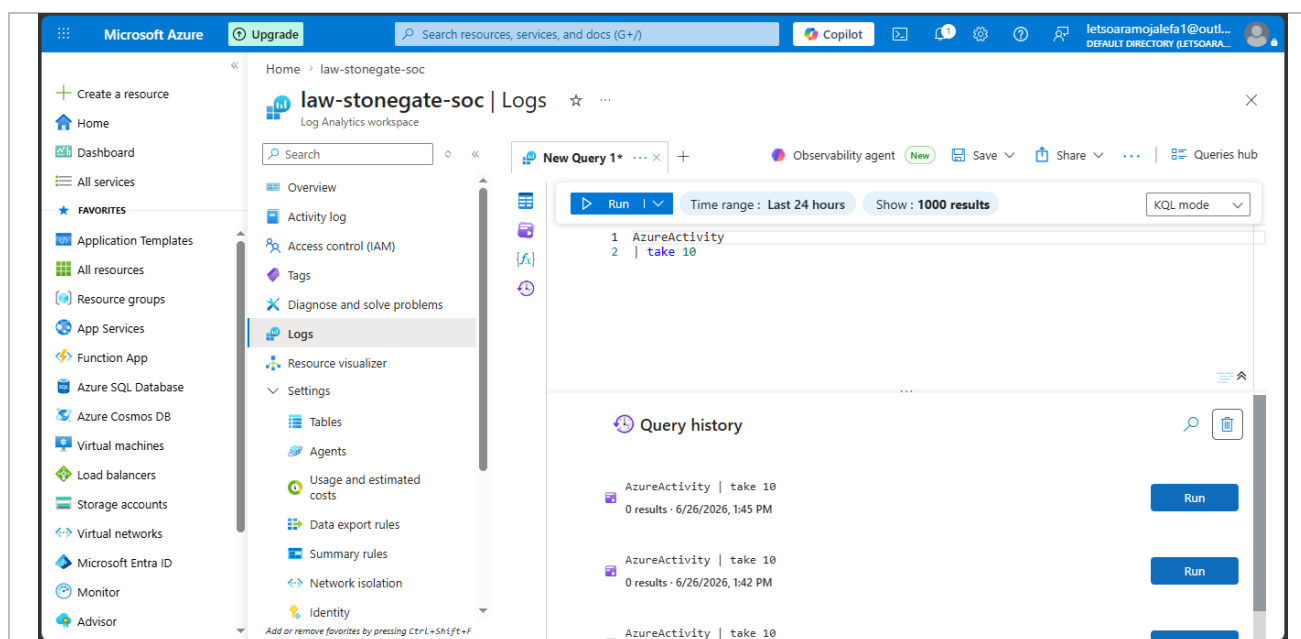
**Figure 20:** Monitor > Diagnostic Settings now showing the saved rule *soc-activity-log-to-law*, with *law-stonegate-soc* listed as the Log Analytics workspace destination.

## 2.5 Troubleshooting: “No Results Found”

Immediately after saving, a query for `AzureActivity` returned zero rows. This is a normal and expected symptom of ingestion delay as diagnostic data typically takes several minutes to begin flowing after a setting is first applied and it prompted a deeper CLI-based verification to rule out a configuration error before simply waiting it out.

### Query attempted in Sentinel Logs:

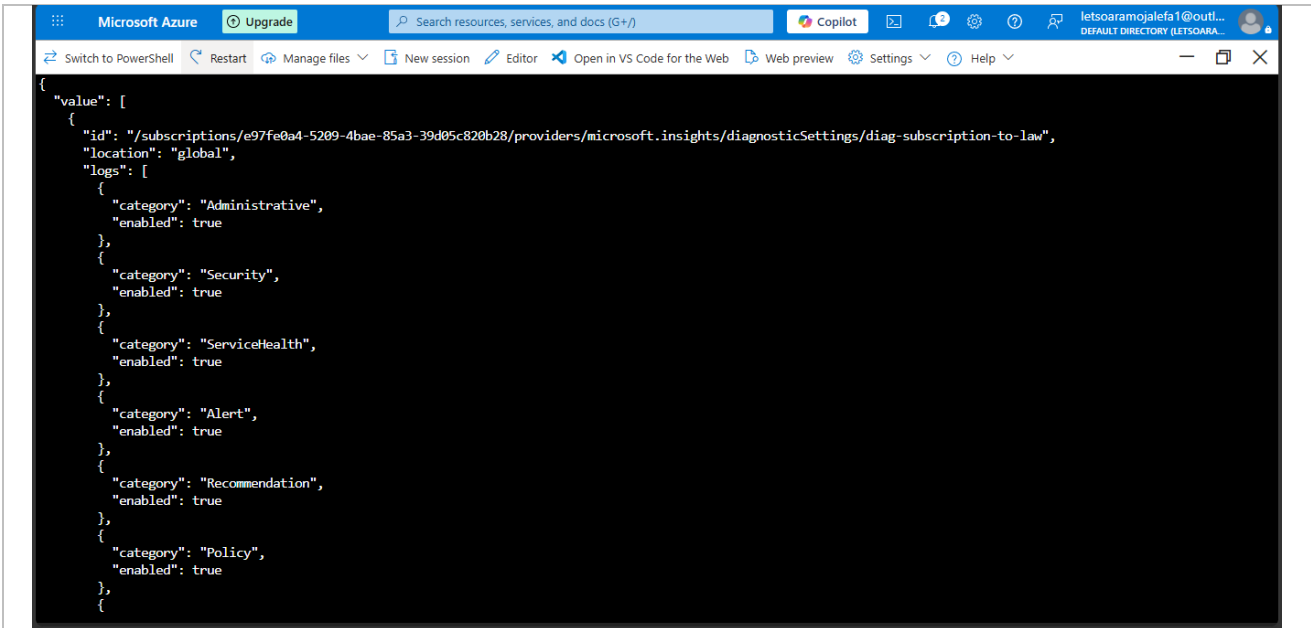
```
AzureActivity
| take 10
```



**Figure 21:** `AzureActivity | take 10` returning zero results immediately after the diagnostic setting was saved, the query history shows repeated 0-result runs.

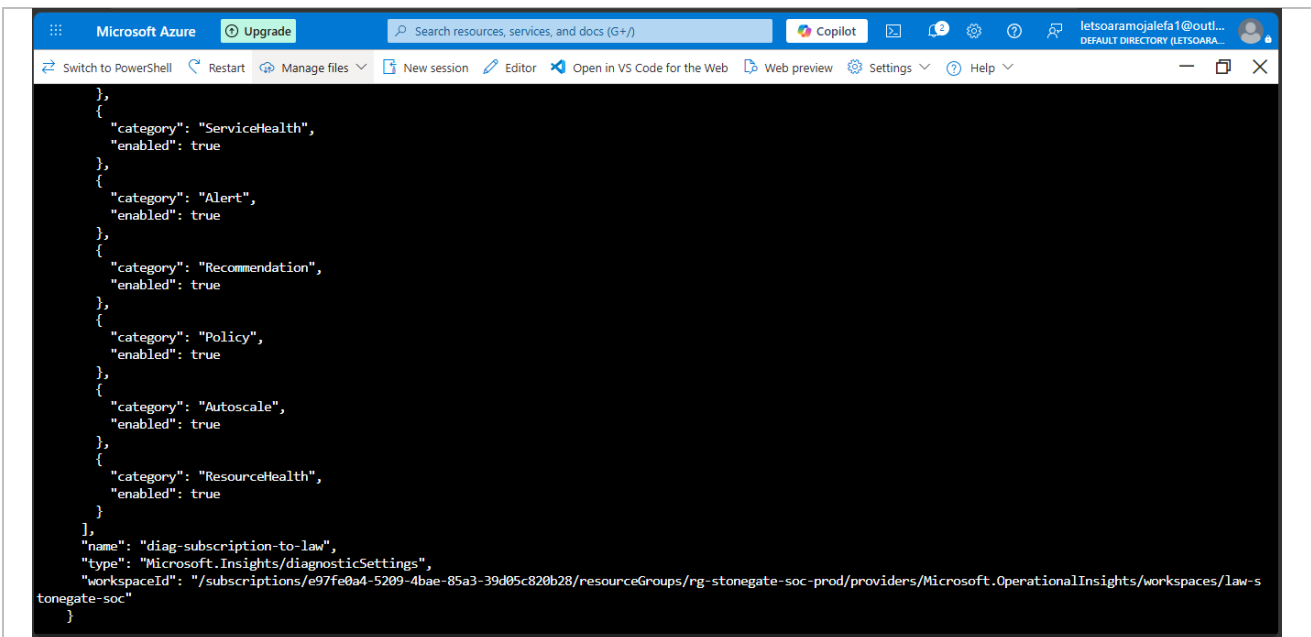
## 2.6 CLI Verification of the Diagnostic Setting

To confirm the configuration itself was correct and independent of ingestion timing, the diagnostic setting was inspected directly via the Azure CLI's JSON output. This confirmed every intended category was enabled and correctly bound to the law-stonegate-soc workspace, isolating the empty query result as a timing issue rather than a misconfiguration.



```
{
  "value": [
    {
      "id": "/subscriptions/e97fe0a4-5209-4bae-85a3-39d05c820b28/providers/microsoft.insights/diagnosticSettings/diag-subscription-to-law",
      "location": "global",
      "logs": [
        {
          "category": "Administrative",
          "enabled": true
        },
        {
          "category": "Security",
          "enabled": true
        },
        {
          "category": "ServiceHealth",
          "enabled": true
        },
        {
          "category": "Alert",
          "enabled": true
        },
        {
          "category": "Recommendation",
          "enabled": true
        },
        {
          "category": "Policy",
          "enabled": true
        },
        {
          "category": "Autoscale",
          "enabled": true
        },
        {
          "category": "ResourceHealth",
          "enabled": true
        }
      ]
    }
  ]
}
```

**Figure 22:** *az monitor diagnostic-settings show output (part 1): Administrative, Security and ServiceHealth categories confirmed enabled: true.*



```
},
{
  "category": "ServiceHealth",
  "enabled": true
},
{
  "category": "Alert",
  "enabled": true
},
{
  "category": "Recommendation",
  "enabled": true
},
{
  "category": "Policy",
  "enabled": true
},
{
  "category": "Autoscale",
  "enabled": true
},
{
  "category": "ResourceHealth",
  "enabled": true
}
],
"name": "diag-subscription-to-law",
"type": "Microsoft.Insights/diagnosticSettings",
"workspaceId": "/subscriptions/e97fe0a4-5209-4bae-85a3-39d05c820b28/resourceGroups/rg-stonegate-soc-prod/providers/Microsoft.OperationalInsights/workspaces/law-stonegate-soc"
}
```

**Figure 23:** *Output continued (part 2): Alert, Recommendation, Policy and Autoscale categories all confirmed enabled.*

```

"category": "Autoscale",
"enabled": true
},
{
"category": "ResourceHealth",
"enabled": true
}
},
"name": "diag-subscription-to-law",
"type": "Microsoft.Insights/diagnosticSettings",
"workspaceId": "/subscriptions/e97fe0a4-5209-4bae-85a3-39d05c820b28/resourceGroups/rg-stonegate-soc-prod/providers/Microsoft.OperationalInsights/workspaces/law-s
tonegate-soc"
}
]
}
}
mojalefa [ ~ ]$ ^C
mojalefa [ ~ ]$ az monitor log-analytics query \
--workspace law-stonegate-soc \
--analytics-query "search * | summarize Count=count() by \table"
The 'cdn' and 'afd' command groups have moved to the 'cdn' CLI extension. Install the latest version with: az extension add --name cdn. The built-in cdn/afd commands i
n azure-cli core will be removed in a future release.
Preview version of extension is disabled by default for extension installation, enabled for modules without stable versions.
Please run 'az config set extension.dynamic_install_allow_preview=true or false' to config it specifically.
The command requires the extension log-analytics. Do you want to install it now? The command will continue to run after the extension is installed. (Y/n): y
Run 'az config set extension.use_dynamic_install=yes_without_prompt' to allow installing extensions without prompt.
No stable version of 'log-analytics' to install. Preview versions allowed.
The installed extension 'log-analytics' is in preview.
(PathNotFoundException) The requested path does not exist
Code: PathNotFoundException
Message: The requested path does not exist
mojalefa [ ~ ]$

```

**Figure 24:** Output continued (part 3): Autoscale and ResourceHealth categories confirmed, workspaceId correctly resolved to law-stonegate-soc. A subsequent attempt to query the workspace by friendly name (law-stonegate-soc) instead of its GUID fails with PathNotFoundException, this is a reminder that the az monitor log-analytics query command requires the workspace GUID, not its display name.

## Phase 3 — Virtual Machine Deployment

**Goal:** Deploy a Windows Server endpoint to act as the simulated corporate workstation, this is the source of all Security, System and Application event telemetry for the remainder of the project.

**Tools:** Azure Portal (Virtual Machines).

**Outcome:** soc-lab-vm01, a Windows Server 2022 Datacenter VM (Standard D2s v3, 2 vCPU / 8 GiB RAM) was deployed in South Africa North with RDP (port 3389) enabled, a new virtual network and a public IP address (40.127.8.19).

A single Windows Server 2022 virtual machine, soc-lab-vm01 was provisioned to serve as the endpoint generating telemetry for the entire detection pipeline. The VM was deliberately deployed into its own auto-generated resource group (soc-lab-vm01\_group\_73077218). This is a default Azure behaviour for VM deployments unless an existing resource group is explicitly reused, which later became directly relevant during AMA agent troubleshooting in Phase 5.

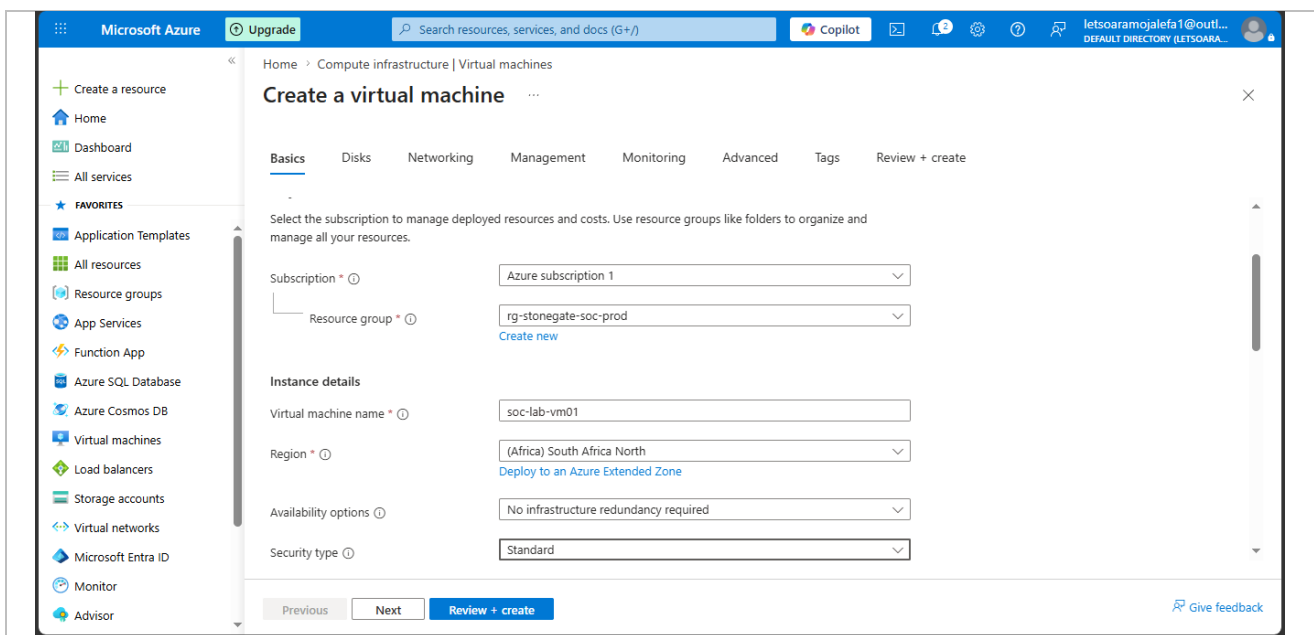
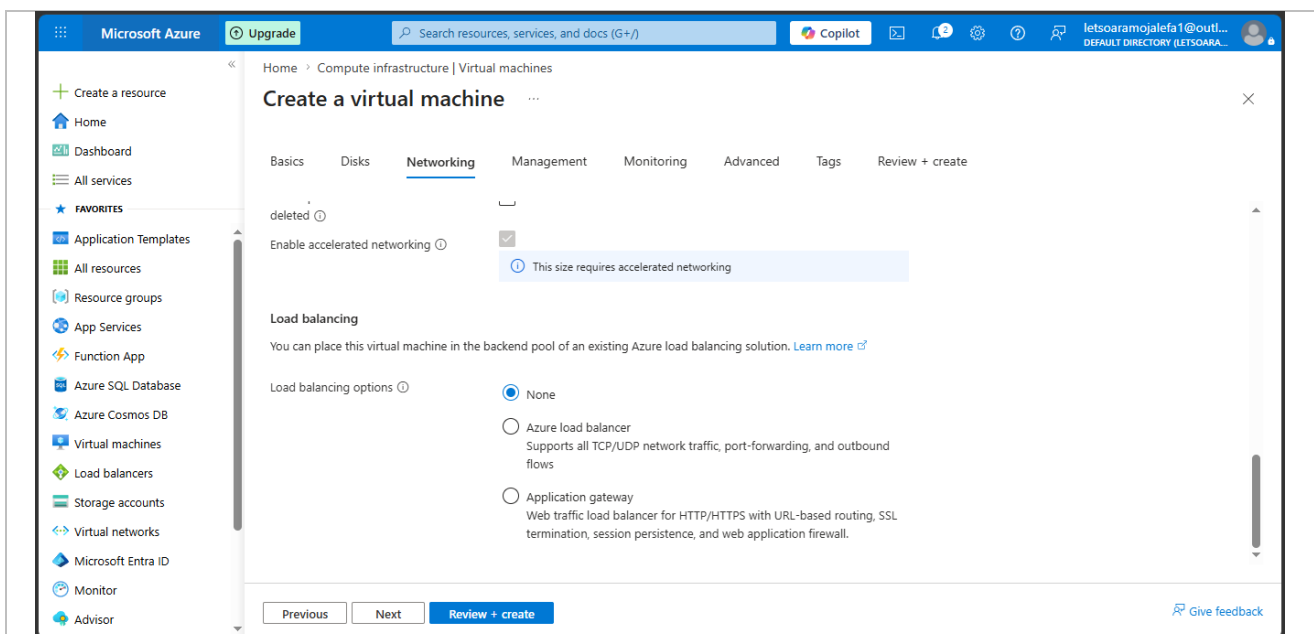
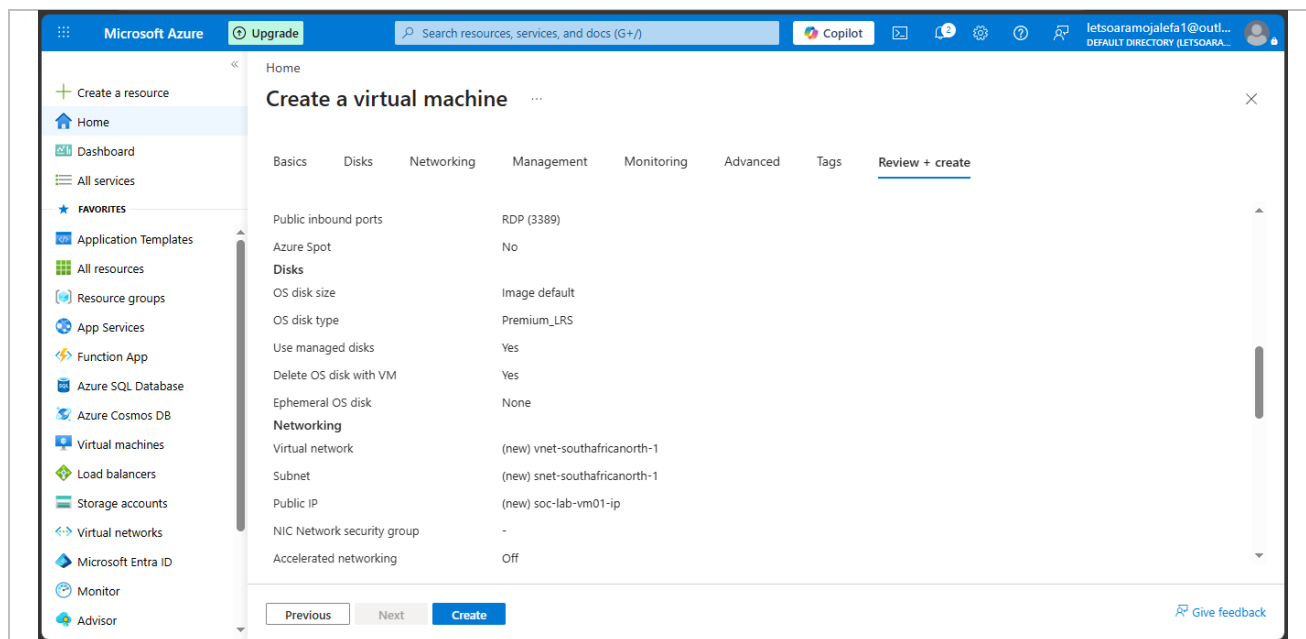


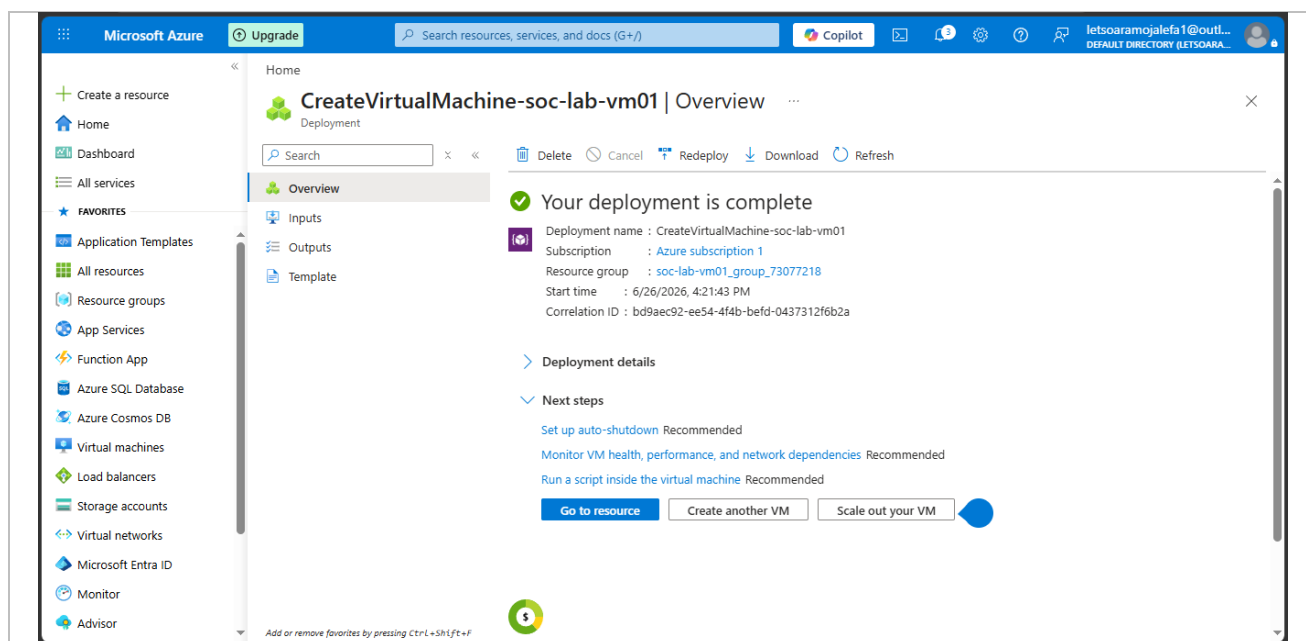
Figure 25: VM basics: name soc-lab-vm01, resource group rg-stonegate-soc-prod, region South Africa North, Standard security type.



**Figure 26:** *Networking tab: no load balancer required for this single-VM lab; accelerated networking enabled by instance size requirement.*



**Figure 27:** *Review + create: RDP (3389) public inbound port, Premium\_LRS OS disk, new virtual network and public IP confirmed.*



**Figure 28:** *Deployment CreateVirtualMachine-soc-lab-vm01 completed successfully.*

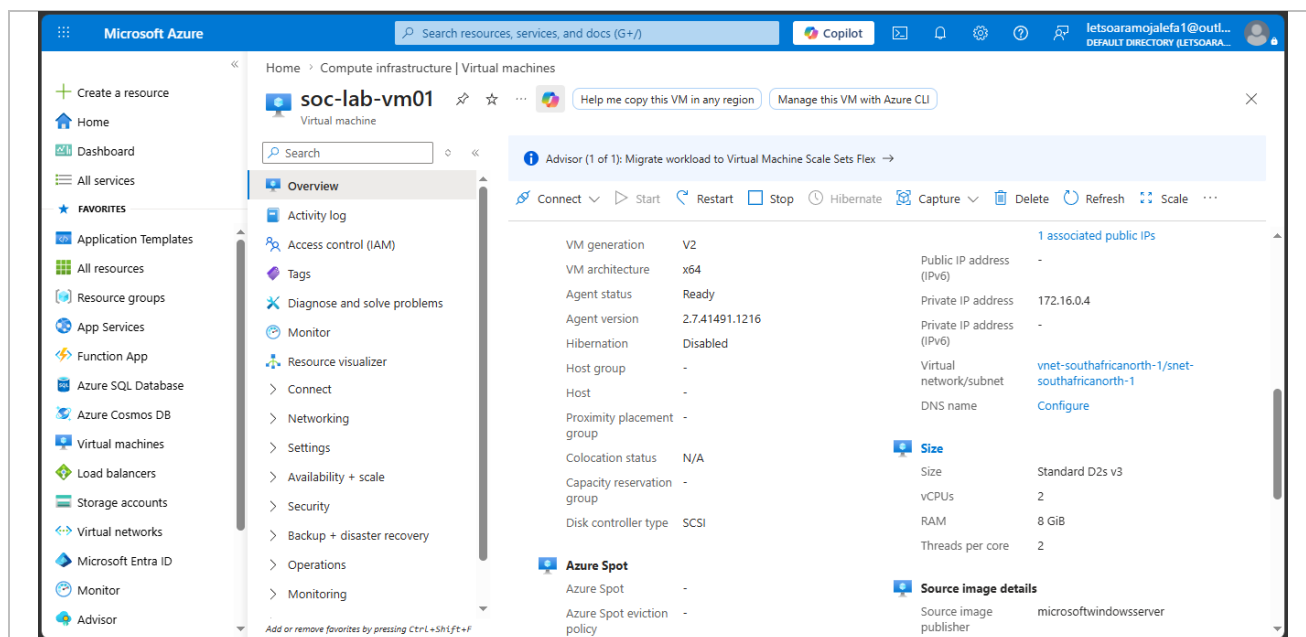


Figure 29: VM Overview confirming Agent status: Ready, Standard D2s v3 size, private IP 172.16.0.4 and one associated public IP.

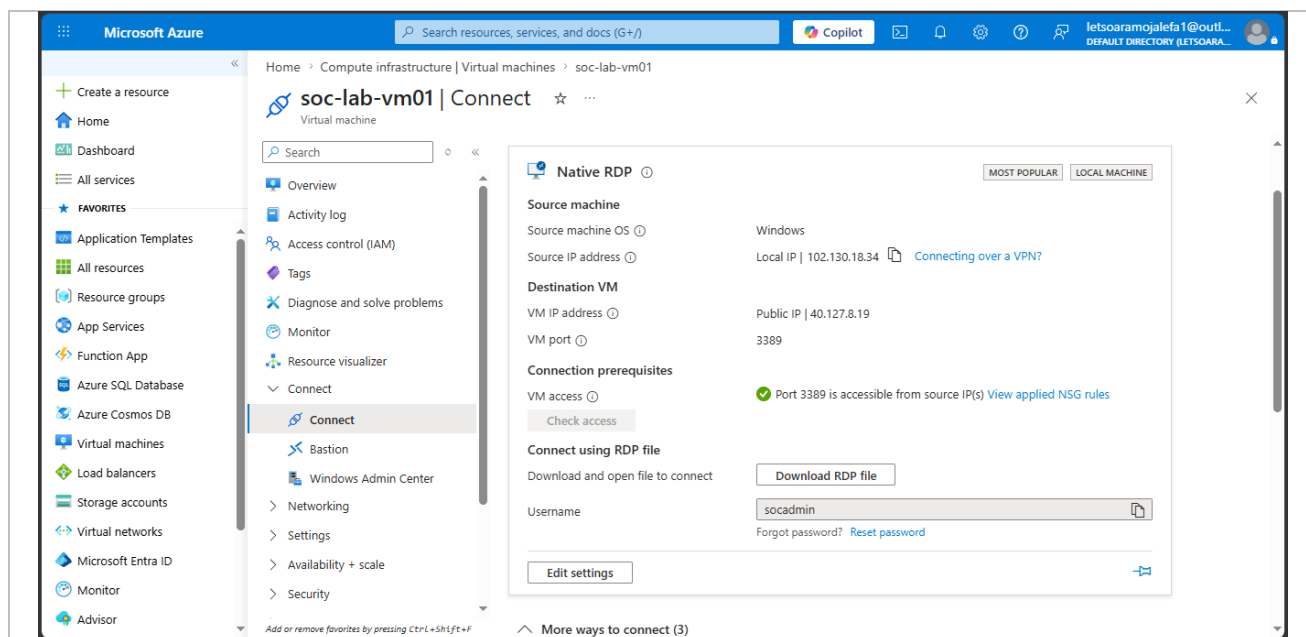


Figure 30: Native RDP connection page confirming port 3389 is accessible from the source IP with username socadmin and public IP 40.127.8.19 ready for connection.

## Phase 4 — Data Collection Rule Creation

**Goal:** Create a Data Collection Rule (DCR) to define which Windows Event Logs are collected from soc-lab-vm01 and where that telemetry should be delivered.

**Tools:** Azure Portal (Monitor > Data Collection Rules).

**Outcome:** dcr-soc-lab was created, associated with soc-lab-vm01 and configured to collect Application, System and Security Windows Event Logs with law-stonegate-soc set as the destination Log Analytics workspace. The deployment succeeded; though, as detailed in Phase 8, a deeper misconfiguration in how the rule routed data was not yet visible at this stage.

A Data Collection Rule (DCR) is the mechanism by which Azure Monitor Agent knows what to collect from a VM and where to send it. dcr-soc-lab was created as an agent-based rule, targeting soc-lab-vm01 directly as its associated resource with Windows Event Logs (Application, System and Security) selected as the data source and law-stonegate-soc set as the Log Analytics workspace destination.

The screenshot displays the 'Create Data Collection Rule' wizard in the Azure Portal. The 'Basics' tab is active, showing the following configuration:

- Rule Name:** dcr-soc-lab
- Subscription:** Azure subscription 1
- Resource Group:** rg-stonegate-soc-prod
- Region:** (Africa) South Africa North
- Type of telemetry:** Agent-based - Windows or Linux (most common)

The 'Review + create' button is highlighted in blue at the bottom right of the form.

**Figure 31:** DCR basics: rule name dcr-soc-lab, resource group rg-stonegate-soc-prod, region South Africa North, telemetry type Agent-based - Windows or Linux.

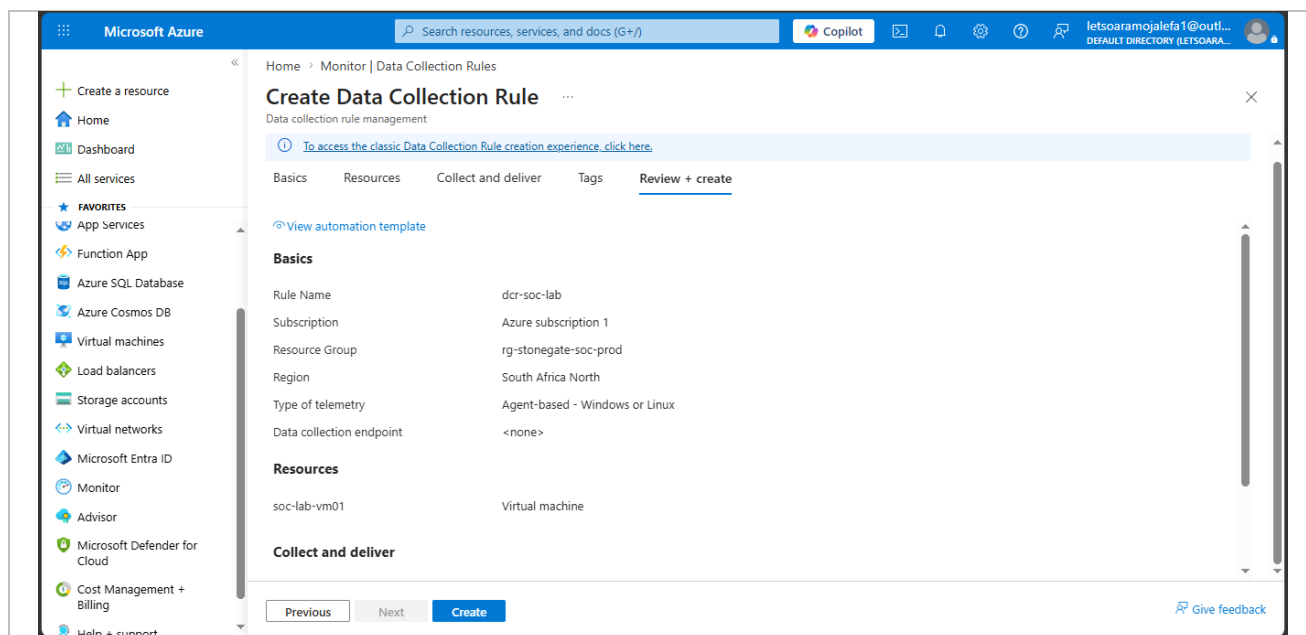


Figure 32: Review + create summary: resource soc-lab-vm01, ready for deployment.

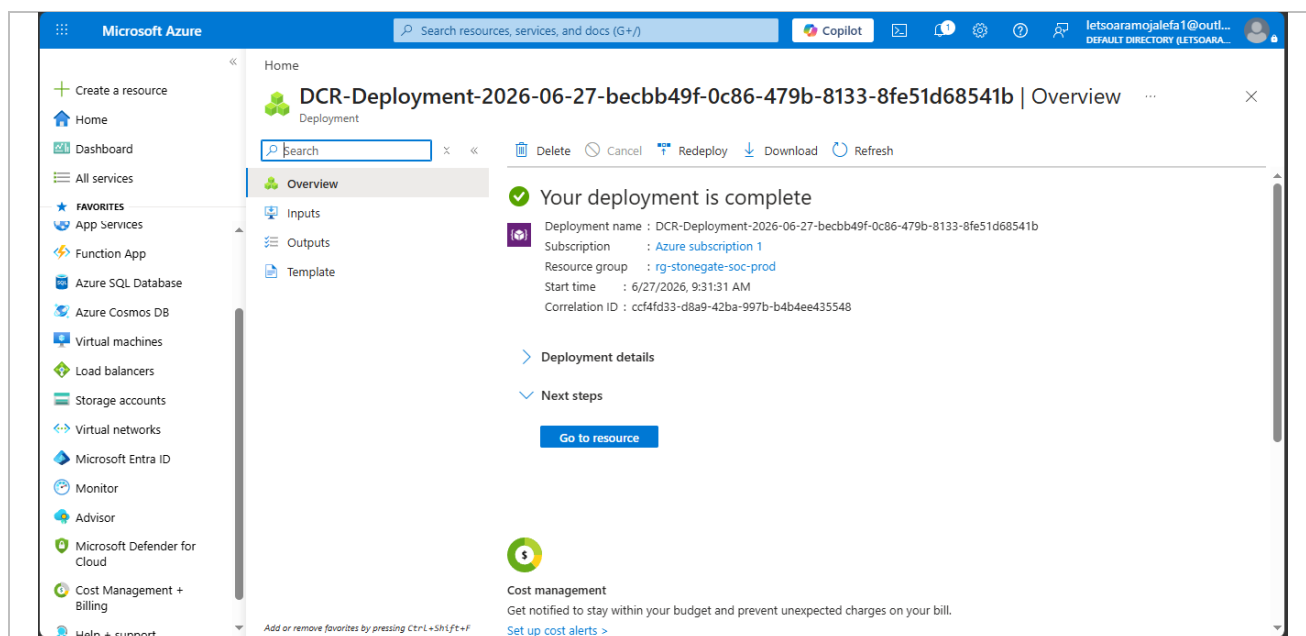


Figure 33: DCR-Deployment confirmed complete - the data collection rule dcr-soc-lab is now live pending agent installation on the target VM.

## Phase 5 — Azure Monitor Agent Installation & Troubleshooting

**Goal:** Install the Azure Monitor Agent (AMA) on soc-lab-vm01 so it can begin collecting and forwarding Windows Event Logs per the DCR and confirm the agent is alive using the Heartbeat table.

**Tools:** Azure Cloud Shell (Azure CLI, az vm extension set), Log Analytics Logs (KQL).

**Outcome:** An initial install attempt failed with ResourceNotFound because the wrong resource group was targeted. After listing VMs to find the correct resource group (auto-generated as SOC-LAB-VM01\_GROUP\_73077218), the AMA extension installed successfully (provisioningState: Succeeded) and the agent began reporting Heartbeat data within minutes.

Installing the Azure Monitor Agent via the portal's Extensions blade did not surface any usable AMA extension in the gallery search, so the install was instead performed directly through the Azure CLI from Cloud Shell. The first attempt failed because the resource group name used (rg-stonegate-soc-prod, which is the project's main resource group) and did not match the resource group the VM had actually been deployed into. This is now a side effect of how Azure auto-generates a dedicated resource group per VM unless an existing one is explicitly selected during creation.

### *First attempt (fails - wrong resource group):*

```
az vm extension set \  
  --publisher Microsoft.Azure.Monitor \  
  --name AzureMonitorWindowsAgent \  
  --vm-name soc-lab-vm01 \  
  --resource-group rg-stonegate-soc-prod
```

(ResourceNotFound) The Resource 'Microsoft.Compute/virtualMachines/soc-lab-vm01' under resource group 'rg-stonegate-soc-prod' was not found.

Rather than guess at the correct resource group name, az vm list -o table was used to list every VM in the subscription alongside its actual resource group, immediately revealing the correct target: SOC-LAB-VM01\_GROUP\_73077218.

### *Corrected command (succeeds):*

```
az vm list -o table
```

| Name         | ResourceGroup               | Location         | Zones |
|--------------|-----------------------------|------------------|-------|
| soc-lab-vm01 | SOC-LAB-VM01_GROUP_73077218 | southafricanorth | 2     |

```
az vm extension set \  
  --publisher Microsoft.Azure.Monitor \  
  --name AzureMonitorWindowsAgent \  
  --vm-name soc-lab-vm01 \  
  --resource-group SOC-LAB-VM01_GROUP_73077218
```

"provisioningState": "Succeeded"

```

mojalefa [ ~ ]$ az vm extension set \
--publisher Microsoft.Azure.Monitor \
--name AzureMonitorWindowsAgent \
--vm-name soc-lab-vm01 \
--resource-group rg-stonegate-soc-prod
(ResourceNotFound) The Resource 'Microsoft.Compute/virtualMachines/soc-lab-vm01' under resource group 'rg-stonegate-soc-prod' was not found. For more details please go
to https://aka.ms/ARMResourceNotFoundFix
Code: ResourceNotFound
Message: The Resource 'Microsoft.Compute/virtualMachines/soc-lab-vm01' under resource group 'rg-stonegate-soc-prod' was not found. For more details please go to https:
//aka.ms/ARMResourceNotFoundFix
mojalefa [ ~ ]$ az vm list -o table
Name                                ResourceGroup                        Location                            Zones
-----
soc-lab-vm01                        SOC-LAB-VM01_GROUP_73077218         southafricanorth                    2
mojalefa [ ~ ]$ az vm extension set \
--publisher Microsoft.Azure.Monitor \
--name AzureMonitorWindowsAgent \
--vm-name soc-lab-vm01 \
--resource-group SOC-LAB-VM01_GROUP_73077218
{
  "autoUpgradeMinorVersion": true,
  "id": "/subscriptions/e97fe0a4-5209-4bae-85a3-39d05c820b28/resourceGroups/SOC-LAB-VM01_GROUP_73077218/providers/Microsoft.Compute/virtualMachines/soc-lab-vm01/extens
ions/AzureMonitorWindowsAgent",
  "location": "southafricanorth",
  "name": "AzureMonitorWindowsAgent",
  "provisioningState": "Succeeded",
  "publisher": "Microsoft.Azure.Monitor",
  "resourceGroup": "SOC-LAB-VM01_GROUP_73077218",
  "type": "Microsoft.Compute/virtualMachines/extensions",
  "typeHandlerVersion": "1.43",
  "typePropertiesType": "AzureMonitorWindowsAgent"
}

```

**Figure 34:** Full troubleshooting sequence in Cloud Shell: the initial ResourceNotFound error, corrective `az vm list -o table` lookup and the successful AzureMonitorWindowsAgent installation.

With the agent installed and reporting, a Heartbeat query confirmed soc-lab-vm01 was alive and checking in regularly. This marks the first piece of concrete evidence that the agent-to-workspace connection was functioning, even before any Security Event data had started flowing.

#### Heartbeat validation query:

Heartbeat  
| summarize Count = count() by Computer

The screenshot shows the Microsoft Sentinel interface with the 'Logs' view selected. A new query is entered in the KQL mode:

```

1 Heartbeat
2 | summarize Count = count() by Computer
3

```

The query results are displayed in a table:

| Computer       | Count |
|----------------|-------|
| > soc-lab-vm01 | 13    |

**Figure 35:** Heartbeat | summarize Count = count() by Computer returning soc-lab-vm01 with a count of 13, confirming the Azure Monitor Agent is installed, connected and actively reporting to law-stonegate-soc.

## Phase 6 — VM Access & Audit Policy Configuration

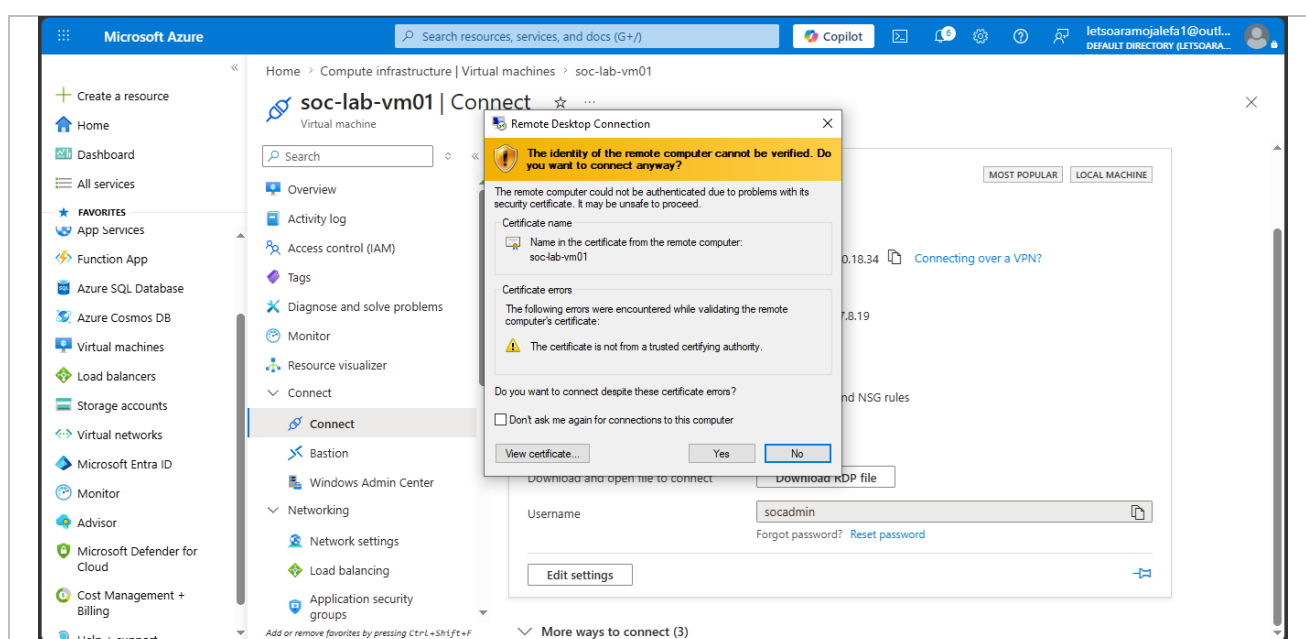
**Goal:** Log into soc-lab-vm01 via RDP and configure the Local Security Policy audit settings so that Windows actually records Security events (logons, privilege use, object access) - without this, the Security event log stays effectively empty regardless of how the ingestion pipeline is configured.

**Tools:** Remote Desktop Connection (RDP), Local Security Policy (secpol.msc), Windows Event Viewer.

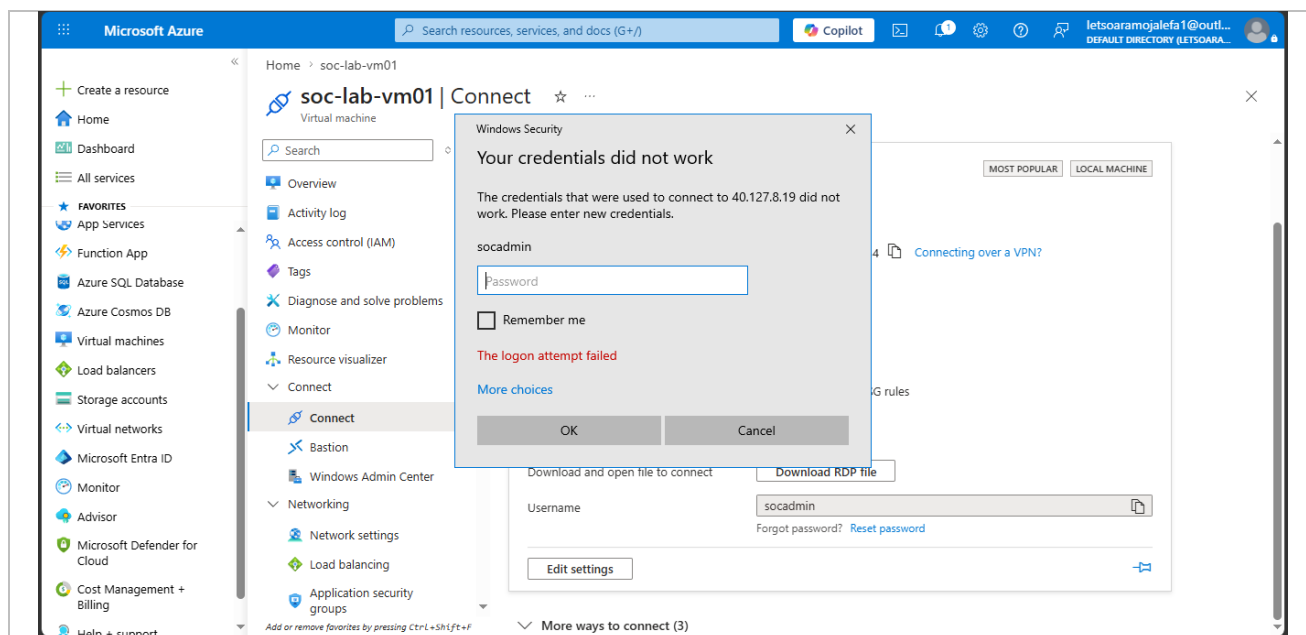
**Outcome:** After resolving an untrusted-certificate warning and one failed login attempt, RDP access to soc-lab-vm01 was established. All eight Local Security Policy audit categories were changed from No auditing to Success, Failure, after which the Security event log grew to 5,018 recorded events.

### 6.1 Establishing RDP Access

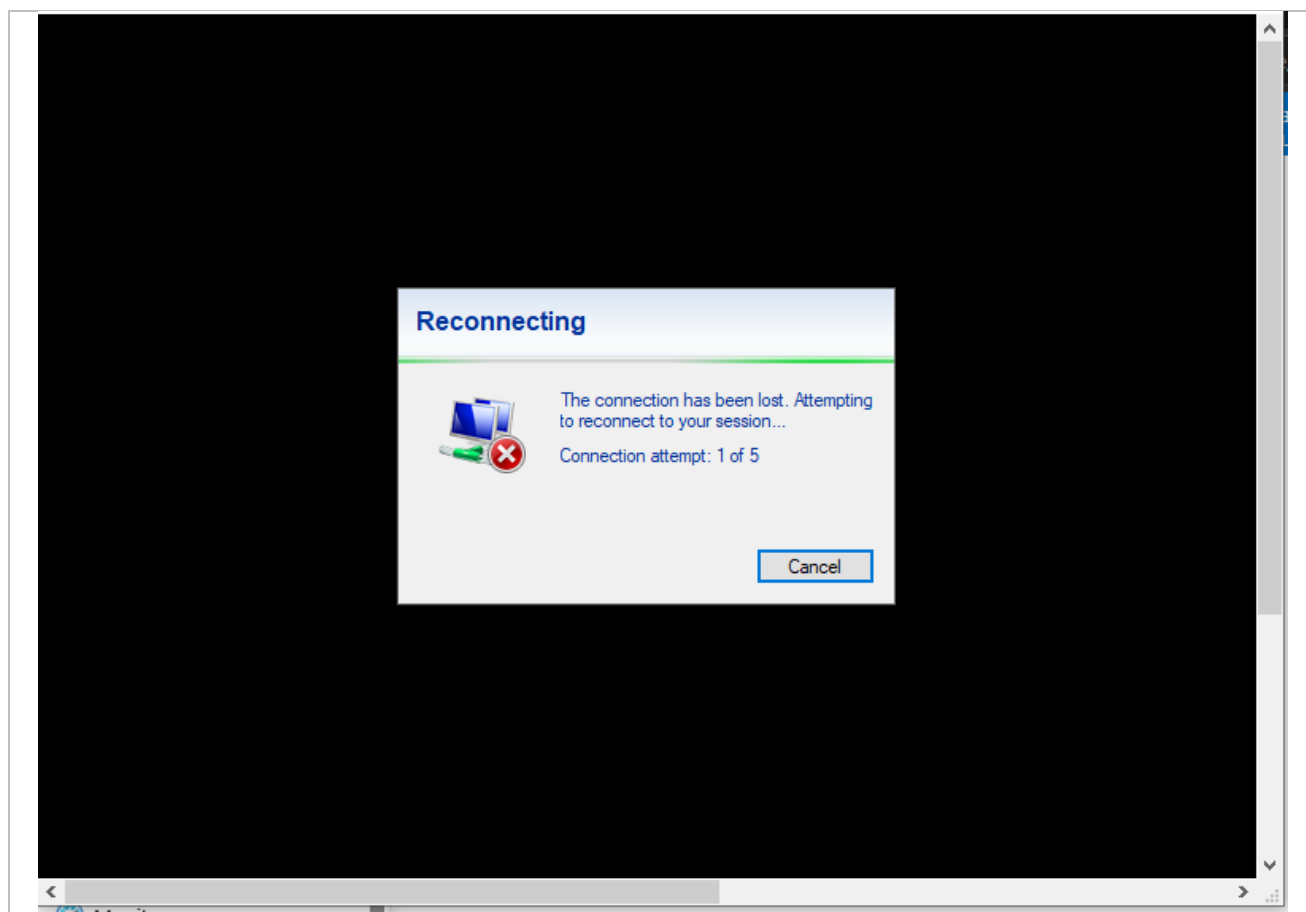
Connecting to the VM for the first time surfaced the standard self-signed-certificate warning expected for a freshly provisioned Azure VM, followed by one failed authentication attempt before the correct credentials were confirmed.



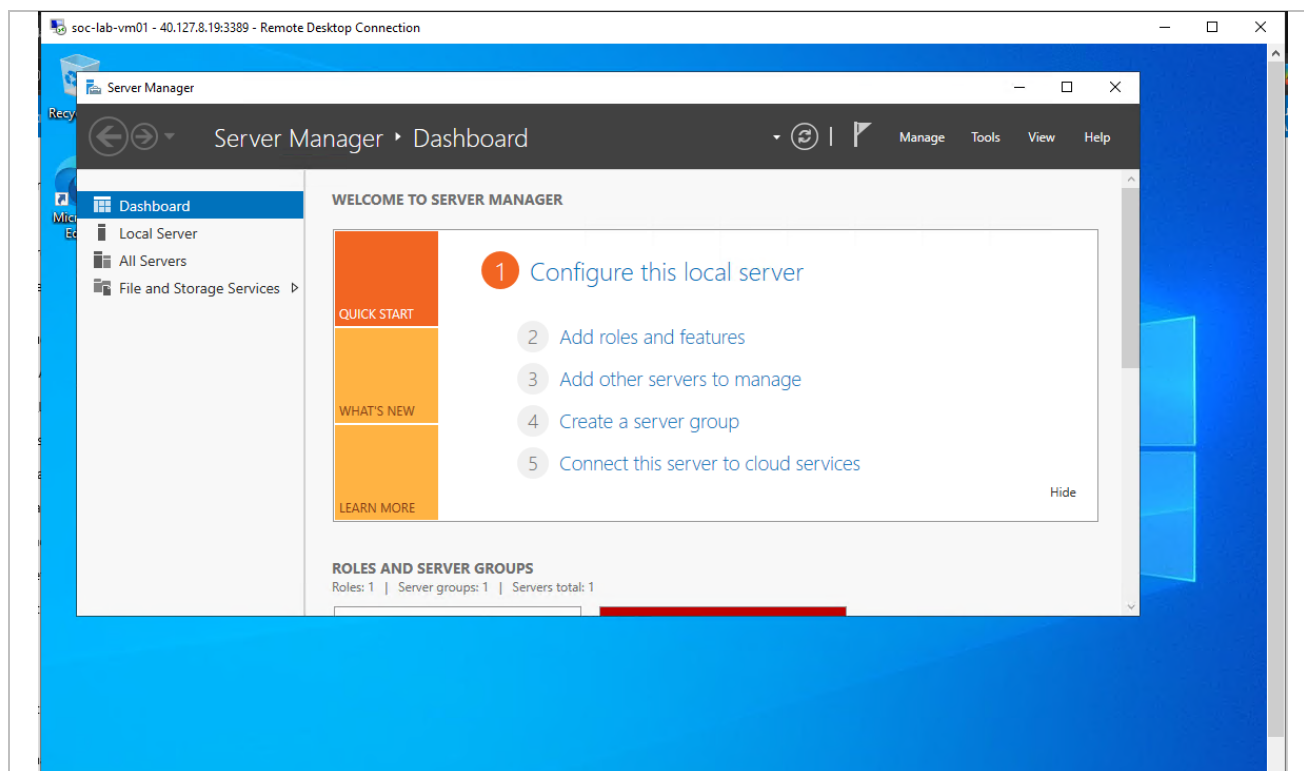
**Figure 36:** Remote Desktop Connection certificate warning: “The identity of the remote computer cannot be verified”, this is expected and safe to accept for a self-managed lab VM.



**Figure 37:** “Your credentials did not work”, this is an early failed logon attempt during initial access, itself a small preview of the EventID 4625 pattern that would later be deliberately generated at scale in Phase 8.



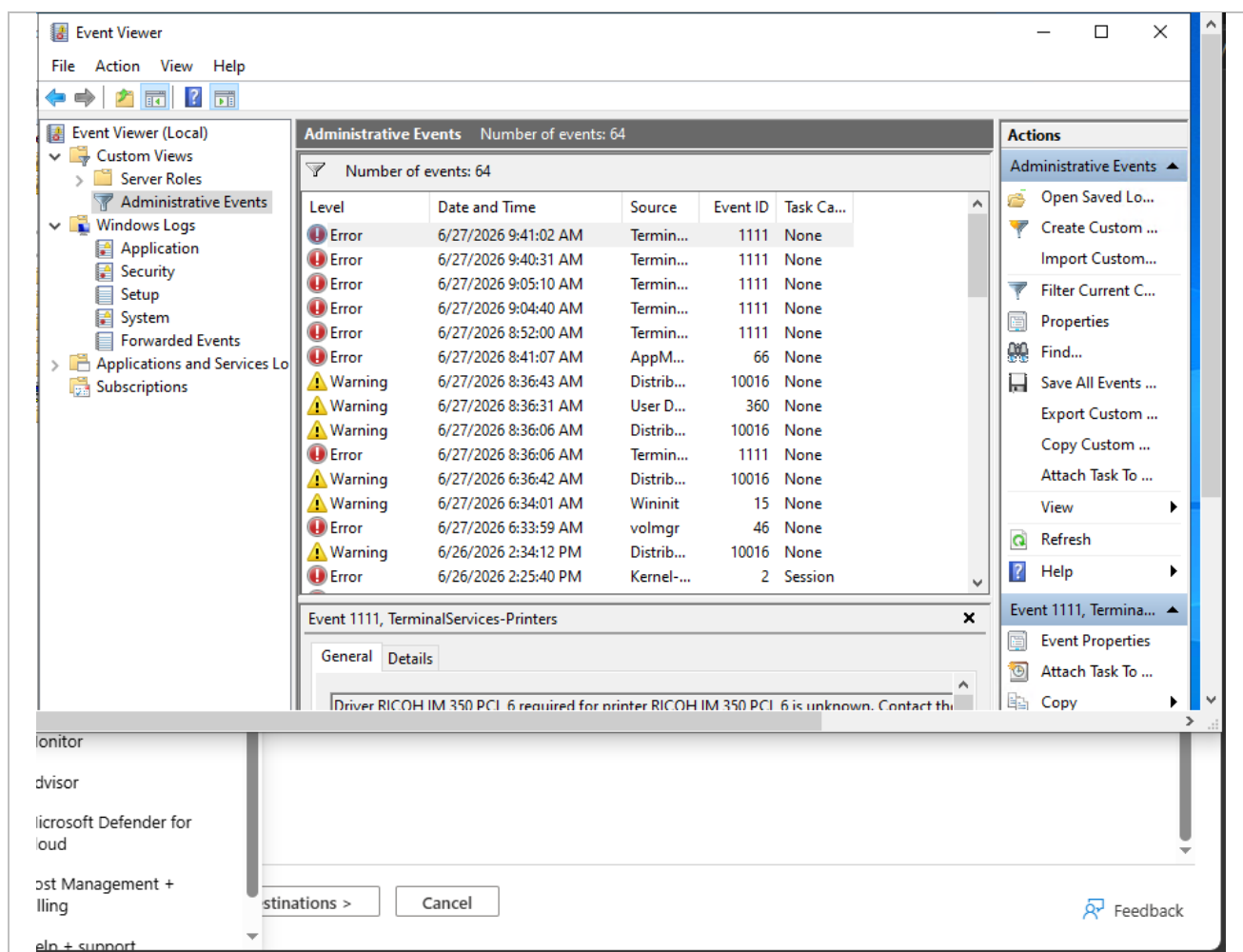
**Figure 38:** RDP session momentarily dropping and attempting to reconnect. This is a transient network blip and it resolved automatically.



**Figure 39:** First successful login: Server Manager Dashboard inside soc-lab-vm01, confirming full desktop access via RDP.

## 6.2 Reviewing Existing Event Logs

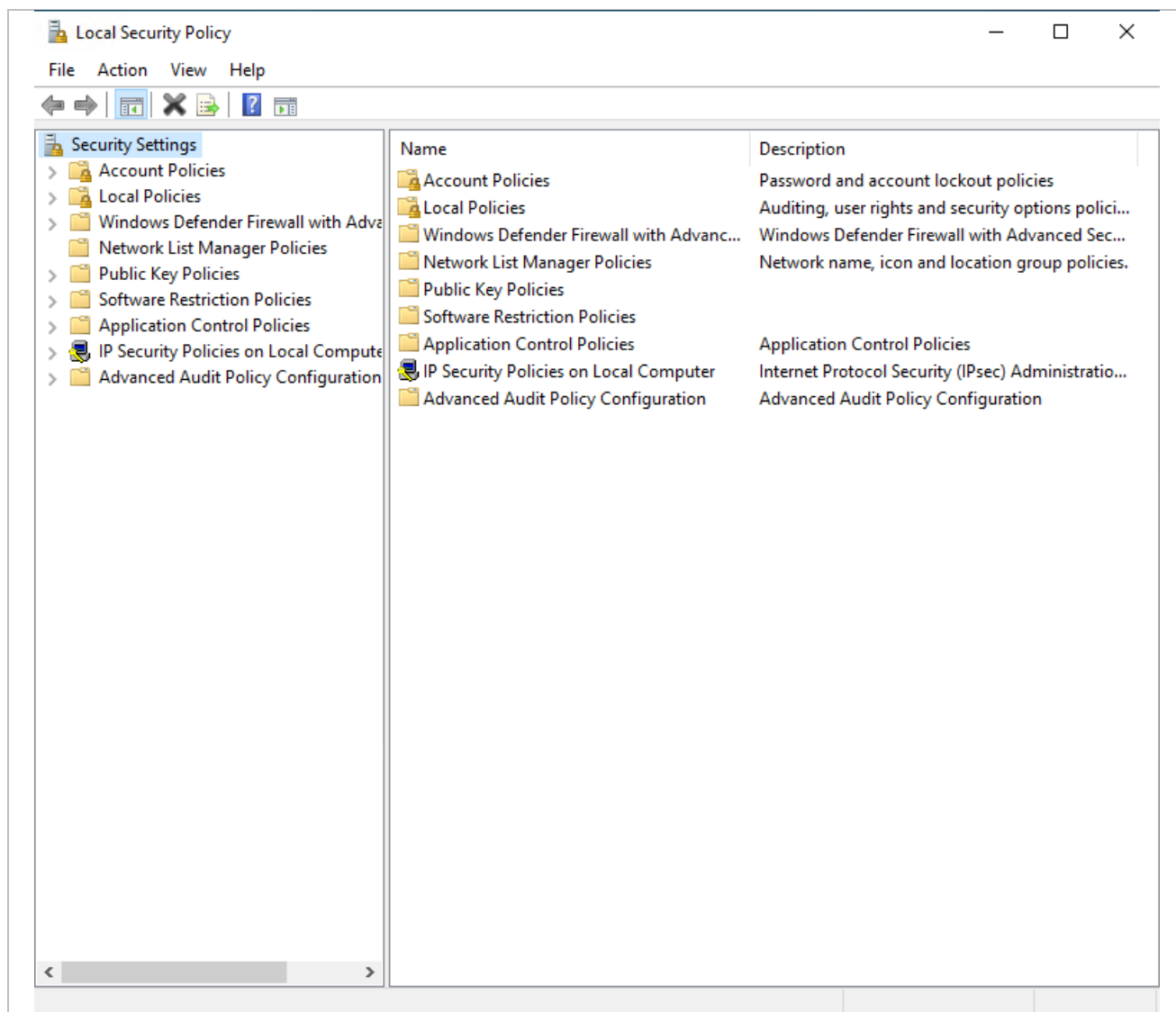
Before changing any audit policy, the existing Event Viewer logs were reviewed to understand the VM's baseline activity. At this point the Security log was still essentially empty of meaningful audit data while the Administrative Events view showed routine warnings (mostly Terminal Services printer redirection errors, these are harmless in a lab context).



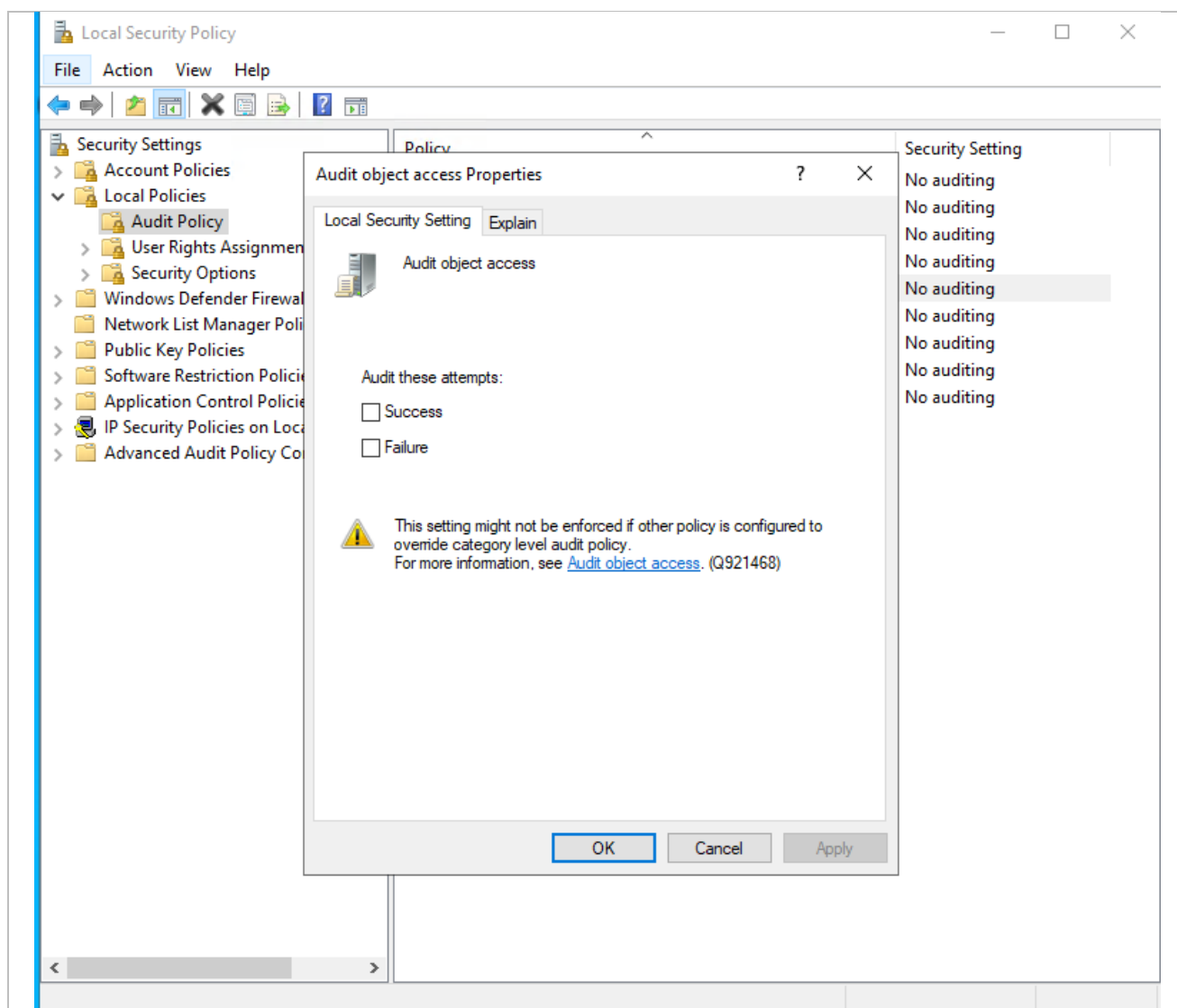
**Figure 40:** Event Viewer > Administrative Events showing 64 events, predominantly TerminalServices-Printers driver warnings. This is background noise and not security-relevant.

### 6.3 Local Security Policy — Before

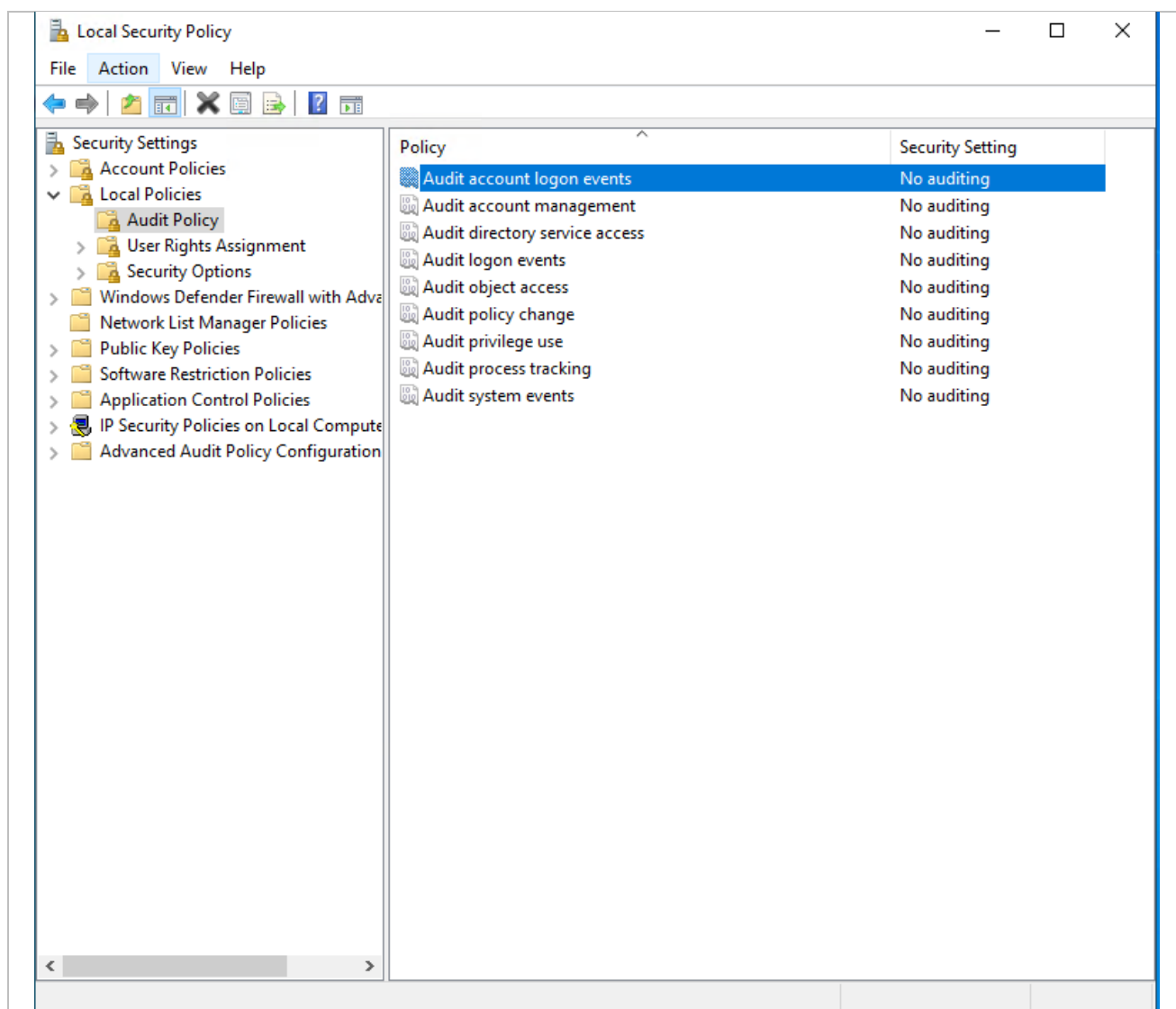
Opening secpol.msc (Local Security Policy) confirmed the root cause of the empty Security log: every audit category - logon events, account management, object access, privilege use and system events was set to No auditing by default. Without these policies enabled, Windows simply does not write the corresponding events regardless of how well the Azure-side ingestion pipeline is configured.



**Figure 41:** Local Security Policy > Security Settings tree expanded to show all available policy categories (Account Policies, Local Policies, Windows Defender Firewall and others).



**Figure 42:** Audit object access Properties dialog: neither Success nor Failure checked, with the background Audit Policy list confirming every category set to No auditing.



**Figure 43:** The full Audit Policy list view, confirming the No auditing baseline across all eight categories including Audit logon events and Audit object access before any changes were made.

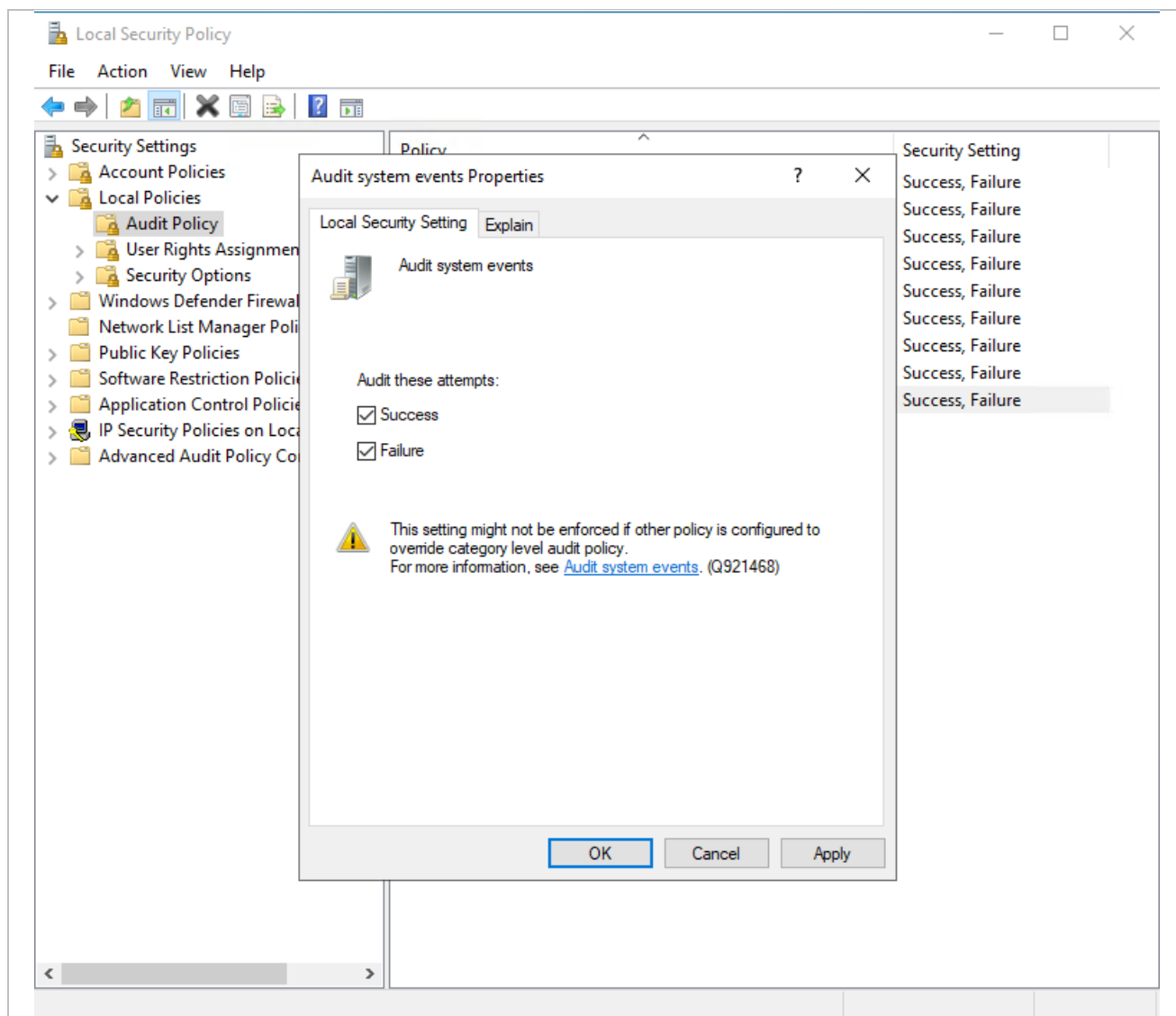
## 6.4 Local Security Policy — After

Each audit category was then enabled for both Success and Failure outcomes. This single change is what makes the Security event log meaningful: it is the difference between a Windows Server that silently allows or denies access and one that produces a forensic trail of every logon, privilege escalation and object access attempt – “success or failure” for Sentinel to ingest.

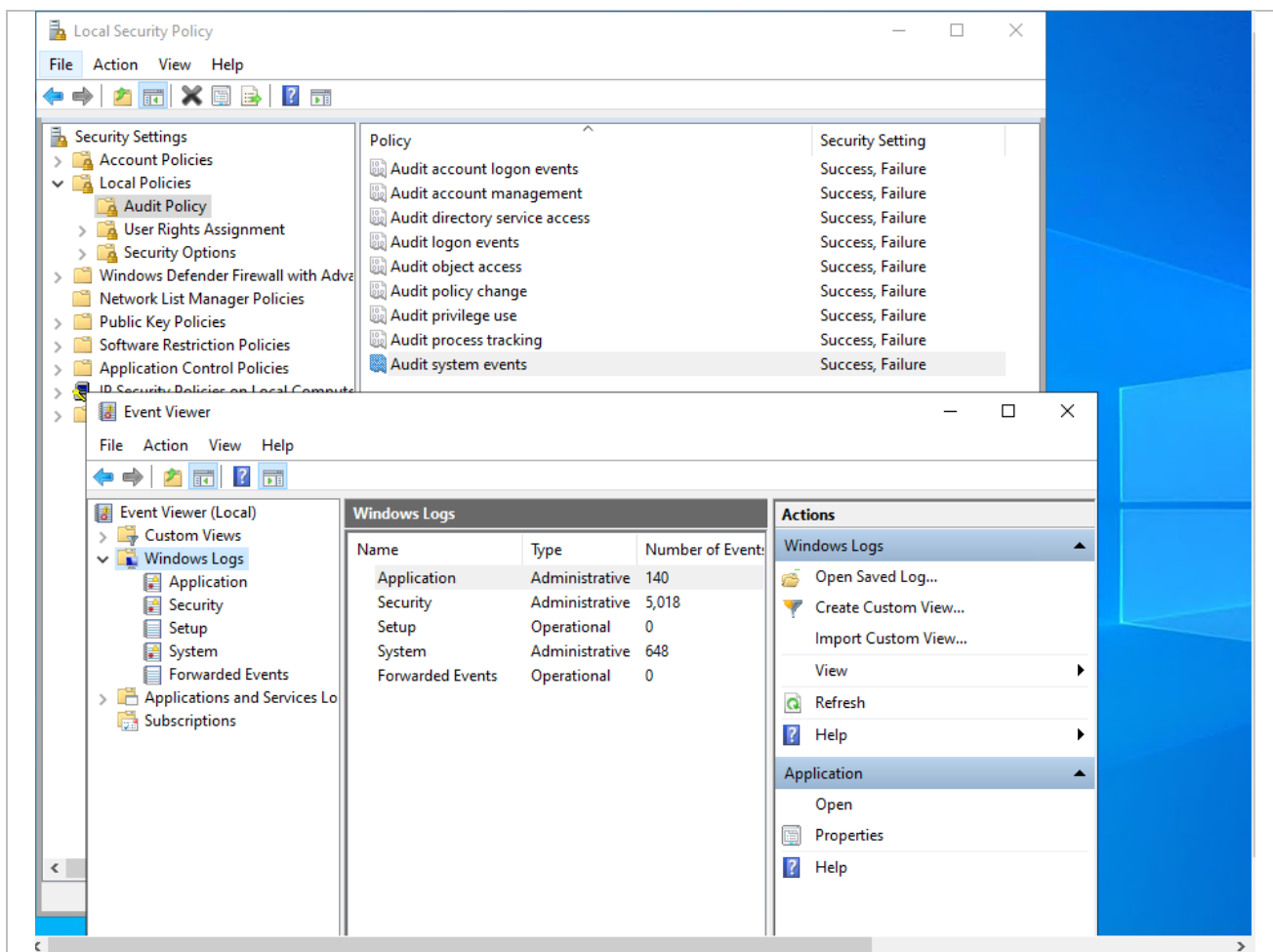
```

Audit account logon events → Success, Failure
Audit account management → Success, Failure
Audit directory service access → Success, Failure
Audit logon events → Success, Failure
Audit object access → Success, Failure
Audit policy change → Success, Failure
Audit privilege use → Success, Failure
Audit process tracking → Success, Failure
Audit system events → Success, Failure

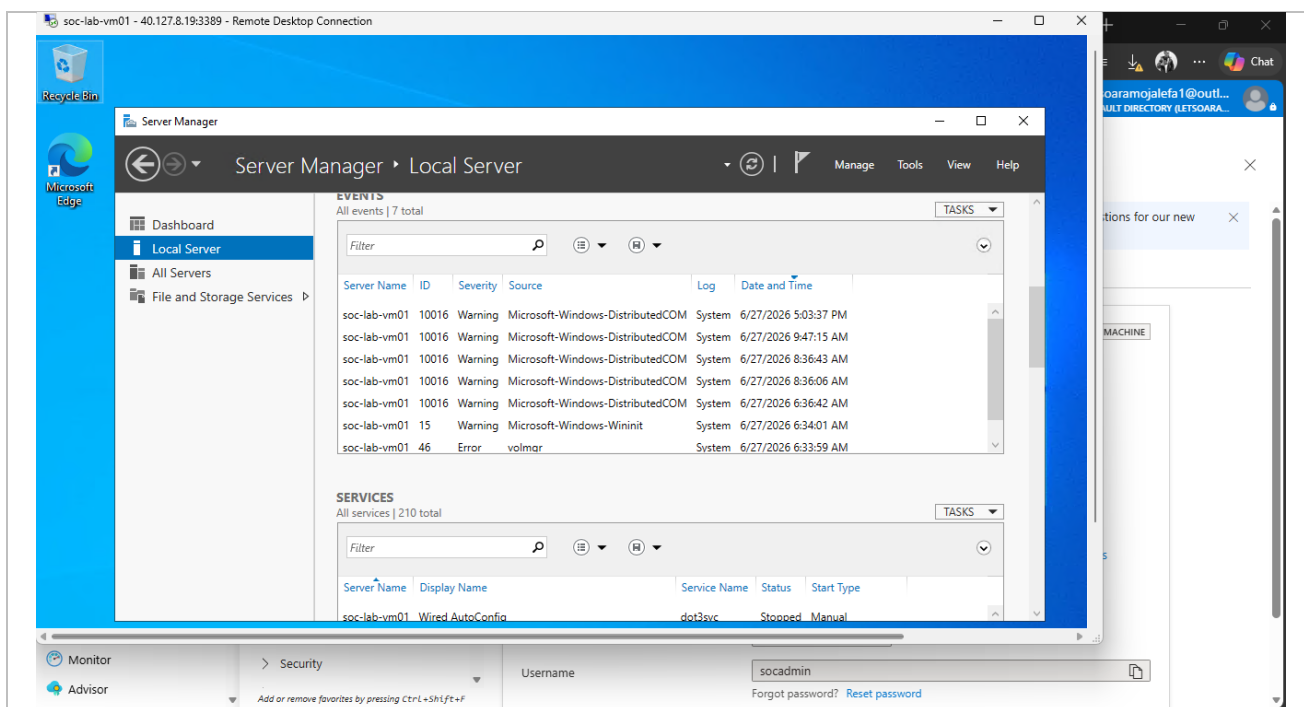
```



**Figure 44:** Audit system events Properties dialog: both Success and Failure now checked with the background Audit Policy list confirming every category has been changed to “Success, Failure.”



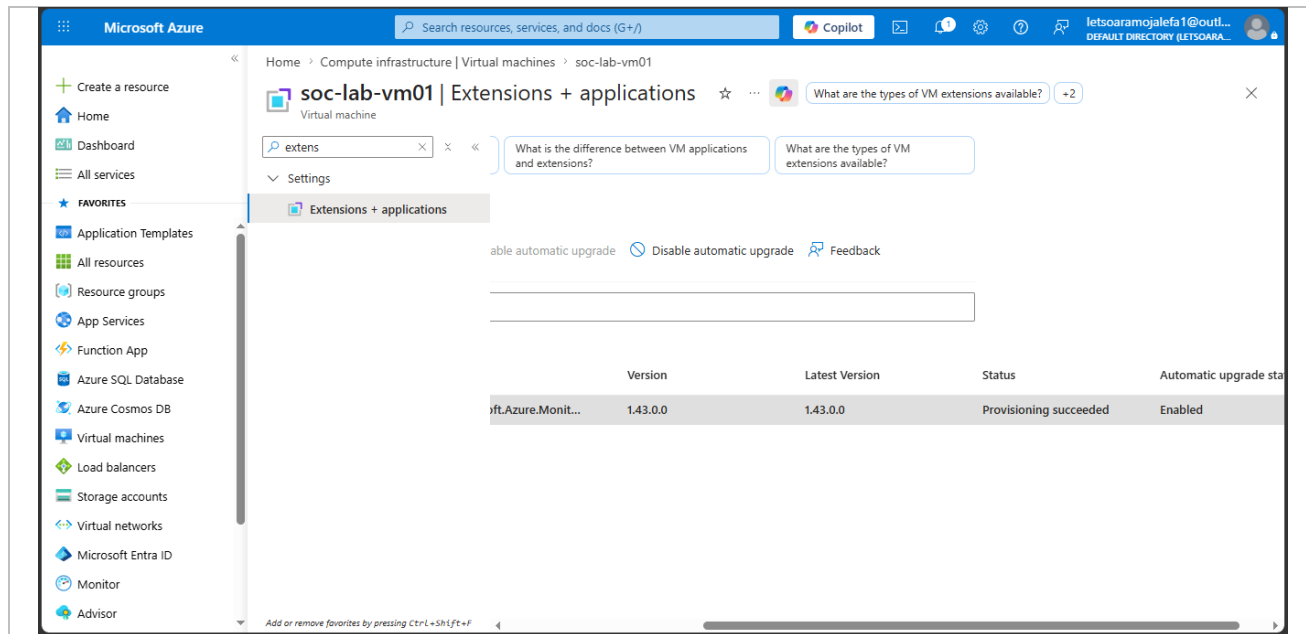
**Figure 45:** Local Security Policy (left) showing every audit category now set to “Success, Failure” alongside Event Viewer (right) confirming the Security log has grown to 5,018 events. This is direct, immediate evidence that shows us that the audit policy change took effect.



**Figure 46:** Server Manager > Local Server events panel inside the RDP session, showing a mix of routine System-level warnings generated during the session.

## 6.5 A Note on the Legacy Diagnostics Extension

While exploring monitoring options on the VM, Azure surfaced a prompt to install the legacy Azure Diagnostics extension with a notice that it would be deprecated on 31 March 2026. Since the Azure Monitor Agent and Data Collection Rule pipeline already in place is the modern, supported path for collecting guest-level logs, this legacy extension was deliberately not installed, avoiding redundant or conflicting telemetry pipelines.



**Figure 47:** Azure Diagnostics extension deprecation notice encountered while reviewing VM monitoring options. AMA + DCR was already the correct and modern approach in use, so this legacy extension was not installed.

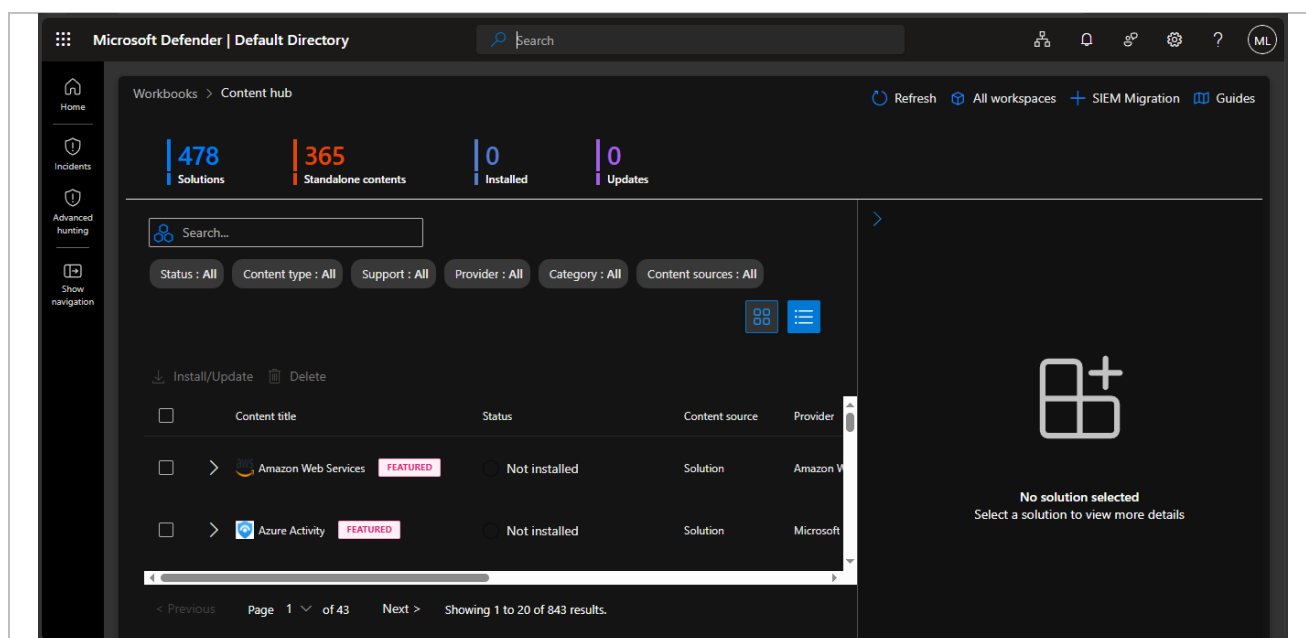
## Phase 7 — Content Hub & Workbook Deployment

**Goal:** Install Microsoft's official Windows Security Events solution from the Sentinel Content Hub to gain pre-built workbook dashboards and assemble a small library of workbooks covering authentication, identity and network telemetry.

**Tools:** Microsoft Defender portal (Content Hub, Workbooks).

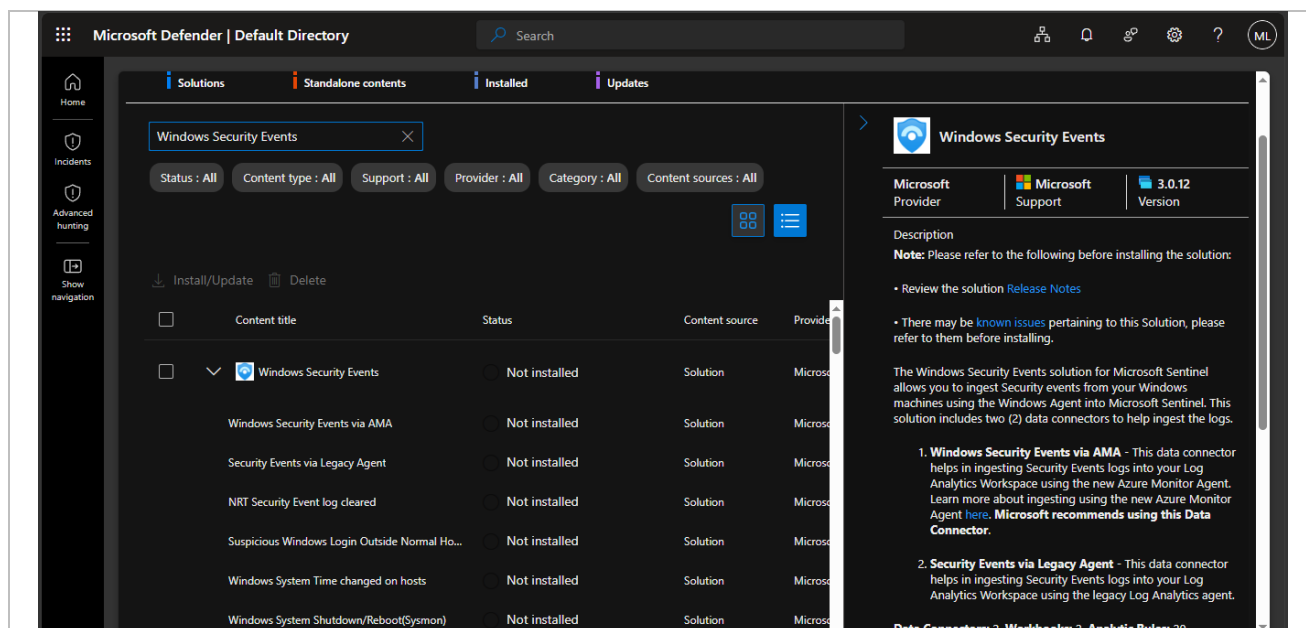
**Outcome:** The Windows Security Events solution was installed from a catalogue of 478 available solutions, adding two workbook templates (Event Analyzer, Identity & Access). Six workbooks were ultimately saved into the workspace, covering Windows Security Events, Identity & Access, Microsoft Defender for Identity, Microsoft Entra ID Sign-ins and Windows Firewall.

Microsoft Sentinel's workbook templates are not pre-loaded into a new workspace; they are distributed as installable solutions via the Content Hub, alongside data connectors, analytics rule templates and hunting queries. The Content Hub catalogue contains 478 solutions and 365 standalone content items spanning every major vendor and use case, from Microsoft's own product suite to community-contributed detections.

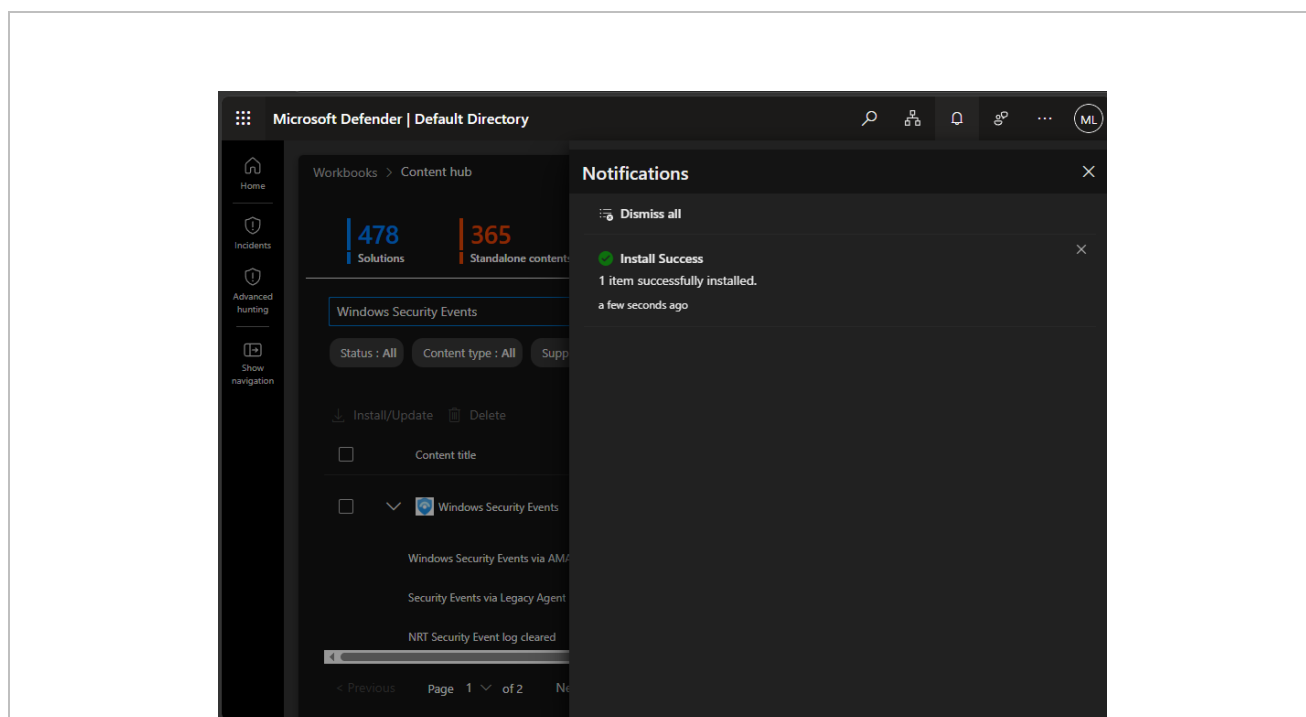


**Figure 48:** Browsing the Content Hub catalogue - 478 solutions available including featured entries for Amazon Web Services and Azure Activity.

Searching specifically for “Windows Security Events” surfaced the official Microsoft solution, which bundles two data connectors (Windows Security Events via AMA and the legacy Security Events via Legacy Agent) alongside its associated workbooks and analytics rule templates.

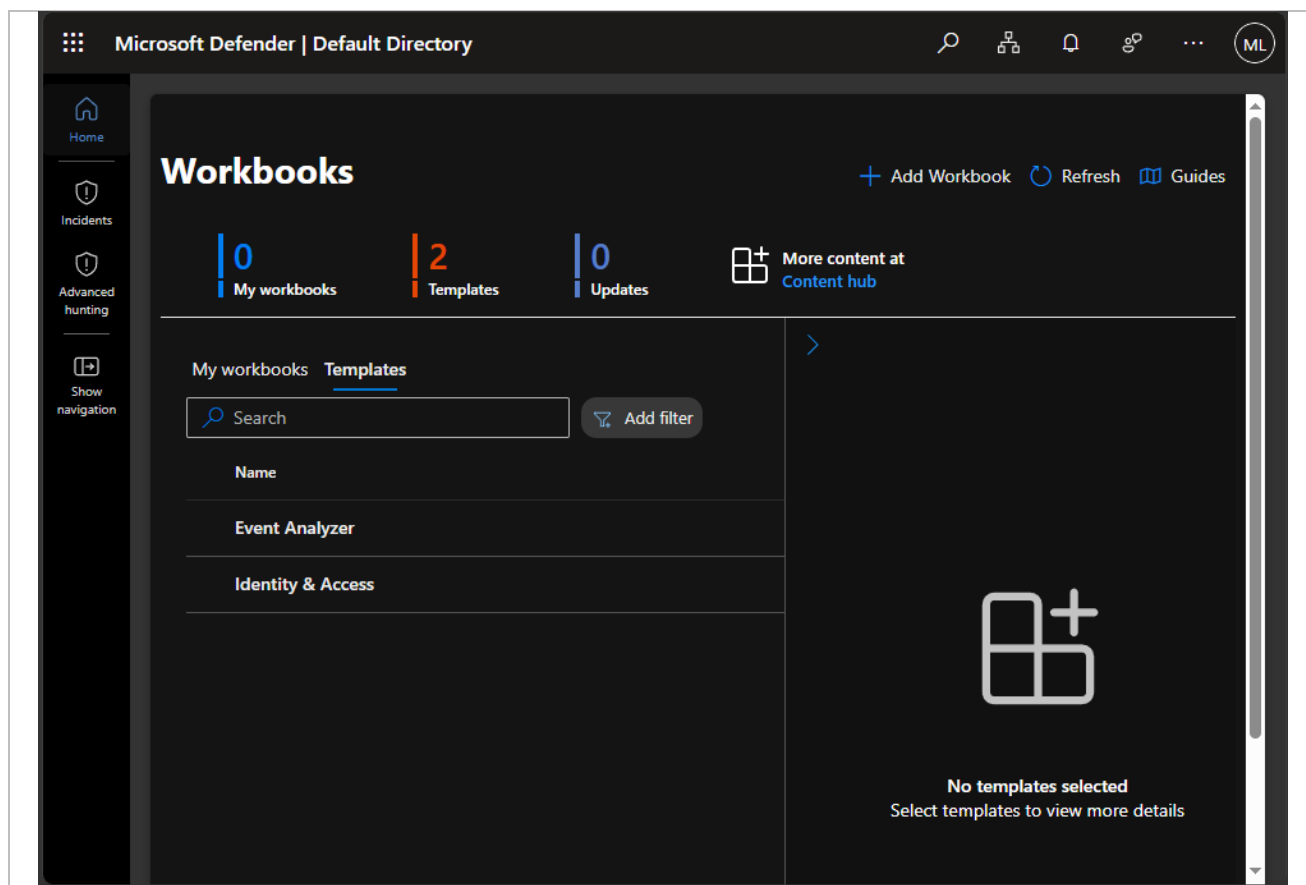


**Figure 49:** Windows Security Events solution detail pane: Microsoft-published, version 3.0.12, bundling 2 data connectors, 2 workbooks and 20 analytic rules.



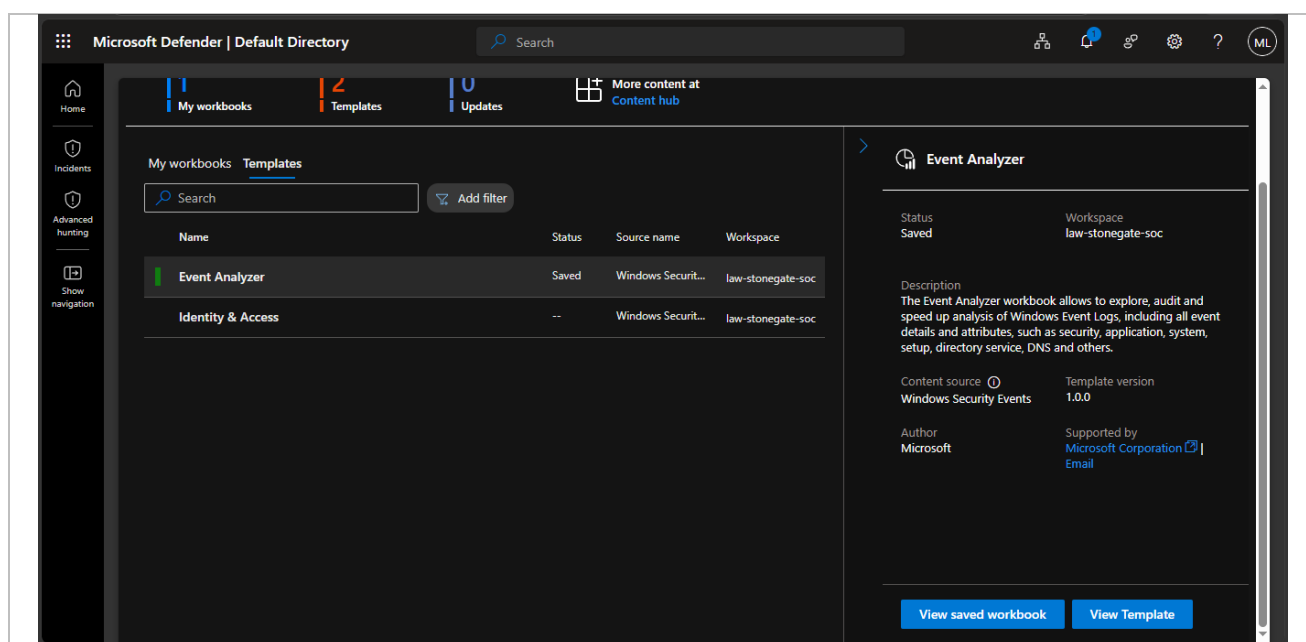
**Figure 50:** Install Success notification confirming the solution was deployed into law-stonegate-soc, adding workbook templates, analytics rules and connector definitions to the workspace.

Once installed, the Workbooks blade's Templates tab that was previously empty and unclickable, was populated with the two templates bundled in the solution: Event Analyzer and Identity & Access.



**Figure 51:** Workbooks > Templates now showing 2 available templates (Event Analyzer, Identity & Access) immediately after the Windows Security Events solution installed.

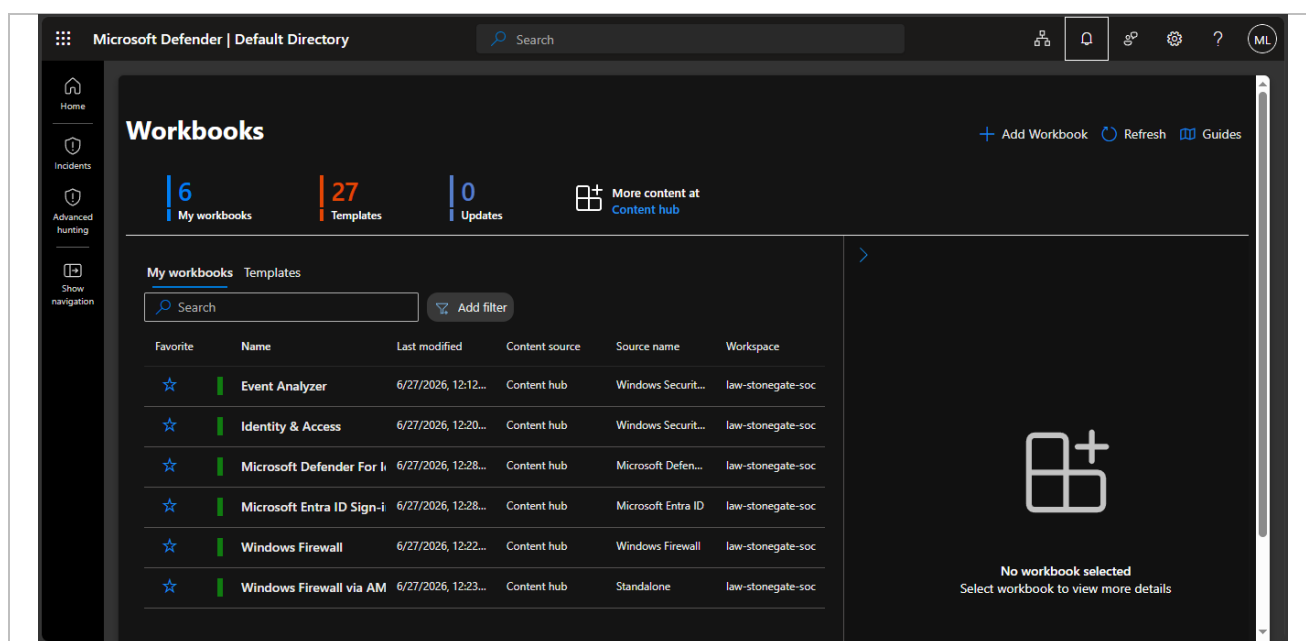
The Event Analyzer template: designed specifically to explore and audit Windows Event Logs across Security, Application, System, Setup, Directory Service and DNS categories was saved into the workspace as the primary working dashboard for validating ingestion.



**Figure 52:** Event Analyzer workbook detail, confirming Status: Saved, Workspace: law-stonegate-soc, sourced from the Windows Security Events content package.

By the end of this phase, six workbooks had been saved into the workspace's My Workbooks list, giving broad visibility across authentication, identity and network log sources - without yet having

any live Security event data to populate them, which is precisely the gap the next phase would resolve.



**Figure 53:** Final set of six saved workbooks: Event Analyzer, Identity & Access, Microsoft Defender For Identity, Microsoft Entra ID Sign-in logs, Windows Firewall and Windows Firewall via AMA.

## Phase 8 — The DCR Stream Misconfiguration: Diagnosis & Fix

**Goal:** Diagnose why SecurityEvent queries kept returning zero results despite a confirmed agent heartbeat, fix the underlying Data Collection Rule misconfiguration and validate that real attack telemetry is finally reaching the SecurityEvent table.

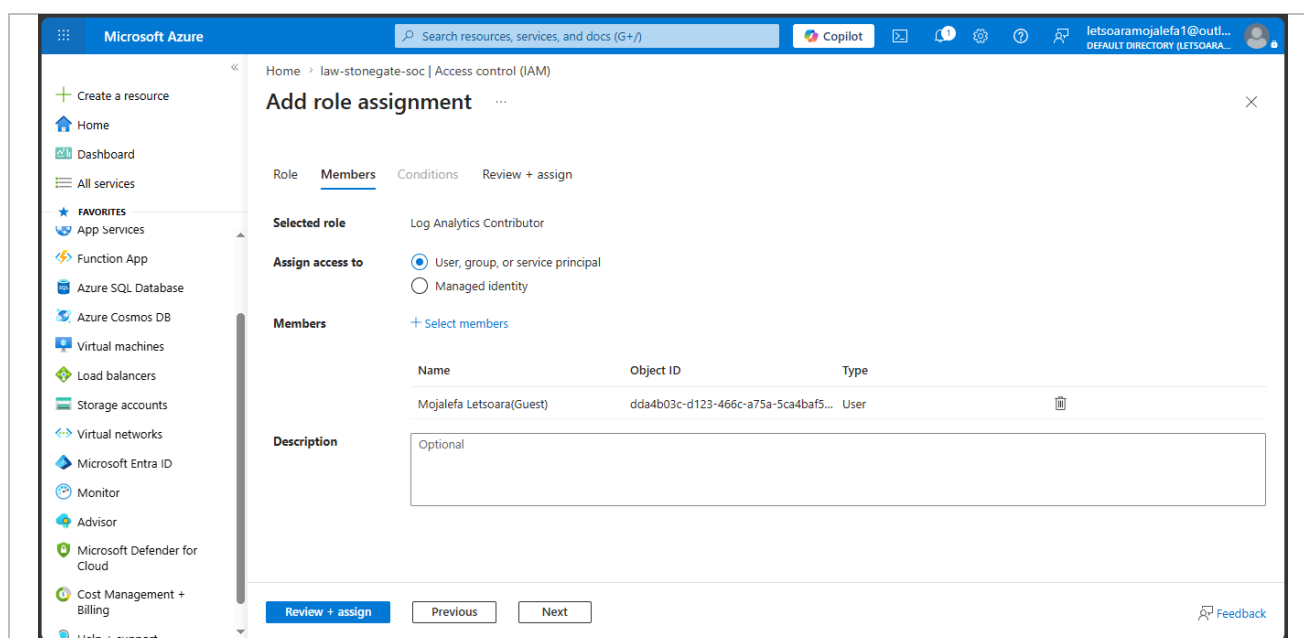
**Tools:** Azure Role-Based Access Control (IAM), Azure Cloud Shell (PowerShell, Get-AzDataCollectionRule, Update-AzDataCollectionRule), Microsoft Sentinel Logs (KQL).

**Outcome:** The DCR's DataFlow and DataSourceWindowsEventLog properties were both found to be streaming into the generic Microsoft-Event table instead of Microsoft-SecurityEvent. Both were corrected via PowerShell. Within minutes, SecurityEvent queries returned real data - 38 distinct EventIDs, including repeated EventID 4625 failed logons from account DESKTOP-KGT91IR\socadmin at IP 41.122.8.167, with a summarised brute-force query confirming 6 failed attempts inside a single 5-minute window.

This phase represents the most significant troubleshooting challenge of the entire build. Every visible signal in the Azure Portal suggested the pipeline was healthy: the VM had a Ready agent status, Heartbeat data was flowing, the DCR showed Provisioning succeeded and the Local Security Policy was correctly generating thousands of Security log entries on the VM itself. And yet, every SecurityEvent query in Sentinel across 24 hours, then 7 days, then 30 days? returned “No results found.”

### 8.1 Granting Workspace Permissions

Before the DCR itself could be properly diagnosed, permissions gap surfaced: workbook dropdowns and certain Logs operations would not reliably persist a workspace selection. The Log Analytics Contributor role was assigned directly on law-stonegate-soc to ensure full read/query and configuration rights were in place and independent of the subscription-level Owner role already held.



**Figure 54:** Add role assignment: Log Analytics Contributor granted to Mojalefa Letsoara (Guest) directly on the law-stonegate-soc workspace, removing any ambiguity around effective permissions.

### 8.2 Isolating the Fault — Heartbeat Works, SecurityEvent Doesn't

With Heartbeat confirmed and Security logs visibly accumulating inside the VM's own Event Viewer (5,018 events, per Phase 6), the fault had to be somewhere between the VM and the SecurityEvent

table.... specifically in how the DCR routed Windows Event Log data once collected. The DCR's underlying JSON definition, inspected directly via PowerShell, revealed the answer that the Portal UI had been hiding.

#### Diagnostic command:

```
Get-AzDataCollectionRule -ResourceGroupName "rg-stonegate-soc-prod" `
-RuleName "dcr-soc-lab" |
Select-Object -ExpandProperty DataSourceWindowsEventLog |
Format-List *
```

Stream : {Microsoft-Event} ← WRONG. Should be Microsoft-SecurityEvent

The DCR's Windows Event Log data source was configured with the correct XPath queries to capture Application, System and Security events including the Security XPath being filtered by the audit keyword bitmask - but the Stream property determining which Log Analytics table that data lands in was set to the generic Microsoft-Event stream, not Microsoft-SecurityEvent. Sentinel's analytics engine and every workbook built on the Windows Security Events solution, reads exclusively from the SecurityEvent table. Data routed to the wrong stream is technically ingested but functionally invisible to every security tool built on top of it.

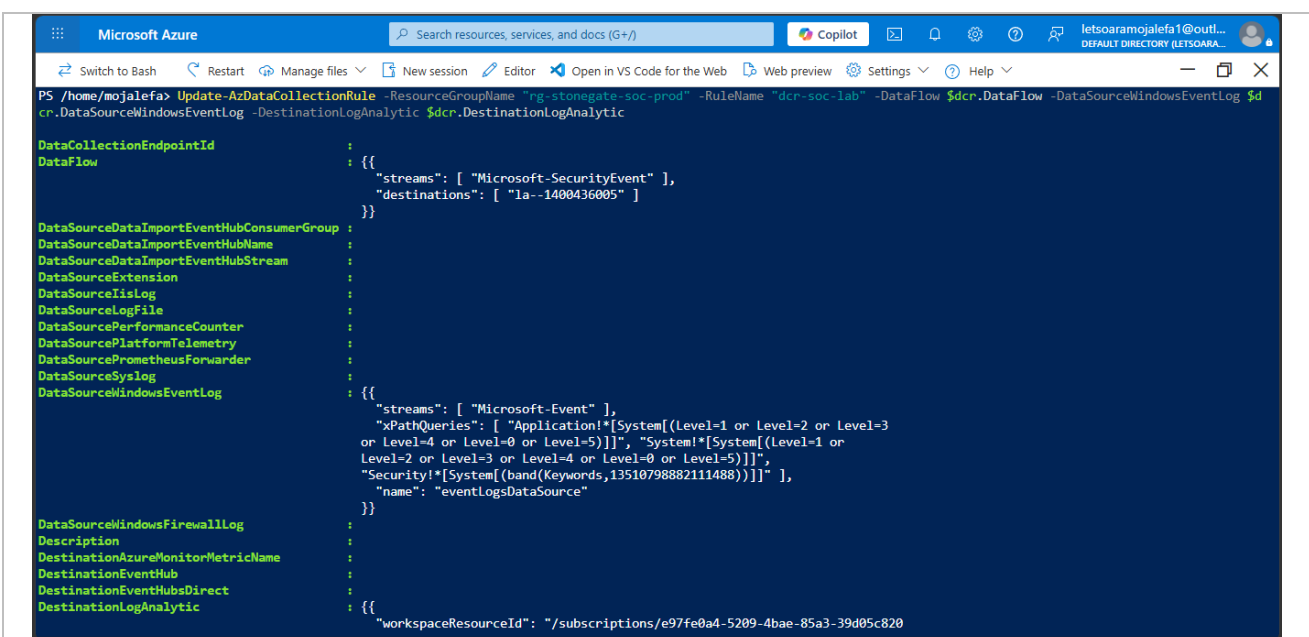
### 8.3 Applying the Fix via PowerShell

The fix required two separate corrections that were applied in sequence: first the DataFlow stream binding, then the DataSourceWindowsEventLog stream declaration itself - both needed to agree for data to land correctly.

#### Fix #1 - correcting the DataFlow stream:

```
$dcr = Get-AzDataCollectionRule -ResourceGroupName "rg-stonegate-soc-prod" -RuleName "dcr-soc-lab"
$dcr.DataFlow[0].Stream = @("Microsoft-SecurityEvent")

Update-AzDataCollectionRule -ResourceGroupName "rg-stonegate-soc-prod" -RuleName "dcr-soc-lab" `
-DataFlow $dcr.DataFlow -DataSourceWindowsEventLog $dcr.DataSourceWindowsEventLog `
-DestinationLogAnalytic $dcr.DestinationLogAnalytic
```



**Figure 55:** First fix applied: DataFlow.Stream is now correctly set to Microsoft-SecurityEvent. However, DataSourceWindowsEventLog still shows Stream: ["Microsoft-Event"] - the two properties were out of sync.

```

DataSourceWindowsFirewallLog
Description
DestinationAzureMonitorMetricName
DestinationEventHub
DestinationEventHubsDirect
DestinationLogAnalytic
    {
      "workspaceResourceId": "/subscriptions/e97fe0a4-5209-4bae-85a3-39d05c820b28/resourceGroups/rg-stonegate-soc-prod/providers/Microsoft.OperationalInsights/workspaces/law-stonegate-soc",
      "workspaceId": "3bb12aee-4a44-46b1-8065-e1e91c8de8e5",
      "name": "1a--1400436005"
    }
DestinationMonitoringAccount
DestinationStorageAccount
DestinationStorageBlobsDirect
DestinationStorageTablesDirect
Etag
    "e80040f5-0000-3000-0000-6a3ffc880000"
Id
    /subscriptions/e97fe0a4-5209-4bae-85a3-39d05c820b28/resourceGroups/rg-stonegate-soc-prod/providers/Microsoft.Insights/dataCollectionRules/dcr-soc-1a-b
IdentityPrincipalId
IdentityTenantId
IdentityType
IdentityUserAssignedIdentity
    {
    }
ImmutableId
    dcr-df4dbdd4f01e4f16a7238c48fcb397d6
Kind
Location
    southafricanorth
MetadataProvisionedBy
MetadataProvisionedByResourceId
Name
    dcr-soc-lab
ProvisioningState
    Succeeded
  
```

**Figure 56:** Scrolled continuation of the same output, confirming the *DestinationLogAnalytic* correctly resolves to *law-stonegate-soc* with *ProvisioningState: Succeeded*.

Re-querying *SecurityEvent* after this first fix still returned no results after seven days, confirming the partial fix was insufficient. A second, more targeted correction was required on the *DataSourceWindowsEventLog* property itself.

#### Fix #2 - correcting the *DataSourceWindowsEventLog* stream:

```

$dc = Get-AzDataCollectionRule -ResourceGroupName "rg-stonegate-soc-prod" -RuleName "dcr-soc-lab"
$dc.DataSourceWindowsEventLog[0].Stream = @("Microsoft-SecurityEvent")

Update-AzDataCollectionRule -ResourceGroupName "rg-stonegate-soc-prod" -RuleName "dcr-soc-lab" `
-DataFlow $dc.DataFlow -DataSourceWindowsEventLog $dc.DataSourceWindowsEventLog `
-DestinationLogAnalytic $dc.DestinationLogAnalytic
  
```

```

ImmutableId
    dcr-df4dbdd4f01e4f16a7238c48fcb397d6
Kind
Location
    southafricanorth
MetadataProvisionedBy
MetadataProvisionedByResourceId
Name
    dcr-soc-lab
ProvisioningState
    Succeeded
ResourceGroupName
    rg-stonegate-soc-prod
StreamDeclaration
    {
    }
SystemDataCreatedAt
    6/27/2026 7:31:36 AM
SystemDataCreatedBy
    letsoaramojalefa1@outlook.com
SystemDataCreatedByType
    User
SystemDataLastModifiedAt
    6/27/2026 4:38:27 PM
SystemDataLastModifiedBy
    letsoaramojalefa1@outlook.com
SystemDataLastModifiedByType
    User
Tag
    {
      "Project": "SOC-Foundation",
      "Environment": "SOC-Lab",
      "Company": "Stonegate Retail \u0026amp; Logistics",
      "Owner": "Mojalefa Letsoara",
      "Workload": "SecurityOperations"
    }
Type
    Microsoft.Insights/dataCollectionRules
  
```

```

PS /home/mojalefa> Get-AzDataCollectionRule -ResourceGroupName "rg-stonegate-soc-prod" -RuleName "dcr-soc-lab" | Select-Object -ExpandProperty DataFlow | Format-List *

BuiltInTransform
Destination
    {1a--1400436005}
OutputStream
Stream
    {Microsoft-SecurityEvent}
TransformKql
  
```

**Figure 57:** DCR resource tags confirmed intact (*Project, Environment, Company, Owner, Workload*), followed by *Get-AzDataCollectionRule | Format-List* on *DataFlow* - now showing *Stream: {Microsoft-SecurityEvent}* cleanly.

## 8.4 Final Verification — Both Streams Aligned

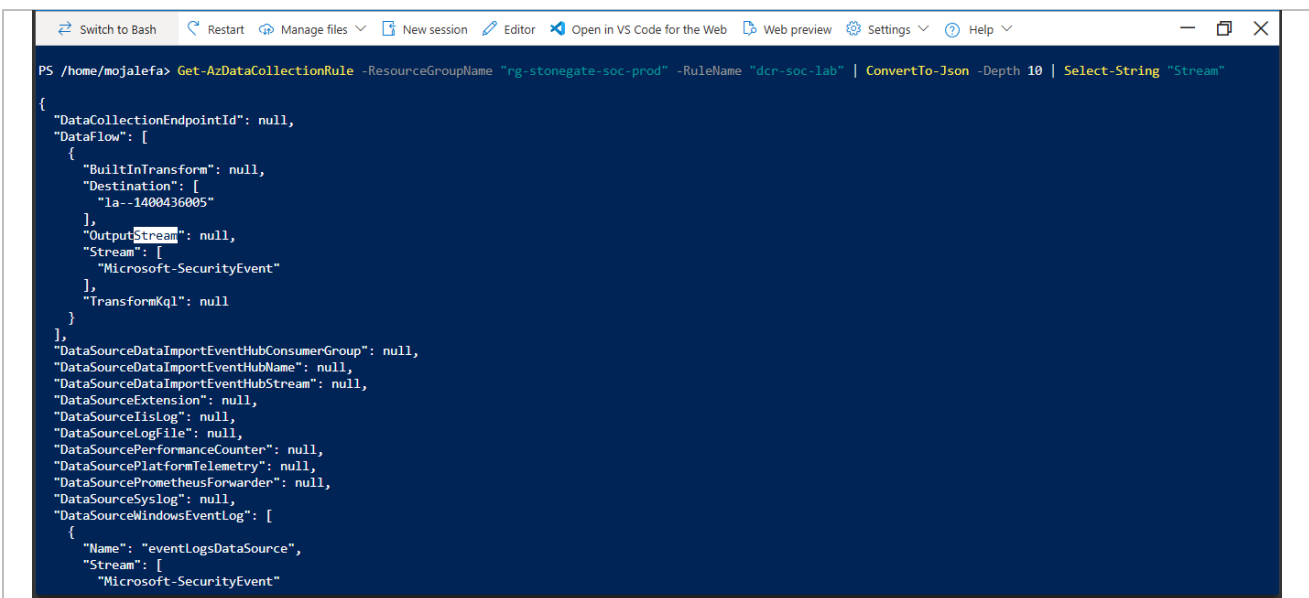
A final verification command confirmed both the DataFlow and DataSourceWindowsEventLog properties were now consistently set to Microsoft-SecurityEvent, with the DCR's ProvisioningState reporting Succeeded.

### Final verification command:

```
Get-AzDataCollectionRule -ResourceGroupName "rg-stonegate-soc-prod" -RuleName "dcr-soc-lab" |
ConvertTo-Json -Depth 10 | Select-String "Stream"
```

```
"Stream": [ "Microsoft-SecurityEvent" ] ← DataFlow
```

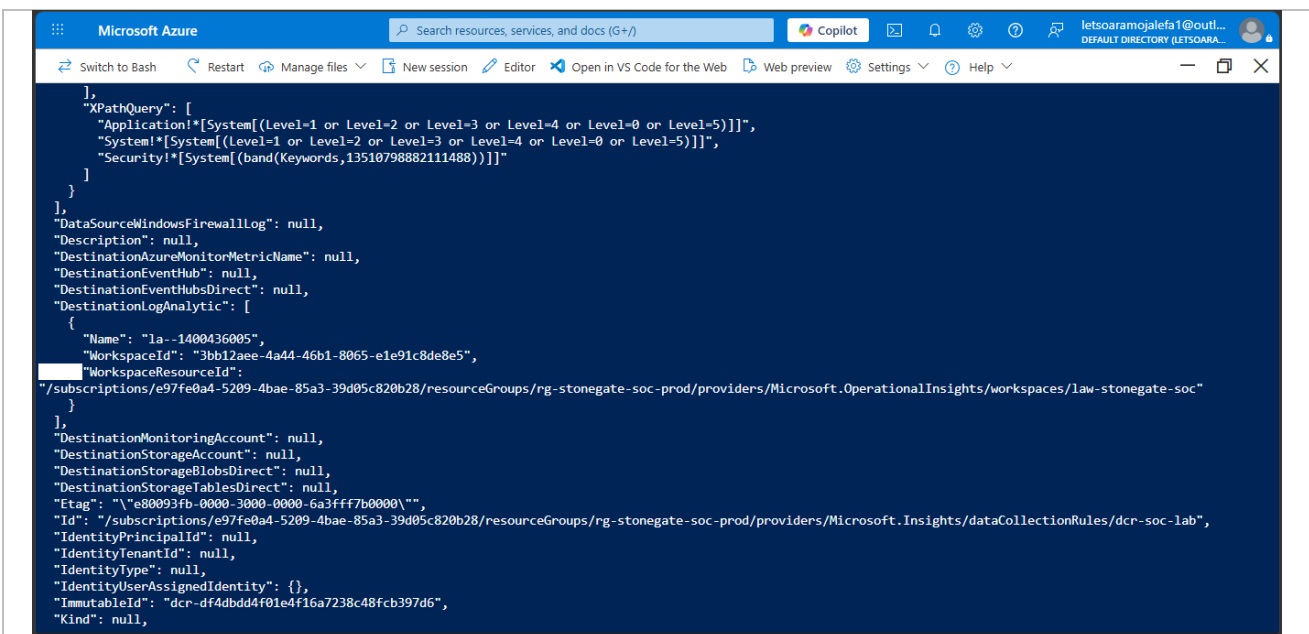
```
"Stream": [ "Microsoft-SecurityEvent" ] ← DataSourceWindowsEventLog
```



```
PS /home/mojalefa> Get-AzDataCollectionRule -ResourceGroupName "rg-stonegate-soc-prod" -RuleName "dcr-soc-lab" | ConvertTo-Json -Depth 10 | Select-String "Stream"

{
  "DataCollectionEndpointId": null,
  "DataFlow": [
    {
      "BuiltInTransform": null,
      "Destination": [
        "la--1400436005"
      ],
      "OutputStream": null,
      "Stream": [
        "Microsoft-SecurityEvent"
      ],
      "TransformKql": null
    }
  ],
  "DataSourceDataImportEventHubConsumerGroup": null,
  "DataSourceDataImportEventHubName": null,
  "DataSourceDataImportEventHubStream": null,
  "DataSourceExtension": null,
  "DataSourceIsLog": null,
  "DataSourceLogFile": null,
  "DataSourcePerformanceCounter": null,
  "DataSourcePlatformTelemetry": null,
  "DataSourcePrometheusForwarder": null,
  "DataSourceSyslog": null,
  "DataSourceWindowsEventLog": [
    {
      "Name": "eventLogsDataSource",
      "Stream": [
        "Microsoft-SecurityEvent"
      ]
    }
  ]
}
```

**Figure 58:** Full ConvertTo-Json output confirming BOTH DataFlow.Stream and DataSourceWindowsEventLog.Stream now read Microsoft-SecurityEvent – now the misconfiguration is fully resolved.



```
},
  "XPathQuery": [
    "Application![System[(Level=1 or Level=2 or Level=3 or Level=4 or Level=0 or Level=5)]]",
    "System![System[(Level=1 or Level=2 or Level=3 or Level=4 or Level=0 or Level=5)]]",
    "Security![System[(band(Keywords,13510798882111488))]]"
  ]
},
  "DataSourceWindowsFirewallLog": null,
  "Description": null,
  "DestinationAzureMonitorMetricName": null,
  "DestinationEventHub": null,
  "DestinationEventHubDirect": null,
  "DestinationLogAnalytic": [
    {
      "Name": "la--1400436005",
      "WorkspaceId": "3bb12aee-4a44-46b1-8065-e1e91c8de8e5",
      "WorkspaceResourceId":
"/subscriptions/e97fe0a4-5209-4bae-85a3-39d05c820b28/resourceGroups/rg-stonegate-soc-prod/providers/Microsoft.OperationalInsights/workspaces/law-stonegate-soc"
    }
  ],
  "DestinationMonitoringAccount": null,
  "DestinationStorageAccount": null,
  "DestinationStorageBlobsDirect": null,
  "DestinationStorageTablesDirect": null,
  "Etag": "\"e80093fb-0000-3000-0000-6a3fff7b0000\"",
  "Id": "/subscriptions/e97fe0a4-5209-4bae-85a3-39d05c820b28/resourceGroups/rg-stonegate-soc-prod/providers/Microsoft.Insights/dataCollectionRules/dcr-soc-lab",
  "IdentityPrincipalId": null,
  "IdentityTenantId": null,
  "IdentityType": null,
  "IdentityUserAssignedIdentity": {},
  "ImmutableId": "dcr-df4dbdd4f01e4f16a7238c48fcb397d6",
  "Kind": null,
}
```

**Figure 59:** Scrolled continuation: DestinationLogAnalytic confirms the correct workspace binding to law-stonegate-soc.

```

"/subscriptions/e97fe0a4-5209-4bae-85a3-39d05c820b28/resourceGroups/rg-stonegate-soc-prod/providers/Microsoft.OperationalInsights/workspaces/law-stonegate-soc"
}
},
"DestinationMonitoringAccount": null,
"DestinationStorageAccount": null,
"DestinationStorageBlobsDirect": null,
"DestinationStorageTablesDirect": null,
"ETag": "\"e80093fb-0000-3000-0000-6a3fff7b0000\"",
"Id": "/subscriptions/e97fe0a4-5209-4bae-85a3-39d05c820b28/resourceGroups/rg-stonegate-soc-prod/providers/Microsoft.Insights/dataCollectionRules/dcr-soc-lab",
"IdentityPrincipalId": null,
"IdentityTenantId": null,
"IdentityType": null,
"IdentityUserAssignedIdentity": {},
"ImmutableId": "dcr-df4dbdd4f01e4f16a7238c48fcb397d6",
"Kind": null,
"Location": "southafricanorth",
"MetadataProvisionedBy": null,
"MetadataProvisionedByResourceId": null,
"Name": "dcr-soc-lab",
"ProvisioningState": "Succeeded",
"ResourceGroupName": "rg-stonegate-soc-prod",
"StreamDeclaration": {},
"SystemDataCreatedAt": "2026-06-27T07:31:36.3668176Z",
"SystemDataCreatedBy": "letsoaramojalefa1@outlook.com",
"SystemDataCreatedByType": "User",
"SystemDataLastModifiedAt": "2026-06-27T16:51:04.4990575Z",
"SystemDataLastModifiedBy": "letsoaramojalefa1@outlook.com",
"SystemDataLastModifiedByType": "User",
"Tag": {},
"Type": "Microsoft.Insights/dataCollectionRules"
}
PS /home/mojalefa>

```

**Figure 60:** Final scroll of the verification output: ProvisioningState: Succeeded, ResourceGroupName: rg-stonegate-soc-prod, tags intact, SystemDataLastModifiedAt timestamp confirming the update was applied.

## 8.5 Detection Validation — Real Telemetry, Finally

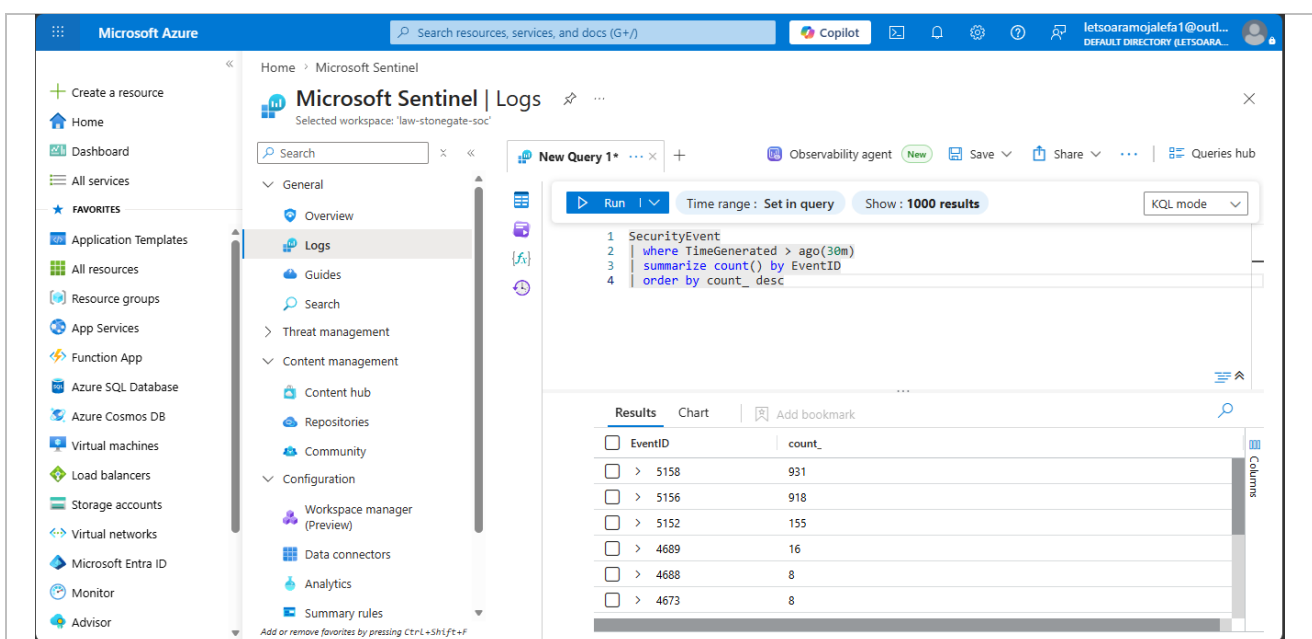
Within minutes of the corrected DCR taking effect, a broad EventID summary query confirmed that real Windows Security Event telemetry was flowing into the SecurityEvent table for the first time - 38 distinct EventIDs dominated by Windows Filtering Platform connection events (5158, 5156, 5152) and privilege/process auditing events (4689, 4688, 4673).

### Validation query:

```

SecurityEvent
| where TimeGenerated > ago(30m)
| summarize count() by EventID
| order by count_desc

```

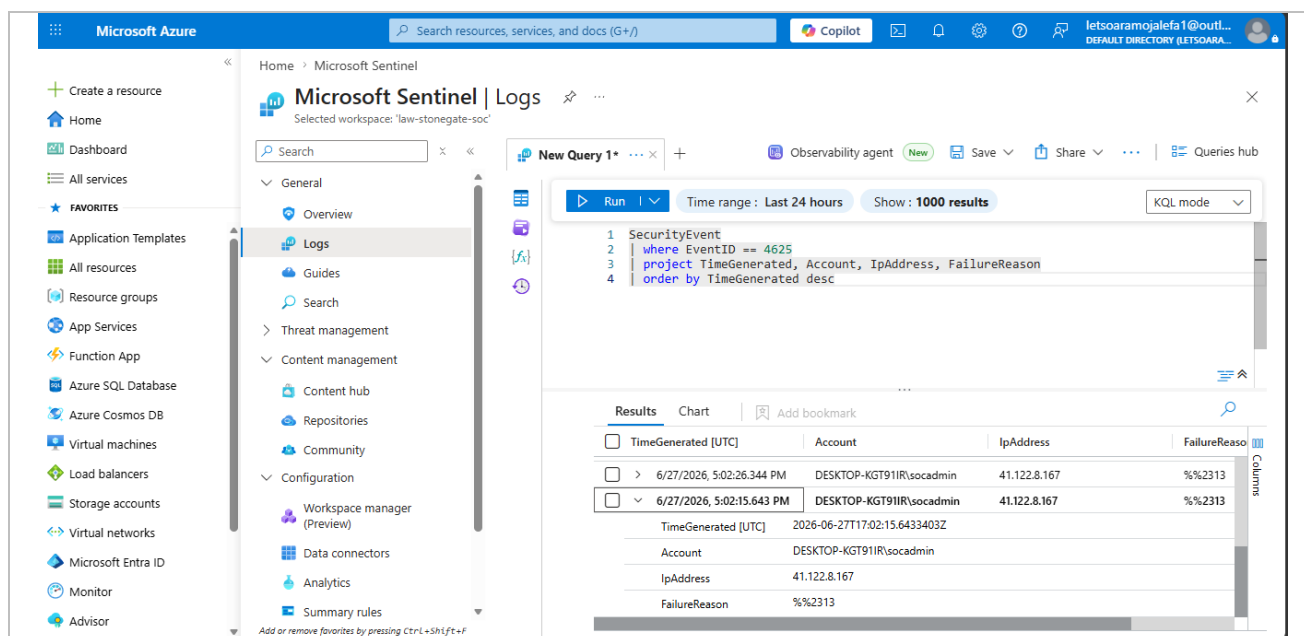


**Figure 61:** SecurityEvent table populated for the first time: 38 unique EventIDs returned, led by EventID 5158 (931 occurrences) and 5156 (918 occurrences) - Windows Filtering Platform connection auditing.

A targeted query for EventID 4625 (failed login) confirmed the specific signal this project was built to detect: repeated failed RDP authentication attempts against the DESKTOP-KGT91IR\socadmin account, originating from IP 41.122.8.167, each tagged with failure reason code %2313 (unknown username or bad password).

### Failed login query:

```
SecurityEvent
| where EventID == 4625
| project TimeGenerated, Account, IpAddress, FailureReason
| order by TimeGenerated desc
```

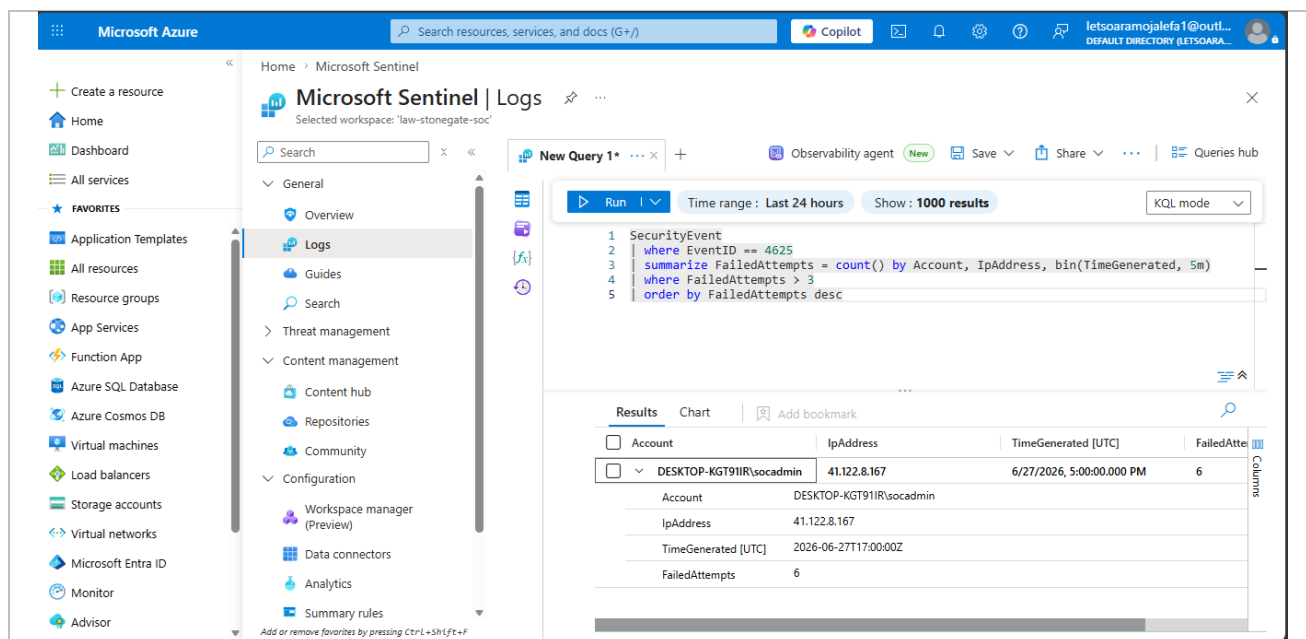


**Figure 62:** EventID 4625 results: six failed login attempts in rapid succession from DESKTOP-KGT91IR\socadmin at 41.122.8.167, each with FailureReason %2313.

Finally, the exact detection logic that would later become the production analytics rule was prototyped and validated directly in Sentinel Logs, confirming the brute-force pattern (more than three failed attempts from the same account/IP within a 5-minute bin) was both detectable and correctly counted.

### Brute-force detection prototype:

```
SecurityEvent
| where EventID == 4625
| summarize FailedAttempts = count() by Account, IpAddress, bin(TimeGenerated, 5m)
| where FailedAttempts > 3
| order by FailedAttempts desc
```



**Figure 63:** Brute-force detection query result: DESKTOP-KGT91IR\socadmin from 41.122.8.167 recorded 6 FailedAttempts within a single 5-minute bin at 17:00 UTC - a textbook brute-force signature and direct proof the detection logic works before it was ever wired into a formal analytics rule.

## Phase 9 — Analytics Rule Engineering

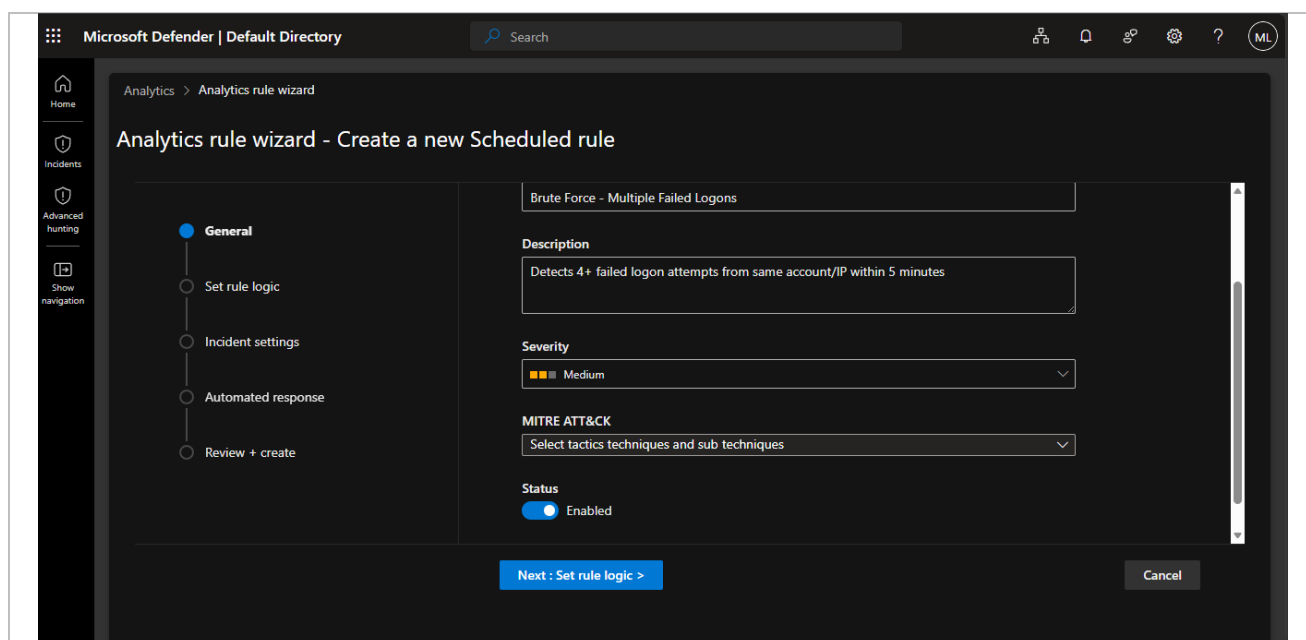
**Goal:** Convert the validated KQL detection logic from Phase 8 into a production Scheduled Analytics Rule in Microsoft Sentinel, mapped to the MITRE ATT&CK framework with correctly configured entity mapping and incident creation.

**Tools:** Microsoft Defender portal (Analytics rule wizard), MITRE ATT&CK framework.

**Outcome:** A Scheduled rule named Brute Force - Multiple Failed Logons was created, mapped to MITRE ATT&CK T1110 (Brute Force) under the Credential Access tactic with Account and IP entity mapping. An initial scheduling misconfiguration (a 5-minute lookback window that was too narrow to reliably catch the 5-minute detection bin) was diagnosed and corrected to a 30-minute lookback, after which the rule began firing reliably.

### 9.1 General Rule Configuration

The rule was named descriptively; Brute Force - Multiple Failed Logons, with a Medium severity rating, appropriate for a controlled lab simulation rather than a confirmed external threat and Status set to Enabled from creation.



**Figure 64:** Analytics rule wizard, General tab: name, description (“Detects 4+ failed logon attempts from same account/IP within 5 minutes”), Medium severity and Enabled status.

### 9.2 MITRE ATT&CK Mapping

Every analytics rule in Sentinel can and should be tagged against the MITRE ATT&CK framework, which provides a standardised taxonomy of adversary tactics and techniques. This rule was mapped to the Credential Access tactic and specifically technique T1110 (Brute Force), this is the correct classification for repeated authentication attempts aimed at guessing valid credentials.

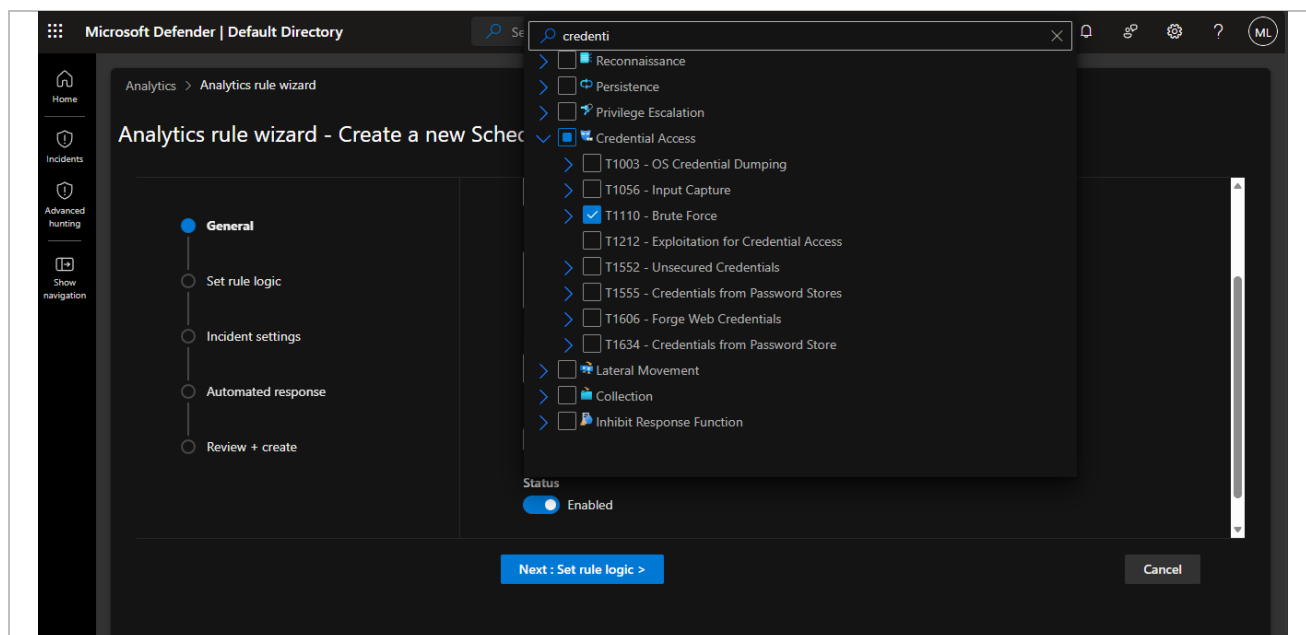


Figure 65: MITRE ATT&CK technique selector: searching “credential access” surfaces T1110 - Brute Force, which is selected and checked.

### 9.3 Rule Logic — The Detection Query

The exact KQL prototype validated in Phase 8 was carried over directly into the rule's query box, ensuring the production detection logic matched what had already been proven against real data.

**Rule query:**

```
SecurityEvent
| where EventID == 4625
| summarize FailedAttempts = count() by Account, IpAddress, bin(TimeGenerated, 5m)
| where FailedAttempts > 3
```

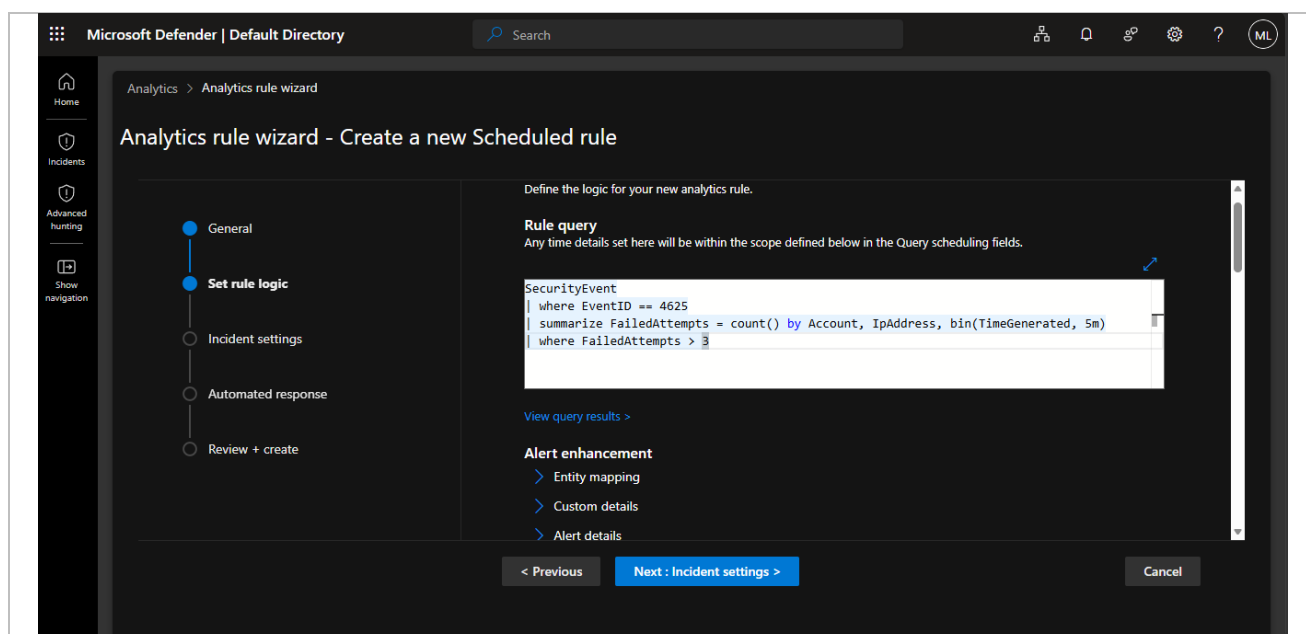
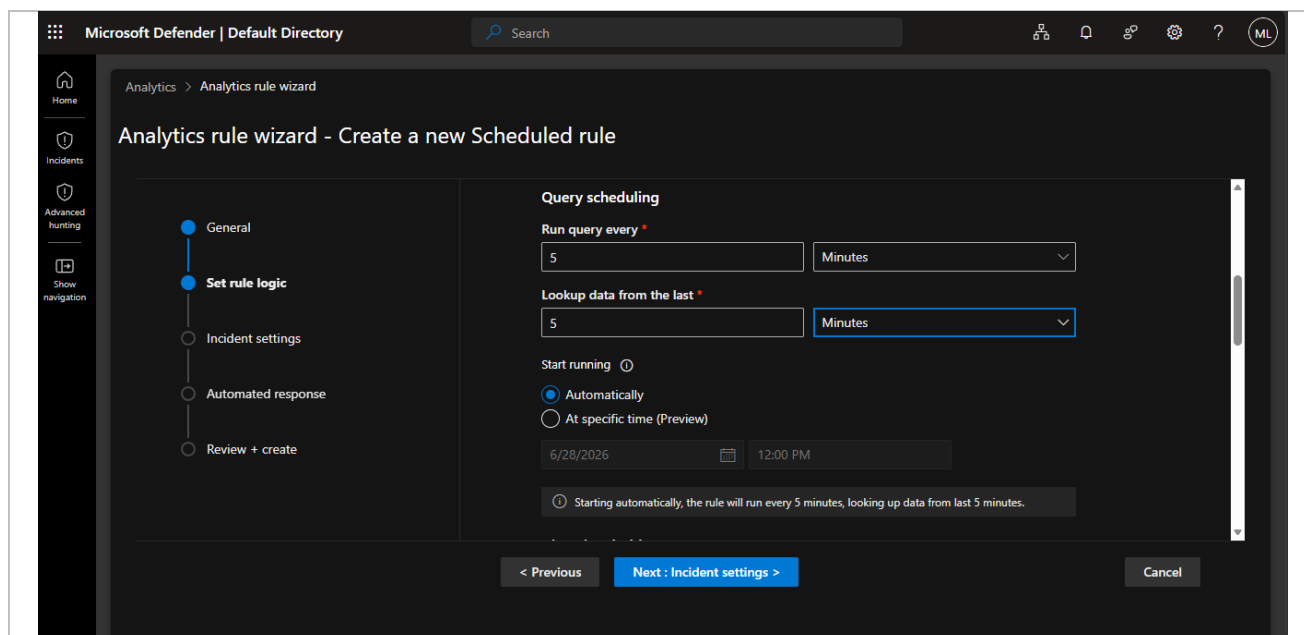


Figure 66: Set rule logic tab: the brute-force detection KQL pasted directly into the Rule query editor.

### 9.4 The Scheduling Bug — A 5-Minute Window That Was Too Narrow

The first configuration of the rule's Query scheduling set both Run query every and Lookup data from the last to 5 minutes. On paper this seemed reasonable, the detection logic itself bins failed attempts into 5-minute windows but in practice created a race condition: if the rule's own execution cycle landed even slightly out of phase with when the failed logons were generated, the 5-minute lookback would not reliably contain a complete 5-minute bin of attack data and the rule would silently fail to fire despite the underlying data being present.

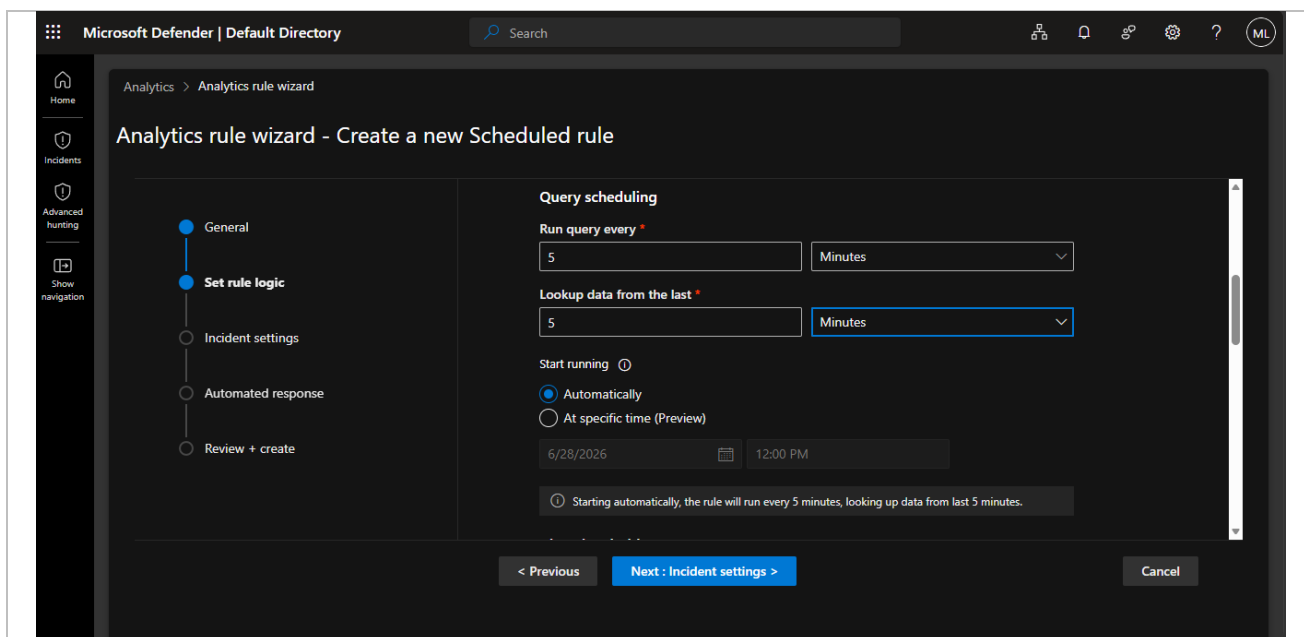


**Figure 67:** Query scheduling, first attempt: Run query every 5 Minutes, Lookup data from the last 5 Minutes – This is the configuration that initially prevented the rule from firing.

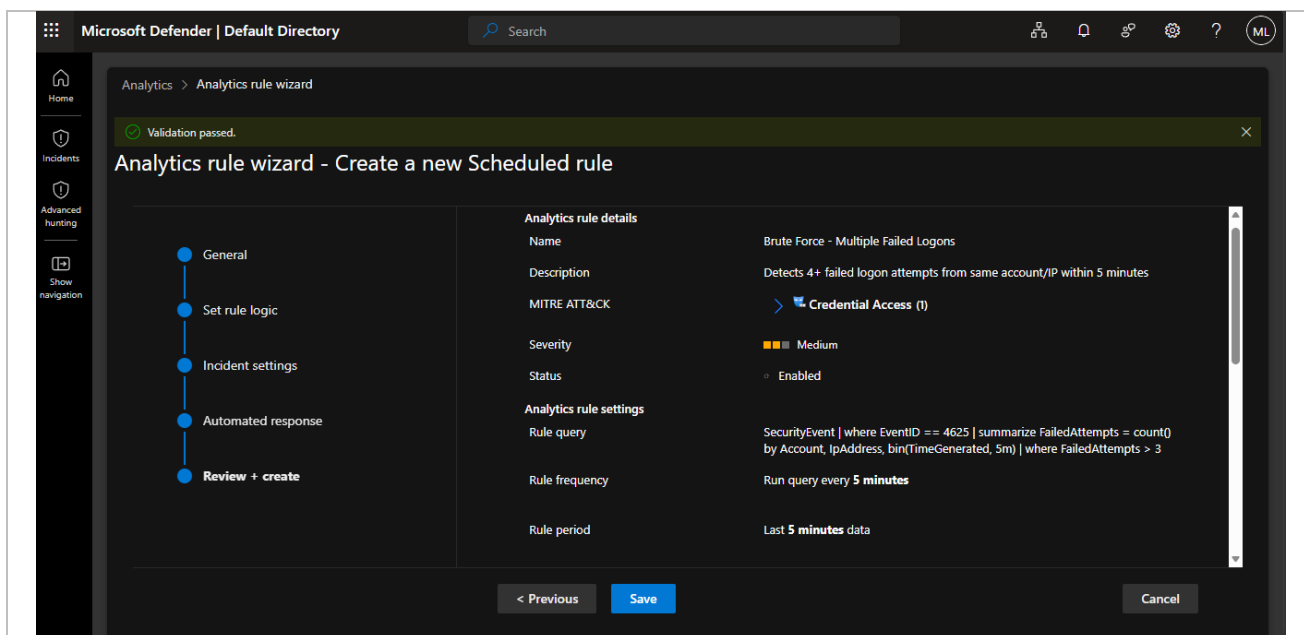
This is a subtle but important lesson: a rule's lookback window must be generously wider than its own detection bin size or genuine attack telemetry can fall just outside the query's scan window purely due to execution timing. The fix was to widen the lookback to 30 minutes while keeping the 5-minute execution cadence, giving the rule six full opportunities to catch any 5-minute brute-force bin that occurred in the preceding half hour.

## 9.5 Entity Mapping & Incident Settings

Two entities were mapped so that Sentinel's incident graph and investigation tooling would correctly recognise the Account and IP fields returned by the query as first-class, clickable entities, these are essential for any downstream investigation, UEBA correlation or automation.



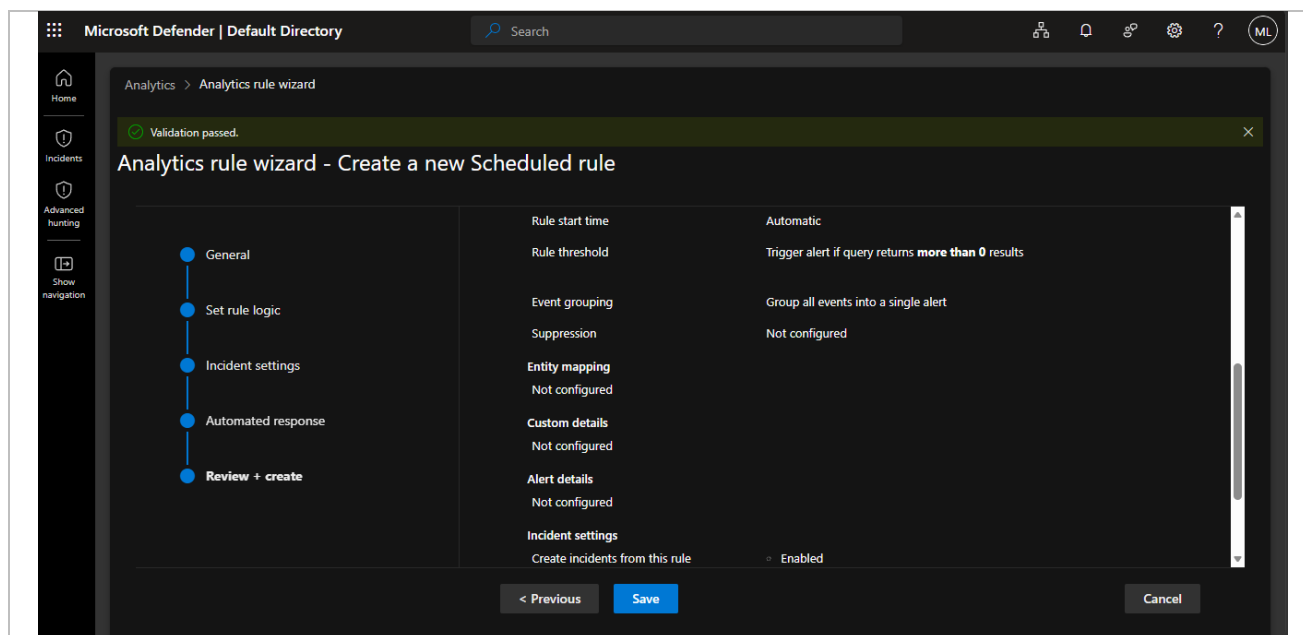
**Figure 68:** Entity mapping: Account entity bound to Identifier 'Name' > query column Account; IP entity bound to Identifier 'Address' > query column IpAddress.



**Figure 69:** Review pane confirming entity mapping (Account — Name/Account, IP — Address/IpAddress), Create incidents from this rule: Enabled and Alert grouping: Disabled.

## 9.6 Re-validating the General Tab

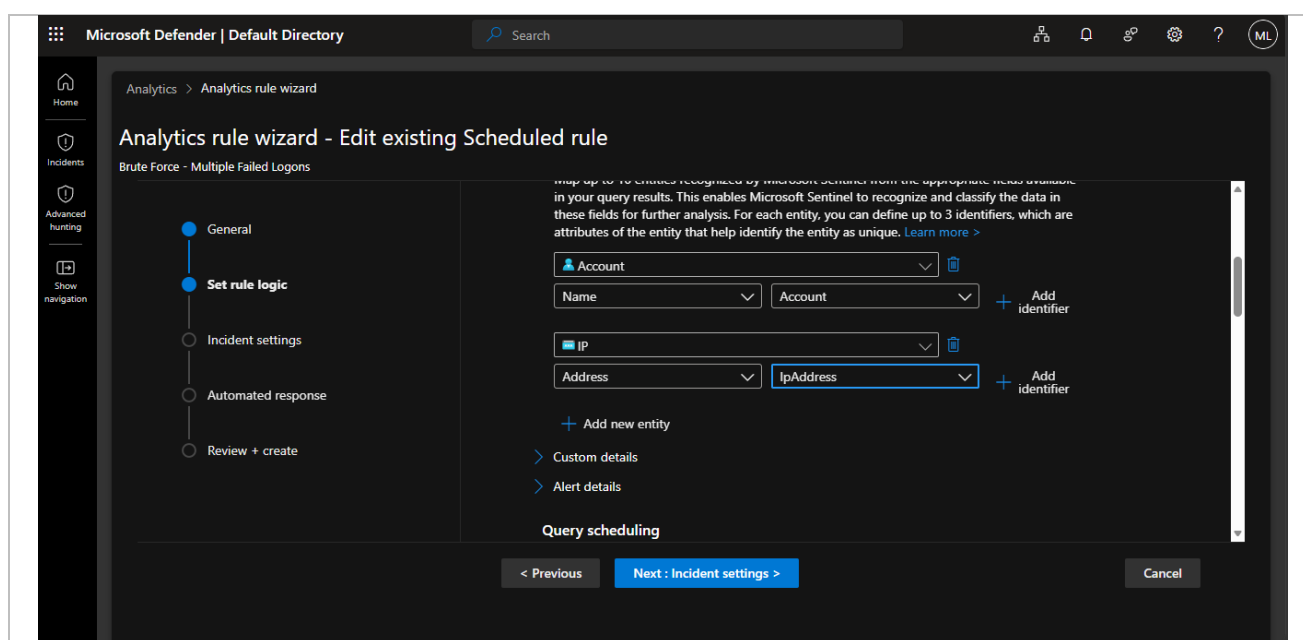
During the edit pass to correct the scheduling window, the General tab was revisited to confirm the rule's name, description, severity and MITRE mapping selector remained correctly set before proceeding back through the wizard.



**Figure 70:** General tab revisited during the edit-and-fix pass: name, description, Medium severity and MITRE ATT&CK selector all confirmed unchanged.

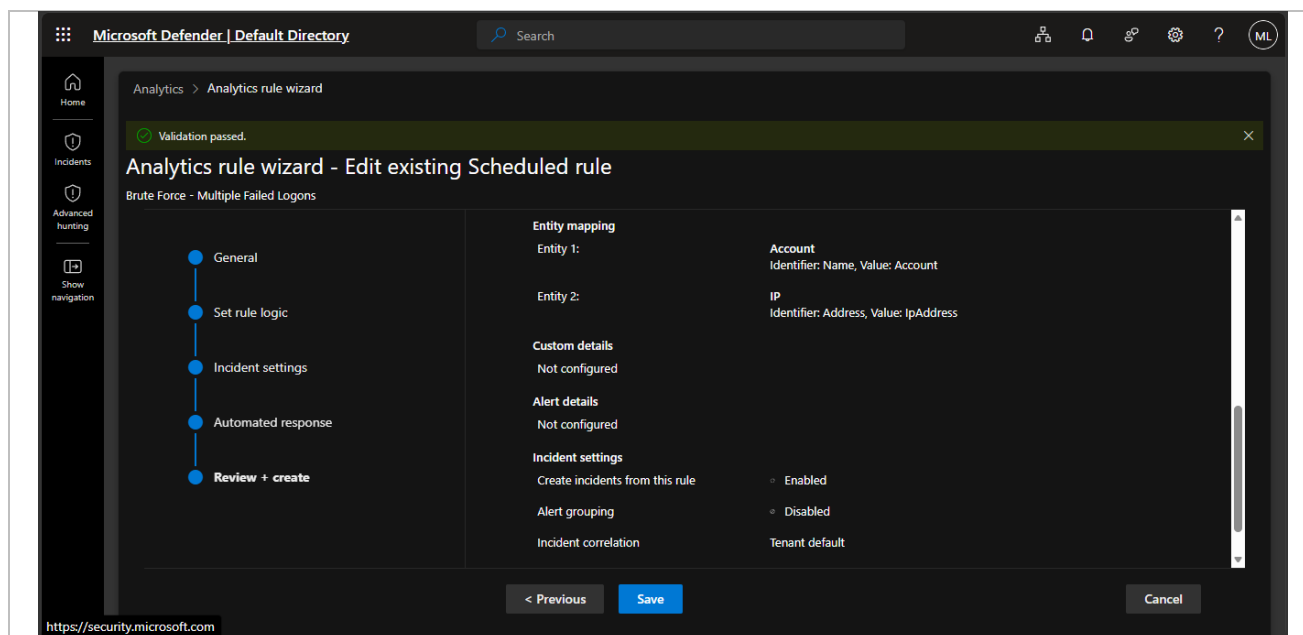
## 9.7 Review + Create — Before and After the Fix

The Review + create summary screen makes the scheduling configuration explicit in plain language. The first review pass clearly shows the problem: “Rule period: Last 5 minutes data” - confirming in the wizard's own words the configuration that would go on to prevent reliable detection.



**Figure 71:** Review + create, validation passed but Rule period still reads “Last 5 minutes’ data” - misconfiguration visible in plain text before the fix was applied.

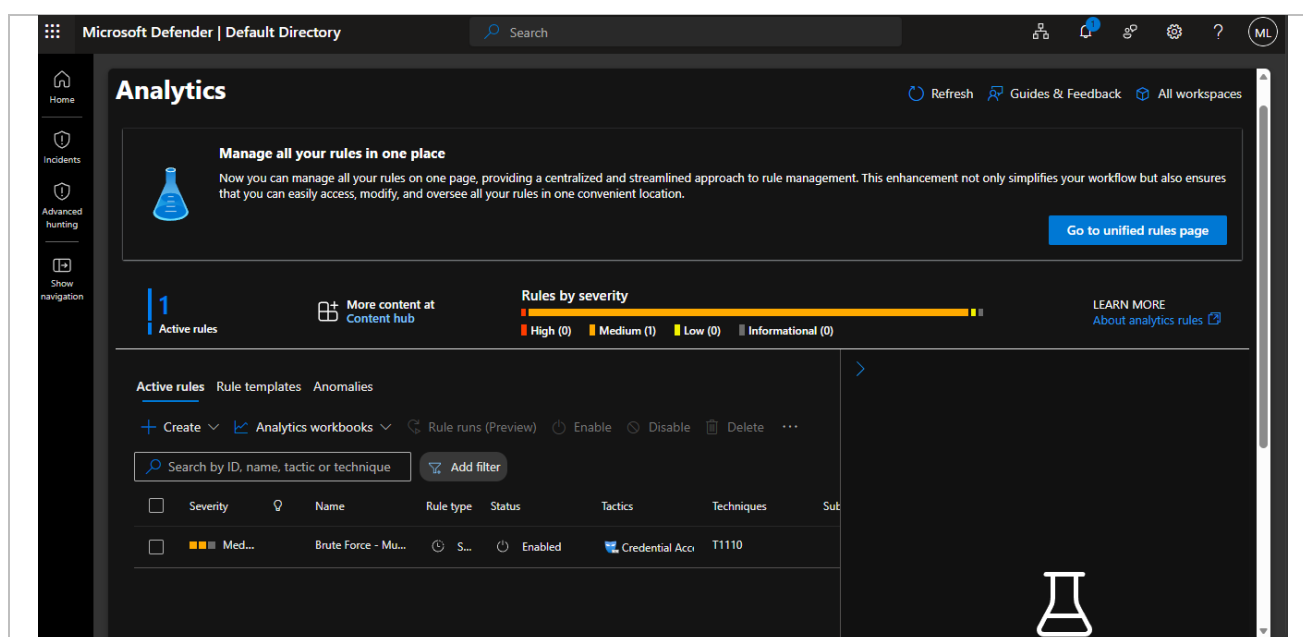
After widening the lookback window, the same review screen now confirms the corrected configuration: a 5-minute execution cadence against a 30-minute data lookback.



**Figure 72:** Corrected Query scheduling: Run query every 5 Minutes, Lookup data from the last 30 Minutes - the fix that allowed the rule to begin firing reliably.

## 9.8 The Rule Goes Live

Once saved with the corrected scheduling, the rule appeared in the Analytics blade as a single Active rule with Medium severity, tagged to the Credential Access tactic and technique T1110, now ready to begin evaluating incoming SecurityEvent data on its five-minute cycle.

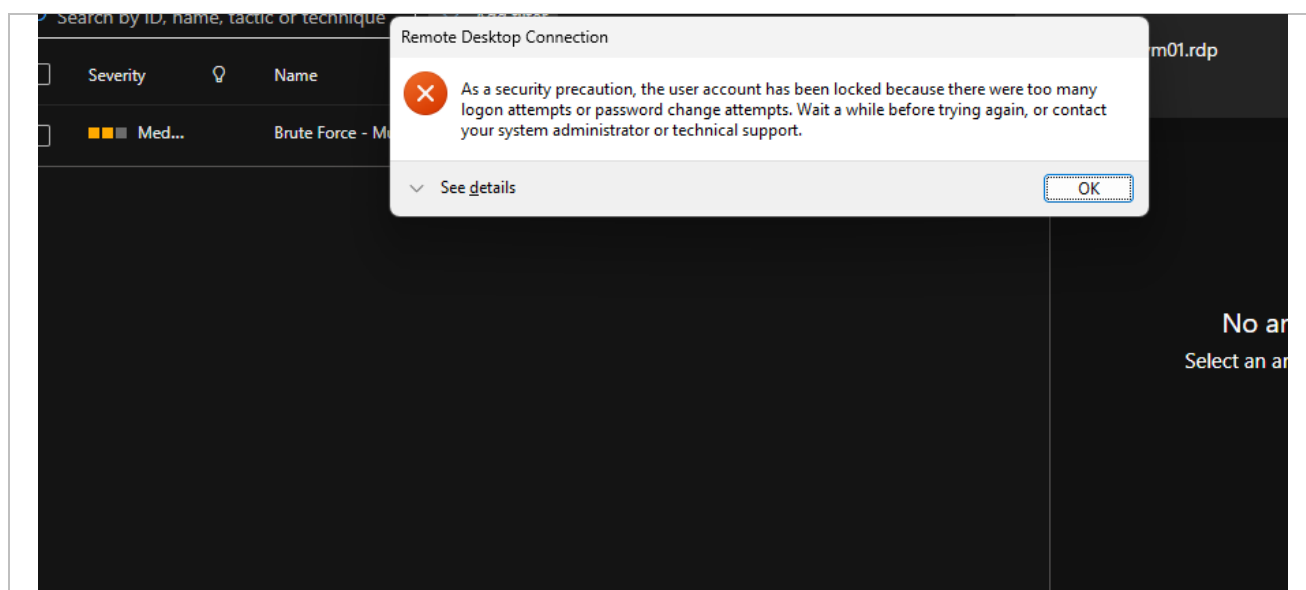


**Figure 73:** Microsoft Sentinel Analytics blade: 1 Active rule — Brute Force - Multiple Failed Logons — Medium severity, Credential Access tactic, technique T1110, Status: Enabled.

## 9.9 The Brute-Force Simulation Succeeds — Account Lockout

To generate fresh, unambiguous attack telemetry for the rule to detect, repeated incorrect-password RDP login attempts were deliberately made against the “socadmin” account. This simulation was in fact, too successful: Windows' own account lockout policy triggered after the threshold number of failed attempts was exceeded, locking the account as a built-in defensive measure. This in itself is a

meaningful secondary signal (EventID 4740, account lockout) that a real SOC would also want to alert on.



**Figure 74:** Remote Desktop Connection: “As a security precaution, the user account has been locked because there were too many logon attempts or password change attempts” – This is direct evidence that the brute-force simulation generated a real, measurable security response from the operating system itself.

## Phase 10 — Incident Response & Closure

**Goal:** Triage, investigate and formally close every incident generated by the brute-force analytics rule, following a structured SOC Level 1 incident response workflow consistent with real-world security operations practice.

**Tools:** Microsoft Defender portal (Incidents queue, Manage incident, Manage alert, bulk incident actions).

**Outcome:** The analytics rule generated 18 incidents over the simulation period. Each was investigated using the incident graph and attack story view, then progressed through “assignment, In Progress status, analyst commentary, true-positive classification and resolution” - first individually to demonstrate the full workflow, then in bulk to efficiently close the remaining queue.

### 10.1 The Incident Queue

With the corrected analytics rule running on its five-minute cycle against a continuously regenerated stream of failed-logon telemetry, incidents began accumulating in the Defender portal's Incidents queue. By the time formal triage began, 13 incidents were observed that later grew to 18 had been auto-generated, each titled Brute Force - Multiple Failed Logons, each carrying Medium severity and the Credential Access category inherited from the underlying analytics rule.

| Incident name                        | Incident Id | Priority score | Tags | Severity | Investigation state | Categories        |
|--------------------------------------|-------------|----------------|------|----------|---------------------|-------------------|
| Brute Force - Multiple Failed Logons | 13          | Not set        |      | Medium   | Not set             | Credential access |
| Brute Force - Multiple Failed Logons | 12          | 3              |      | Medium   | Not set             | Credential access |
| Brute Force - Multiple Failed Logons | 11          | 3              |      | Medium   | Not set             | Credential access |
| Brute Force - Multiple Failed Logons | 10          | 3              |      | Medium   | Not set             | Credential access |
| Brute Force - Multiple Failed Logons | 9           | 3              |      | Medium   | Not set             | Credential access |
| Brute Force - Multiple Failed Logons | 8           | 3              |      | Medium   | Not set             | Credential access |
| Brute Force - Multiple Failed Logons | 7           | 3              |      | Medium   | Not set             | Credential access |
| Brute Force - Multiple Failed Logons | 6           | 3              |      | Medium   | Not set             | Credential access |
| Brute Force - Multiple Failed Logons | 5           | 3              |      | Medium   | Not set             | Credential access |
| Brute Force - Multiple Failed Logons | 4           | 3              |      | Medium   | Not set             | Credential access |
| Brute Force - Multiple Failed Logons | 3           | 3              |      | Medium   | Not set             | Credential access |

**Figure 75:** Incidents queue showing 13 auto-generated incidents, each titled Brute Force - Multiple Failed Logons, Medium severity, Credential Access category.

### 10.2 First-Pass Incident Detail

Selecting an individual incident surfaces its detail panel: priority assessment (not yet set on a freshly created incident, pending automatic prioritisation), assignment status, classification and the precise first-activity and creation timestamps tying the incident back to the underlying alert.

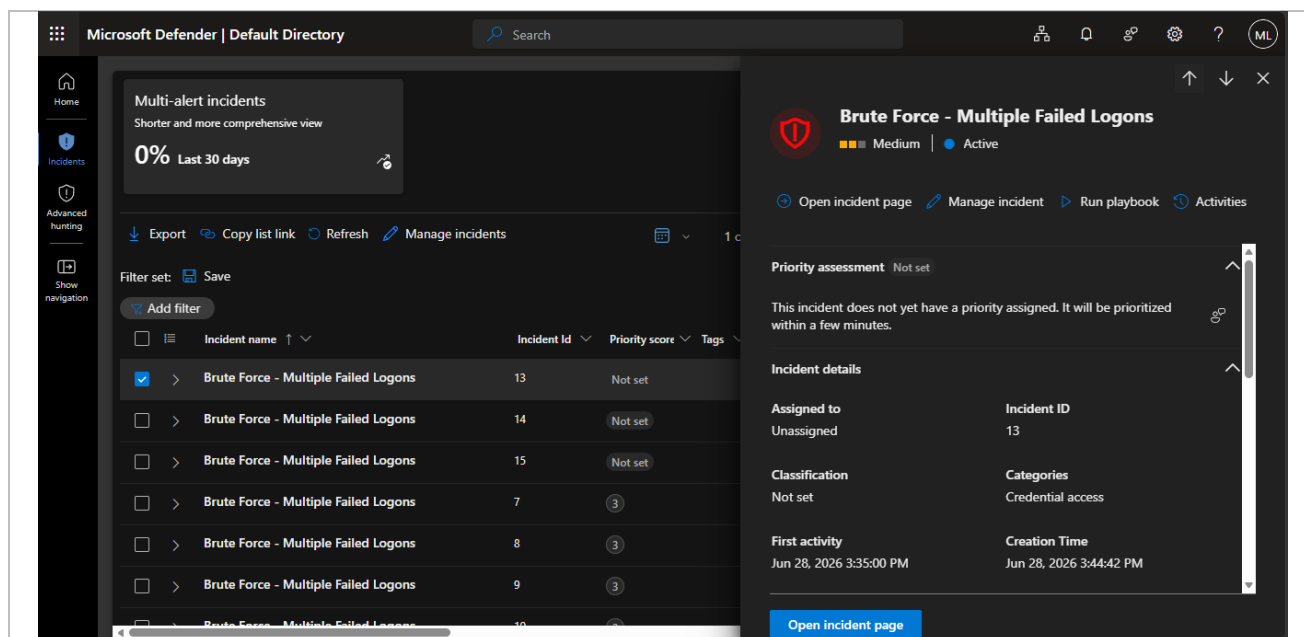


Figure 76: Incident #13 detail pane: Unassigned, Unclassified, Credential access category, first activity timestamped to the exact moment the brute-force telemetry occurred.

### 10.3 The Attack Story — Incident Graph

Opening the full incident page reveals the Attack story view, including an interactive incident graph that visually connects the affected account (DESKTOP-KGT91IR\socadmin) to the source IP address (41.123.68.123) via a Communication/Association edge. This is exactly the kind of entity-relationship visualisation that the entity mapping configured in Phase 9 makes possible. This is precisely the investigative view a SOC analyst would use to understand at a glance what happened and which assets were involved.

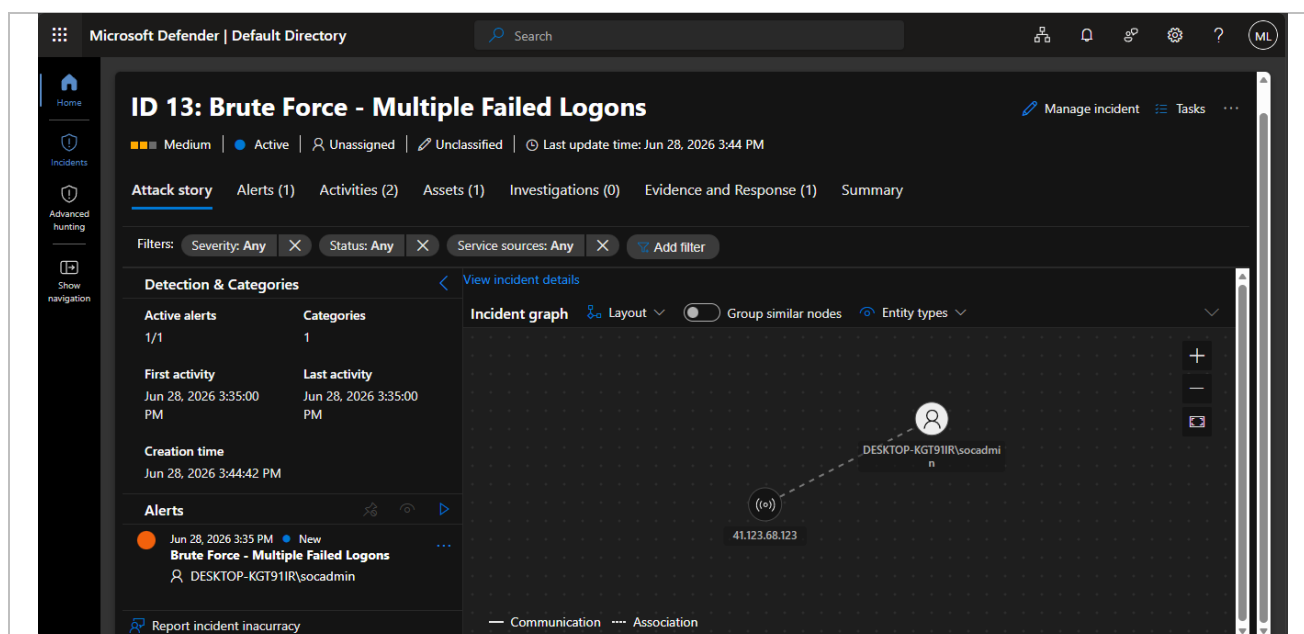
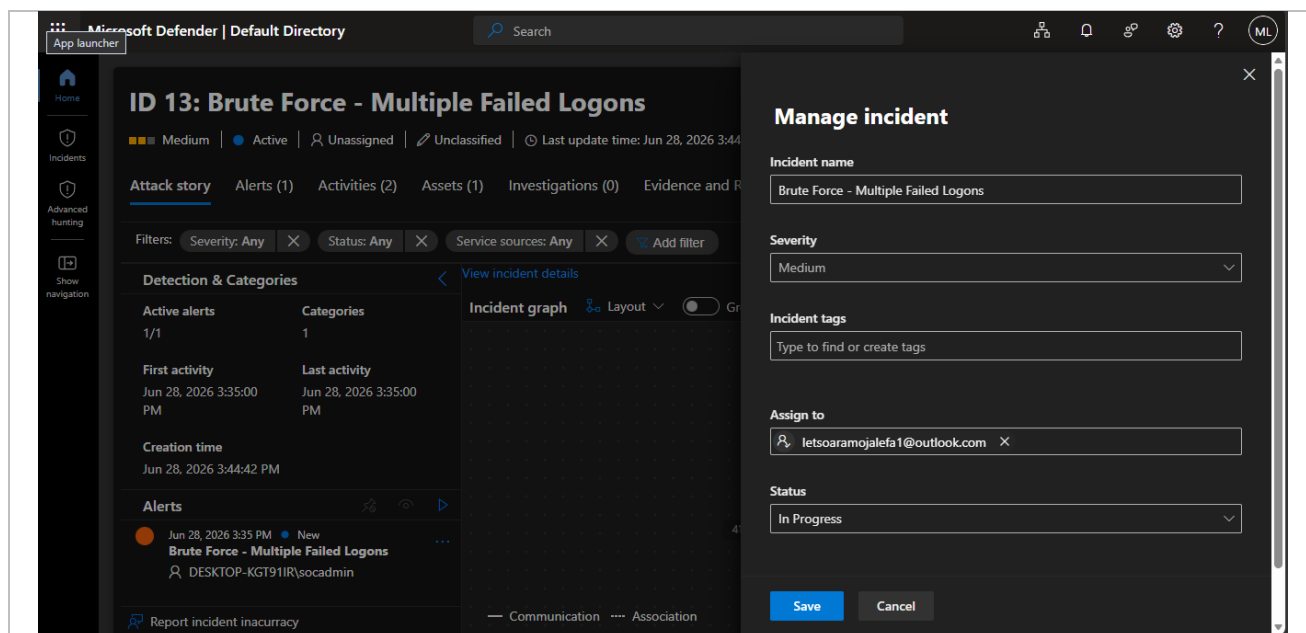


Figure 77: Attack story view: incident graph showing the DESKTOP-KGT91IR\socadmin account node connected to the 41.123.68.123 IP node, with full Detection & Categories metadata in the left panel.

### 10.4 Triage Step 1 — Assignment & Status

Following standard SOC Level 1 practice, the incident was first assigned to the analyst (letsoaramojalefa1@outlook.com) and its status was progressed from New to In Progress, signalling that active investigation had begun.



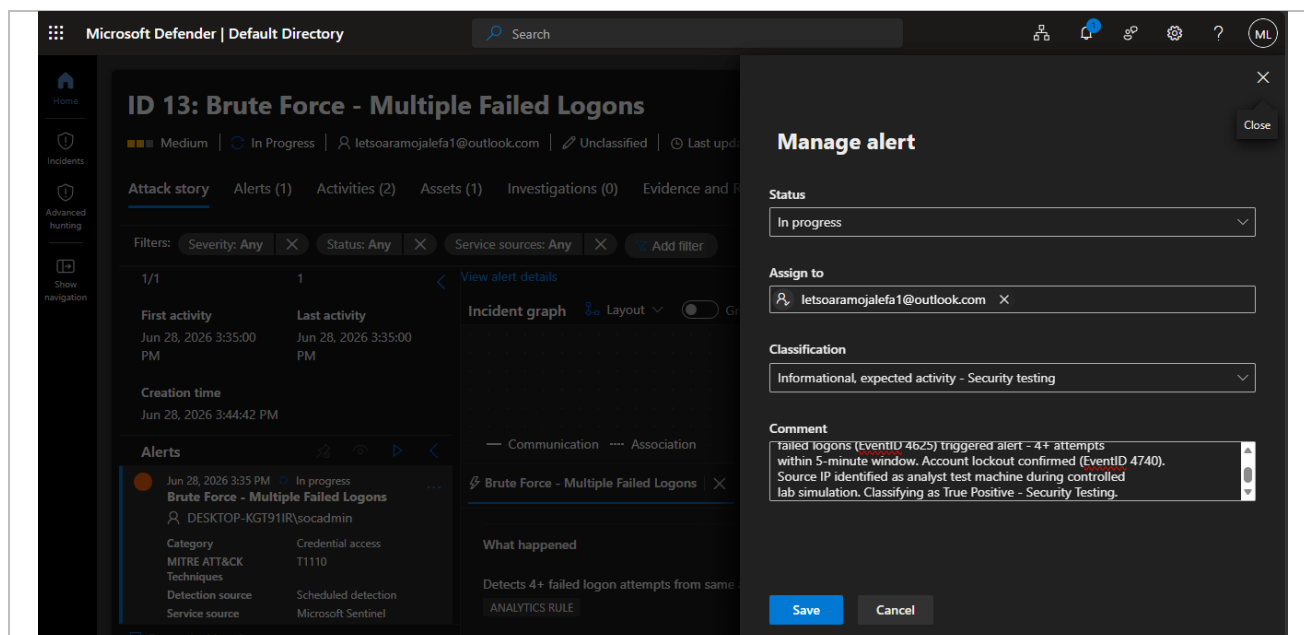
**Figure 78:** Manage incident panel: Assign to set to letsoaramojalefa1@outlook.com, Status changed to In Progress.

## 10.5 Triage Step 2 — Classification & Analyst Commentary

At the alert level, the incident was classified as Informational, expected activity with a determination of Security testing which is the correct classification for a deliberately engineered, controlled lab simulation rather than a genuine external threat. A detailed analyst comment was logged, documenting exactly what was observed, which EventIDs were involved and the reasoning behind the classification mirroring the documentation discipline expected of any real SOC analyst's case notes.

### *Analyst comment logged on the incident:*

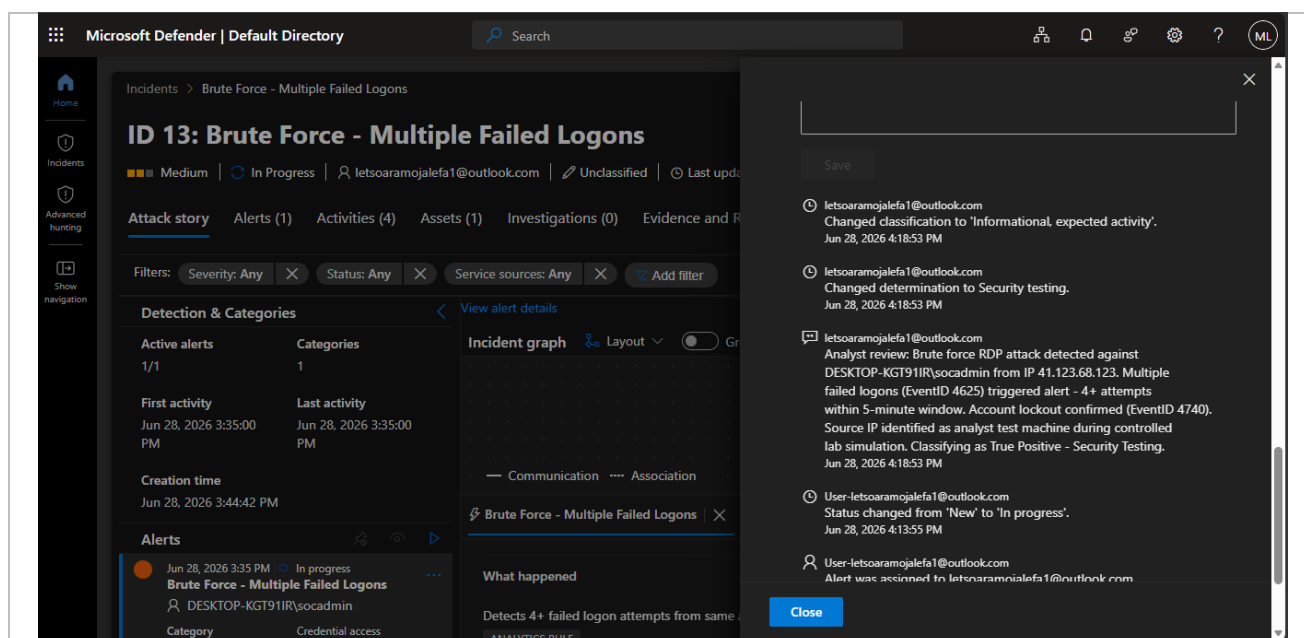
Analyst review: Brute force RDP attack detected against DESKTOP-KGT91IR\socadmin from IP 41.123.68.123. Multiple failed logons (EventID 4625) triggered alert — 4+ attempts within 5-minute window. Account lockout confirmed (EventID 4740). Source IP identified as analyst test machine during controlled lab simulation. Classifying as True Positive — Security Testing.



**Figure 79:** Manage alert panel: Status In progress, Classification set to Informational, expected activity - Security testing with the full analyst comment entered in the Comment field.

## 10.6 Full Audit Trail

Every action taken against the incident - the classification change, the determination update, the analyst's free-text comment, the assignment and the status transition is automatically timestamped and retained in the Comments & history panel. This produces a complete, non-repudiable audit trail of the investigation which is exactly the kind of evidence a compliance or audit review would expect to see attached to any closed security incident.



**Figure 80:** Comments & history panel showing the full chronological audit trail: classification change, determination change, the analyst's detailed comment and the status transition from New to In progress, each attributed and timestamped.

## 10.7 Triage Step 3 — Resolution

With investigation and documentation complete, the incident was formally resolved. Microsoft Defender requires an explicit justification when resolving an incident, reinforcing accountability: the analyst must state in their own words why the incident is being closed.

### Resolution explanation provided:

Controlled lab simulation — brute force test conducted by analyst

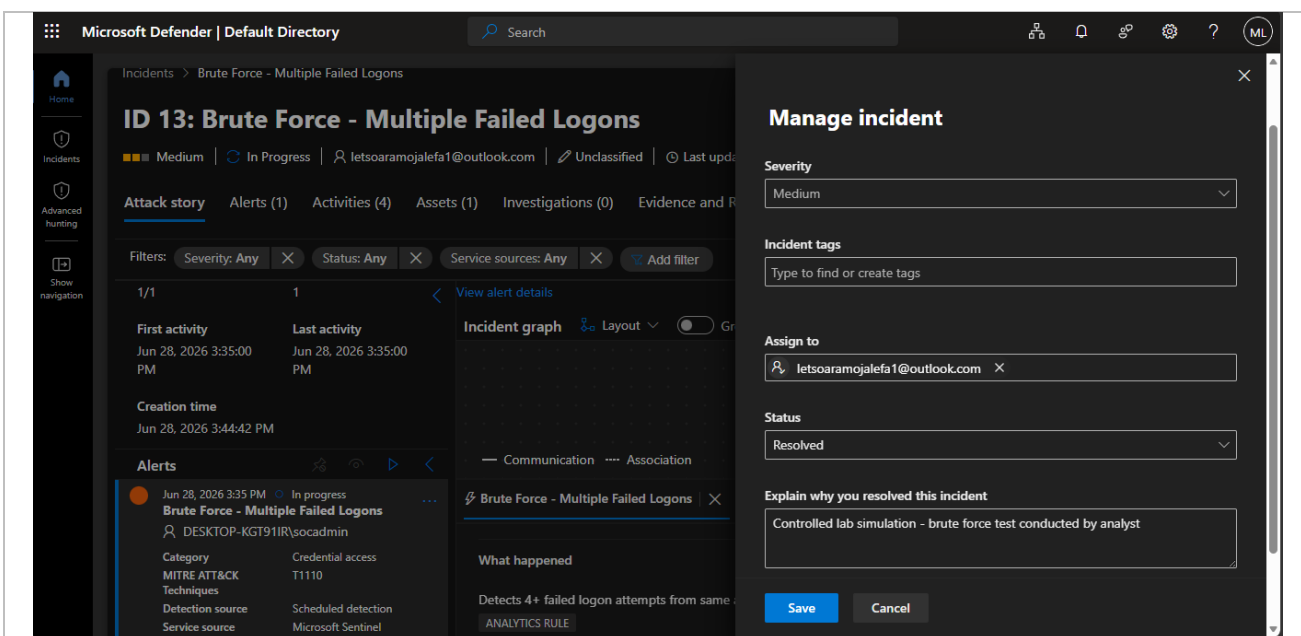
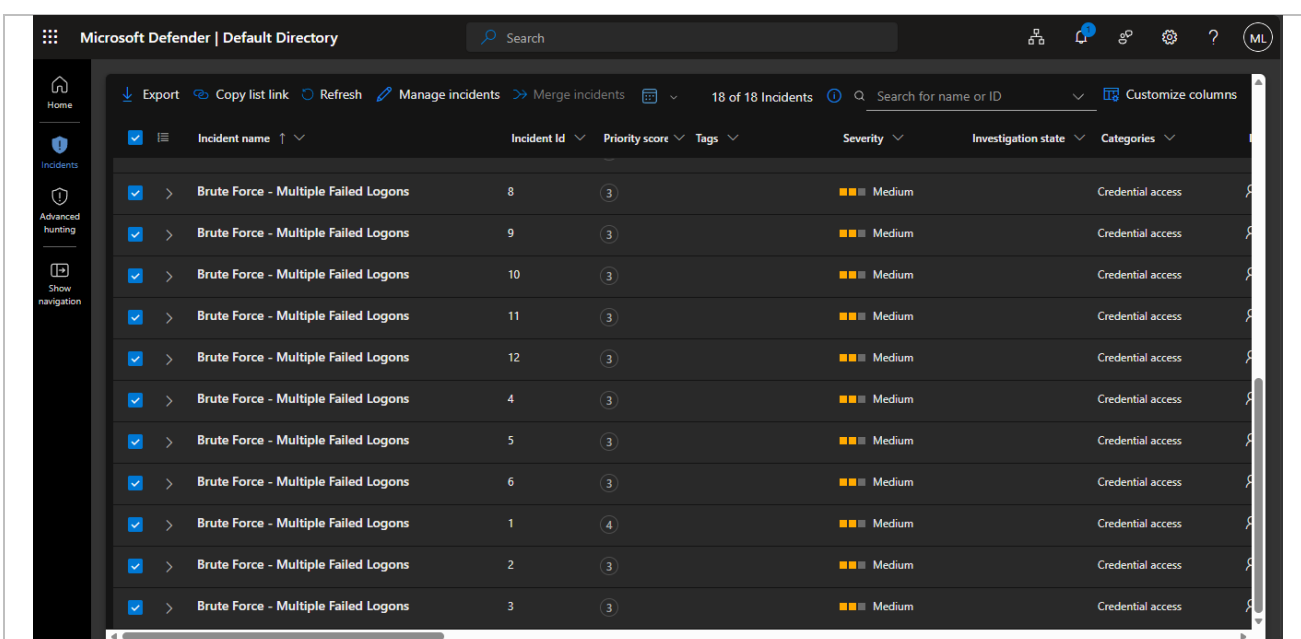


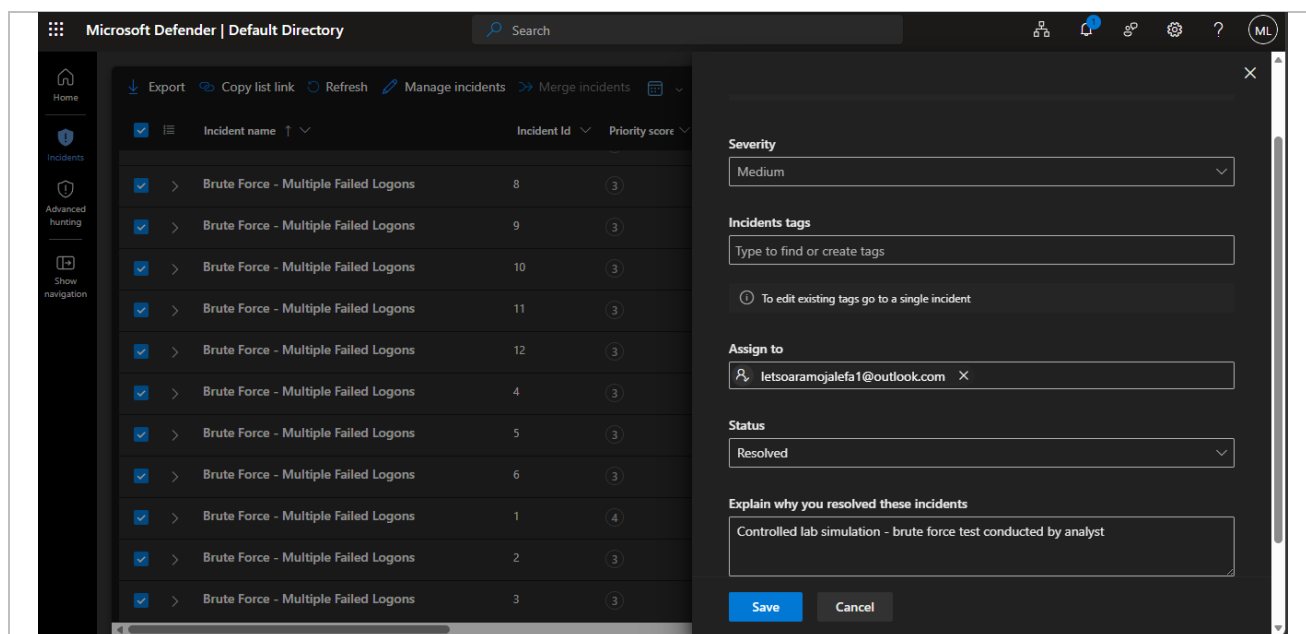
Figure 81: Manage incident panel: Status changed to Resolved with the required explanation - “Controlled lab simulation - brute force test conducted by analyst” entered before saving.

## 10.8 Scaling Up — Bulk Incident Management

Having demonstrated the full individual triage workflow end-to-end on the first incident, the remaining incidents in the queue, which had grown to 18 in total as the analytics rule continued firing on its five-minute cycle were closed efficiently using Microsoft Defender's bulk incident management capability selecting multiple incidents simultaneously and applying the same classification, assignment and resolution in a single action.



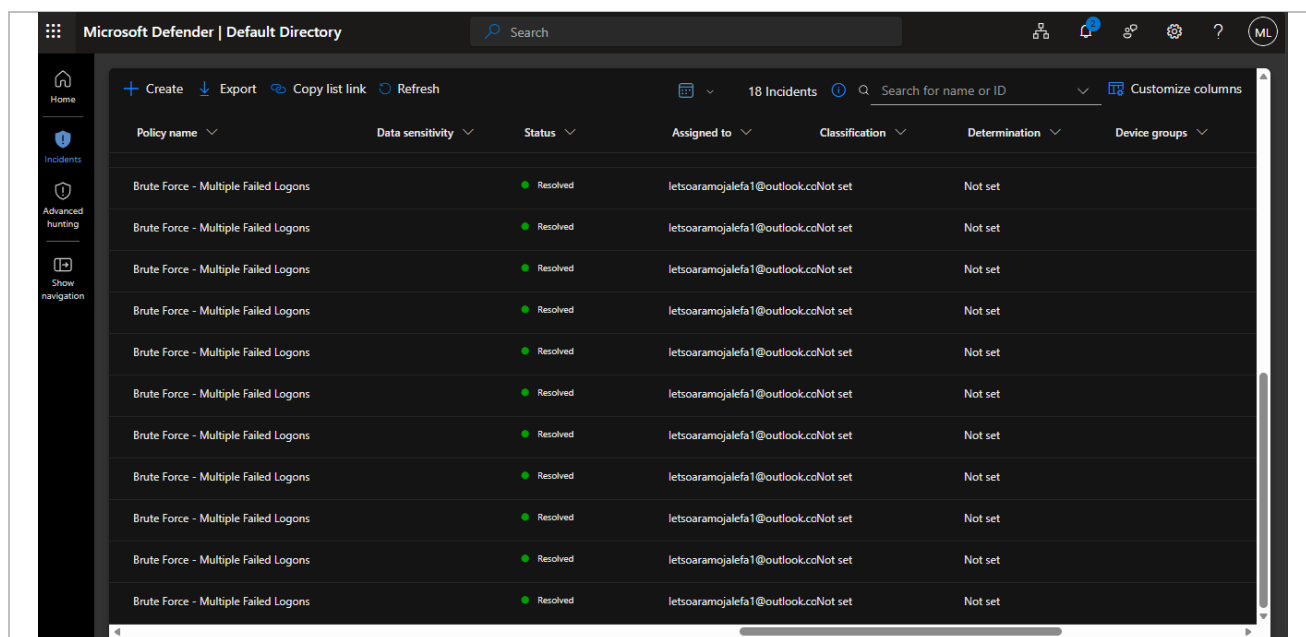
**Figure 82:** Bulk selection: all remaining Brute Force - Multiple Failed Logons incidents checked simultaneously in the Incidents queue, out of 18 total incidents.



**Figure 83:** Bulk Manage incidents panel: Severity Medium, Assign to letsoaramojalefa1@outlook.com, Status Resolved, with the same lab-simulation explanation applied across every selected incident in one operation.

## 10.9 Final State — A Clean, Fully Resolved Queue

The end result is an incident queue with all 18 incidents in a Resolved state, each correctly attributed to the analyst, each carrying a consistent and defensible classification. This represents a complete, evidence-backed demonstration of the full SOC Level 1 lifecycle: detection, alerting, triage, investigation, documentation and closure repeated reliably across an entire incident queue, not just a single cherry-picked example.



**Figure 84:** Final incident queue state: all 18 incidents showing Status: Resolved, Assigned to: letsoaramojalefa1@outlook.com, Determination: Not set (informational classification applied at the alert level), confirming a fully closed and documented SOC queue.

## SC-200 Exam Coverage

This project was deliberately scoped to exercise practical, hands-on skills directly aligned with the Microsoft SC-200 (Security Operations Analyst Associate) certification syllabus. The table below maps specific project activities to the corresponding SC-200 exam domains.

| SC-200 Domain                               | Project Activity Demonstrating Competency                                                                                                              |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mitigate threats using Microsoft Sentinel   | Built law-stonegate-soc from scratch; onboarded Sentinel; configured data connectors and the Content Hub solution install workflow                     |
| Configure protections / detections          | Authored a custom Scheduled Analytics Rule with KQL detection logic, MITRE ATT&CK mapping, and entity mapping                                          |
| Manage incident response                    | Full incident lifecycle: assignment, status progression, analyst commentary, classification, resolution - individually and in bulk across 18 incidents |
| Kusto Query Language (KQL)                  | Authored and validated multiple KQL queries: ingestion checks, Heartbeat validation, EventID filtering and time-binned brute-force aggregation         |
| Data Collection Rules & Azure Monitor Agent | Diagnosed and corrected a real DCR stream misconfiguration using PowerShell; installed and validated AMA via Azure CLI                                 |
| MITRE ATT&CK framework                      | Mapped the brute-force detection to Credential Access / T1110 with rationale documented in the analytics rule                                          |
| Workbooks & Content Hub                     | Installed the Windows Security Events solution; deployed six workbooks covering authentication, identity and network telemetry                         |
| Azure RBAC for security operations          | Diagnosed and resolved a workspace permissions gap by assigning the Log Analytics Contributor role                                                     |

## Lessons Learned & Recommendations

---

### Lessons Learned

- A green provisioning status does not guarantee a working pipeline. The DCR reported “Provisioning succeeded” from the moment it was first created and continued to do so throughout the entire period it was silently misrouting data to the wrong table. Provisioning state confirms a resource was created; it says nothing about whether its internal logic is correct. The only reliable verification is querying the destination table itself.
- Portal UI can hide configuration that PowerShell and JSON expose directly. The Stream property mismatch between DataFlow and DataSourceWindowsEventLog was never visible anywhere in the Azure Portal's DCR editing experience. It only became visible by inspecting the raw resource definition via `Get-AzDataCollectionRule | ConvertTo-Json`. Comfort with the underlying CLI and resource model and not just the portal is essential for real troubleshooting.
- A detection rule's lookback window must be wider than its own detection bin. Setting both the execution frequency and the lookback window to an identical 5 minutes created a timing race condition that silently prevented the rule from firing, despite the underlying data and query logic being completely correct. Lookback windows should generously exceed the bin size used inside the query itself.
- Windows Security event logging requires explicit Local Security Policy configuration. Out of the box, Windows Server does not record meaningful Security audit events. Without enabling Success/Failure auditing across all nine audit categories, the entire downstream pipeline however correctly configured, would have had no source data to detect in the first place.
- Default Azure VM deployment behaviour can create resource group sprawl. Deploying a VM without explicitly targeting an existing resource group results in Azure auto-generating a new one. This is easy to overlook and directly caused the first AMA installation failure since the CLI command targeted the project's main resource group rather than the VM's actual one.

### Recommendations for a Production Deployment

1. Adopt Infrastructure as Code (Bicep or Terraform) for DCR definitions. Defining the DCR's stream bindings declaratively in version-controlled code would have made the Microsoft-Event vs Microsoft-SecurityEvent mismatch visible at review time, rather than requiring live production troubleshooting to uncover.
2. Build a standing “pipeline health” workbook that continuously validates Heartbeat, SecurityEvent and AzureActivity ingestion volumes alerting if any expected table goes quiet turning the manual verification process used throughout this project into an always-on monitoring control.
3. Extend the analytics rule library beyond brute force to cover privilege escalation (EventID 4672), account lockouts (EventID 4740) and object access anomalies (EventID 4663), all of which were observed flowing into the workspace during this project but not yet formally alerted on.
4. Introduce a SOAR playbook to automate the repetitive elements of Phase 10's triage workflow, automatic assignment, initial classification and analyst notification - freeing analyst time for genuine investigation rather than administrative incident handling.
5. Apply least-privilege RBAC reviews on a recurring basis. The Log Analytics Contributor role was assigned reactively, mid-project to resolve a permissions gap; a production environment should define and audit required roles as part of initial environment design, not as a troubleshooting afterthought.

## Analyst Profile & Certification Roadmap

This project was undertaken by Mojalefa Letsoara as direct, hands-on preparation for the Microsoft SC-200 (Security Operations Analyst Associate) certification, with the SC-300 (Identity and Access Administrator Associate) certification scheduled for the end of July 2026 as the next milestone in a broader identity-and-security-operations career path.

The project was deliberately designed to go beyond passive exam study, instead building and breaking a real Azure SOC environment end-to-end: provisioning infrastructure, diagnosing a genuine production-style misconfiguration without external assistance beyond standard troubleshooting tools, engineering a working detection rule from first principles and carrying that detection through to a fully documented, closed incident queue.

| Certification                              | Status                        | Relevance to This Project                                                                                                   |
|--------------------------------------------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| SC-200 — Security Operations Analyst       | Candidate, actively preparing | Direct subject matter: Sentinel, KQL, DCRs, AMA, incident response, MITRE ATT&CK                                            |
| SC-300 — Identity and Access Administrator | Scheduled end of July 2026    | Complementary identity-security skill set; builds on the RBAC and Entra ID Sign-in log workbooks introduced in this project |

## Appendix — KQL Reference Library

The complete set of KQL queries authored and validated during this project is consolidated below as a quick-reference library, in the order they were used through the project lifecycle.

### A.1 Ingestion & Pipeline Health

#### *Baseline workspace check (any table):*

```
search *  
| where TimeGenerated > ago(1h)  
| summarize Count=count() by $table
```

#### *Agent connectivity (Heartbeat):*

```
Heartbeat  
| summarize Count = count() by Computer
```

#### *Broad SecurityEvent ingestion check:*

```
SecurityEvent  
| where TimeGenerated > ago(30m)  
| summarize count() by EventID  
| order by count_desc
```

### A.2 Authentication & Brute-Force Detection

#### *Failed logons (EventID 4625):*

```
SecurityEvent  
| where EventID == 4625  
| project TimeGenerated, Account, IPAddress, FailureReason  
| order by TimeGenerated desc
```

#### *Successful logons (EventID 4624):*

```
SecurityEvent  
| where EventID == 4624  
| project TimeGenerated, Account, LogonType, IPAddress  
| order by TimeGenerated desc
```

#### *Brute-force aggregation (production analytics rule query):*

```
SecurityEvent  
| where EventID == 4625  
| summarize FailedAttempts = count() by Account, IPAddress, bin(TimeGenerated, 5m)  
| where FailedAttempts > 3  
| order by FailedAttempts desc
```

## A.3 Privilege Use & Object Access (Identified for Future Detections)

### *Privilege escalation (EventID 4672):*

```
SecurityEvent
| where EventID == 4672
| project TimeGenerated, Computer, Account, Privileges
| order by TimeGenerated desc
```

### *Object / file access (EventID 4663):*

```
SecurityEvent
| where EventID == 4663
| project TimeGenerated, Account, ObjectName, AccessMask
| order by TimeGenerated desc
```

## A.4 Azure Activity & Diagnostic Validation

### *Activity log ingestion check:*

```
AzureActivity
| take 10
```

---

## End of Report

Stonegate SOC - Microsoft Sentinel Incident Response Simulation  
Mojalefa L. Letsoara